



UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE INGENIERÍA MECÁNICA

IMPLEMENTACIÓN DE DETECCIÓN DE OBJETOS PARA LA IDENTIFICACIÓN DE FALLAS EN CORREAS TRANSPORTADORAS

MEMORIA PARA OPTAR AL TÍTULO DE INGENIERO CIVIL MECÁNICO

FERNANDO IGNACIO NAVARRETE URRUTIA

PROFESORA GUÍA:
VIVIANA MERUANE NARANJO

PROFESOR CO-GUÍA:
HAROLD VALENZUELA COLOMA

COMISIÓN:
MATÍAS LASEN

SANTIAGO DE CHILE

2026

RESUMEN EJECUTIVO DE LA MEMORIA PARA
OPTAR AL TÍTULO DE INGENIERO CIVIL MECÁNICO
POR: FERNANDO IGNACIO NAVARRETE URRUTIA
FECHA: 2026
PROF. GUÍA: VIVIANA MERUANE NARANJO
PROF. CO-GUÍA: HAROLD VALENZUELA COLOMA

IMPLEMENTACIÓN DE DETECCIÓN DE OBJETOS PARA LA IDENTIFICACIÓN DE FALLAS EN CORREAS TRANSPORTADORAS

La detección temprana de fallas en correas transportadoras es fundamental para evitar paradas no programadas y pérdidas operativas en industrias críticas. Los métodos tradicionales basados en inspección visual presentan limitaciones en velocidad, costo y precisión. En contraste, las técnicas de aprendizaje profundo, especialmente aquellas basadas en redes neuronales convolucionales (CNN) y arquitecturas *Transformer*, ofrecen soluciones automatizadas y escalables para la clasificación de fallas mediante visión computacional, favoreciendo el mantenimiento predictivo y la supervisión continua.

Este trabajo de memoria de título se enfoca en implementar modelos de detección de fallas en correas transportadoras mediante técnicas de visión computacional y aprendizaje profundo. El objetivo general es implementar y evaluar algoritmos de detección de objetos basados en inteligencia artificial. Entre los objetivos específicos destacan la implementación y entrenamiento con bases de datos etiquetadas, la programación de modelos basados en CNN (YOLO y SSD) y modelos *Transformer* (DETR y DINO). Por último, se realiza una evaluación comparativa mediante métricas estándar de visión por computadora.

La metodología contempla la implementación y entrenamiento de modelos en un entorno virtual, utilizando bases de datos etiquetadas y particionadas en entrenamiento, validación y prueba. Los resultados posicionan al modelo YOLOv5 como la arquitectura más eficiente y de mayor viabilidad industrial, superando a SSD512 y a los modelos de atención global. Los detectores basados en *Transformers* demostraron ventajas al aislar defectos complejos como perforaciones, pero el desempeño general se vio penalizado por la escala y etiquetado de los datos. Se propone como trabajo futuro la exploración de arquitecturas híbridas y una optimización de hiperparámetros. La estructura de trabajo, registros de métricas y detalles se encuentran almacenados en un repositorio de código abierto (*Implementation of Object Detection for Conveyor Belt Defect Detection*).

En la elaboración de este trabajo de memoria de título se utilizaron herramientas de inteligencia artificial generativa, en particular Google Gemini. Estas fueron empleadas en las Secciones 2, 3 y 6 en la generación de diagramas, código y tablas. Todo el contenido fue revisado y validado por el autor, quien asume plena responsabilidad sobre el trabajo presentado.

Agradecimientos

Agradezco por sobre todo a mi querida familia, quienes fueron testigos y contribuyentes en la mayor parte de mi desarrollo personal y profesional; por enseñarme lo correcto, lo incorrecto y entregarme las herramientas básicas para afrontar el peso del mundo, las consecuencias de mis decisiones y la admiración por todos los detalles que me recuerdan cada día el mundo donde vivo, existo y pertenezco.

Otro agradecimiento a todos aquellos que me acompañaron y me siguen acompañando durante mi proceso educativo: profesores, compañeros, colegas, conocidos y colaboradores. Bien es sabido que el conocimiento y la ciencia se construyen entre todos nosotros que hemos pisado un aula y quienes son transversales al proceso.

Por último, a mis mascotas, en especial a mi perro y a mi gato. Ambos me siguen acompañando en diferentes etapas de mi crecimiento; mientras que mi perro me acompañó desde que era niño, mi gato recién llegado, lo hizo desde que comencé la especialidad en ingeniería mecánica. Atribuyo un cariño especial al gato, el cual siempre me está visitando cuando trabajo, recordándome que debo alimentarlo.

Tabla de Contenido

1. Introducción	1
1.1. Objetivo General	2
1.2. Objetivos Específicos	2
1.3. Alcances	3
2. Antecedentes	4
2.1. Estado del arte en la identificación de fallas en correas transportadoras . . .	4
2.1.1. Fallas en correas transportadoras	4
2.1.2. Aprendizaje profundo (<i>Deep Learning</i>)	5
2.2. Detección de objetos en visión por computadora	6
2.3. Redes Neuronales Convolucionales (CNN)	8
2.3.1. Contextualización	8
2.3.2. Arquitectura	9
2.3.3. Hiperparámetros en modelos de detección basados en CNN	10
2.4. Mecanismo de atención y arquitectura Encoder-Decoder	12
2.4.1. Mecanismo de atención en Transformers	12
2.4.2. Arquitectura Encoder-Decoder	13
2.5. Modelos de detección de objetos de una sola etapa	15
2.5.1. Contextualización	15
2.5.2. Selección de modelos	16
2.5.3. YOLO (You Only Look Once)	17
2.5.3.1. Arquitectura del modelo YOLOv5	18
2.5.3.2. Bloques convolucionales: backbone, neck y head	19
2.5.3.3. Funciones de activación y funciones de salida en YOLOv5	20
2.5.3.4. Diferencias clave de YOLOv5 respecto a versiones anteriores	21
2.5.4. SSD (Single Shot Multibox Detector)	21
2.5.4.1. Arquitectura del modelo SSD	22
2.5.4.2. Bloques convolucionales: backbone, capas adicionales y cabezas de predicción	22
2.5.4.3. Funciones de activación y funciones de salida en SSD	24
2.6. Modelos de detección de objetos basados en Transformers	25
2.6.1. Contextualización	25

2.6.2.	Arquitectura general	25
2.6.3.	DETR (DEtection TRansformer)	26
2.6.3.1.	Arquitectura general de DETR	26
2.6.3.2.	Bloques fundamentales: Backbone, Encoder, Decoder y FFN	27
2.6.3.3.	Pérdida de emparejamiento bipartito y el Algoritmo Húngaro	27
2.6.4.	DINO (DETR with Improved DeNoising Anchor Boxes)	28
2.6.4.1.	Evolución y arquitectura general de DINO	28
2.6.4.2.	Eliminación de ruido contrastiva (<i>Contrastive DeNoising</i>)	29
2.6.4.3.	Selección mixta de consultas (<i>mixed query selection</i>)	30
2.6.4.4.	Proyección anticipada doble (<i>look forward twice</i>)	30
2.6.5.	Hiperparámetros en modelos de detección basados en Transformers	31
2.6.6.	Selección de modelos	32
2.7.	Métricas	32
2.7.1.	Precisión, Exhaustividad y F-Score	33
2.7.2.	Intersección sobre la unión (IoU)	33
2.7.3.	Precisión promedio (Average Precision, AP) y precisión media promedio (mean Average Precision, mAP)	34
2.7.4.	Matriz de confusión (Confusion Matrix)	35
3.	Metodología	36
3.1.	Identificación de fallas	36
3.2.	Base de datos	37
3.2.1.	Base de datos elaborada	37
3.2.2.	Recopilación y limpieza	37
3.2.3.	Estructura topológica y formato de anotación	37
3.2.4.	Adaptación de etiquetas	39
3.3.	Entorno virtual y arquitectura de la implementación	40
3.3.1.	Anatomía de la Estructura Maestra	41
3.4.	Implementación de modelos One-Stage (YOLOv5 y SSD)	45
3.5.	Implementación de modelos basados en Transformers (DETR y DINO)	45
4.	Resultados y Discusión	47
4.1.	Base de datos implementada	47
4.2.	Tiempos y rendimiento del entrenamiento	50
4.3.	Métricas y resultados de los modelos implementados	51
4.3.1.	Resumen de resultados	51
4.3.2.	YOLOv5 (<i>You Only Look Once</i>)	52
4.3.3.	SSD (<i>Single Shot Multibox Detector</i>)	60
4.3.4.	DETR (<i>DEtection TRansformer</i>)	68
4.3.5.	DINO (<i>DETR with Improved DeNoising Anchor Boxes</i>)	76
4.4.	Inferencia comparativa entre modelos	84
4.5.	Discusión general de resultados	91

5. Conclusiones	94
Bibliografía	95
6. Anexos	97
6.1. Tablas de Hiperparámetros	97

Índice de Tablas

2.3.1. Hiperparámetros comunes en modelos de detección basados en CNN	11
2.6.1. Hiperparámetros representativos en arquitecturas Transformer	31
2.6.2. Comparativa de rendimiento y complejidad evolutiva para detectores <i>Transformer</i> en MS-COCO. Datos extraídos y consolidados de [16] y [17].	32
2.7.1. Definiciones generales <i>Machine Learning</i> [18]	33
3.4.1. Hiperparámetros base estandarizados para el entrenamiento comparativo de los modelos YOLOv5 y SSD. Ambos incorporan el optimizador de Descenso de Gradiente Estocástico (<i>Stochastic Gradient Descent</i>).	45
3.5.1. Hiperparámetros base estandarizados compartidos por las arquitecturas Transformer (DETR y DINO).	46
4.1.1. Resumen de las propiedades y distribución del conjunto de datos.	47
4.2.1. Rendimiento computacional y estimación de tiempo total de entrenamiento (300 épocas) por variante.	50
4.3.1. Resumen de métricas de rendimiento, complejidad paramétrica y costo computacional para los modelos YOLOv5, SSD, DETR y DINO.	51
6.1.1. Hiperparámetros principales seleccionados para el entrenamiento de YOLOv5.	97
6.1.2. Hiperparámetros principales seleccionados para el entrenamiento de SSD.	98
6.1.3. Hiperparámetros principales seleccionados para el entrenamiento de DETR.	99
6.1.4. Hiperparámetros principales seleccionados para el entrenamiento de DINO.	100

Índice de Ilustraciones

2.1.1. Tipos de falla comunes en correas transportadoras. Imágenes extraídas y adaptadas de [1].	5
2.1.2. Machine Learning vs Deep Learning. Diagrama adaptado de [5].	6
2.2.1. Ejemplos de detección de objetos simples y múltiples. Imagen extraída de [6]. .	7
2.2.2. Ejemplo de detección de fallas utilizando detección de objetos en un caso industrial. Imagen extraída de [3].	8
2.3.1. Arquitectura típica de Redes Neuronales Convolucionales. Diagrama adaptado de [5].	9
2.4.1. Atención escalada por producto punto (izquierda). Atención Multicabezal (derecha). Imagen extraída de [12].	13
2.4.2. Arquitectura del Modelo Transformer. Imagen extraída de [12].	14
2.5.1. Clasificación de varias técnicas de detección de objetos. Imagen extraída de [6].	15
2.5.2. Una revisión del rendimiento de los detectores de una sola etapa. Imagen extraída de [4].	16
2.5.3. Arquitectura de YOLOv5, basada en una estructura de tipo <i>backbone-neck-head</i> . Imagen extraída de [14].	18
2.5.4. Arquitectura general de SSD. Imagen extraída y procesada de [15].	22
2.6.1. Arquitectura general de DETR. Imagen extraída de [16].	27
2.6.2. Arquitectura DINO. Imagen extraída de [17].	29
2.7.1. Representación visual de Intersección sobre la Unión. Imagen extraída de [18].	34
3.2.1. Representación visual de las coordenadas espaciales del vector de formato de etiquetas. Elaboración Propia.	39
3.3.1. Topología modular <i>single-socket</i> implementada. Los orquestadores interpretan las configuraciones y dictan las directrices operativas a la estructura maestra, cargando las Bases de Datos y entregando los resultados requeridos tras pasar por las etapas internas de entrenamiento y validación. Sólo una arquitectura (o rama) se acopla a la vez.	41
3.3.2. Vista detallada (<i>zoom</i>) de la interacción interna. La arquitectura neuronal opera como una caja negra intocable (no se modifica). Los 3 módulos principales se conectan mediante los dos ejecutores principales: Entrenador (<i>Trainer</i>) y Validador (<i>Validator</i>)	42
4.1.1. Distribución de frecuencia de las clases de fallas a través de las particiones del conjunto de datos.	48

4.1.2. Composición detallada de las particiones del conjunto de datos, diferenciando entre imágenes con objetos y casos de fondo.	48
4.1.3. Muestras representativas de la base de datos confeccionada. Se exhiben las 5 fallas y el caso saludable con sus respectivas cajas delimitadoras y conteo de etiquetas.	49
4.3.1. Pérdida total promedio vs época de la validación en cada variante del modelo YOLOv5.	52
4.3.2. Precisión Media Promedio (mAP) vs época con umbral IoU 0,5 para todas las variantes del modelo YOLOv5.	53
4.3.3. Precisión Media Promedio (mAP) vs época con umbral IoU desde 0,5 hasta 0,95 para todas las variantes del modelo YOLOv5.	54
4.3.4. F1-Score vs época para todas las variantes del modelo YOLOv5.	54
4.3.5. Trade-off de rendimiento entre todas las variantes del modelo YOLOv5. Muestra el rendimiento mAP@0.5:0.95 en función de la cantidad de GFlops de cómputo utilizado por la variante.	55
4.3.6. Curvas de F1-Score en función de la confianza (<i>confidence</i>) de detección de la variante <i>small</i> (s) de YOLOv5.	56
4.3.7. Curvas de Precisión en función de la confianza (<i>confidence</i>) de detección de la variante <i>small</i> (s) de YOLOv5.	57
4.3.8. Curvas de Exhaustividad en función de la confianza (<i>confidence</i>) de detección de la variante <i>small</i> (s) de YOLOv5.	58
4.3.9. Matriz de confusión de la variante <i>small</i> (s) de YOLOv5.	59
4.3.10. Pérdida total promedio vs época de la validación en cada variante del modelo SSD.	60
4.3.11. Precisión Media Promedio (mAP) vs época con umbral IoU 0,5 para todas las variantes del modelo SSD.	61
4.3.12. Precisión Media Promedio (mAP) vs época con umbral IoU desde 0,5 hasta 0,95 para todas las variantes del modelo SSD.	61
4.3.13. F1-Score vs época para todas las variantes del modelo SSD.	62
4.3.14. Tradeoff de rendimiento entre todas las variantes del modelo SSD. Muestra el rendimiento mAP@0.5:0.95 en función de la cantidad de GFlops de cómputo utilizado por la variante.	63
4.3.15. Curvas de F1-Score en función de la confianza (<i>confidence</i>) de detección de la variante SSD512.	64
4.3.16. Curvas de Precision en función de la confianza (<i>confidence</i>) de detección de la variante SSD512.	65
4.3.17. Curvas de Exhaustividad en función de la confianza (<i>confidence</i>) de detección de la variante SSD512.	66
4.3.18. Matriz de confusión de la variante SSD512.	67
4.3.19. Pérdida total promedio vs época de la validación en cada variante del modelo DETR.	68

4.3.20. Precisión Media Promedio (mAP) vs época con umbral IoU 0,5 para todas las variantes del modelo DETR.	69
4.3.21. Precisión Media Promedio (mAP) vs época con umbral IoU desde 0,5 hasta 0,95 para todas las variantes del modelo DETR.	70
4.3.22. F1-Score vs época para todas las variantes del modelo DETR.	70
4.3.23. Tradeoff de rendimiento entre todas las variantes del modelo DETR. Muestra el rendimiento mAP@0.5:0.95 en función de la cantidad de GFlops de cómputo utilizado por la variante.	71
4.3.24. Curvas de F1-Score en función de la confianza (<i>confidence</i>) de detección de la variante DETR-R50 (<i>ResNet-50</i>).	72
4.3.25. Curvas de Precision en función de la confianza (<i>confidence</i>) de detección de la variante DETR-R50 (<i>ResNet-50</i>).	73
4.3.26. Curvas de Exhaustividad en función de la confianza (<i>confidence</i>) de detección de la variante DETR-R50 (<i>ResNet-50</i>).	74
4.3.27. Matriz de confusión de la variante DETR-R50 (<i>ResNet-50</i>).	75
4.3.28. Pérdida total promedio vs época de la validación en cada variante del modelo DINO.	76
4.3.29. Precisión Media Promedio (mAP) vs época con umbral IoU 0,5 para todas las variantes del modelo DINO.	77
4.3.30. Precisión Media Promedio (mAP) vs época con umbral IoU desde 0,5 hasta 0,95 para todas las variantes del modelo DINO.	78
4.3.31. F1-Score vs época para todas las variantes del modelo DINO.	78
4.3.32. Tradeoff de rendimiento entre todas las variantes del modelo DINO. Muestra el rendimiento mAP@0.5:0.95 en función de la cantidad de GFlops de cómputo utilizado por la variante.	79
4.3.33. Curvas de F1-Score en función de la confianza (<i>confidence</i>) de detección de la variante DINO-R50-4scale (<i>ResNet-50</i>).	80
4.3.34. Curvas de Precision en función de la confianza (<i>confidence</i>) de detección de la variante DINO-R50-4scale (<i>ResNet-50</i>).	81
4.3.35. Curvas de Exhaustividad en función de la confianza (<i>confidence</i>) de detección de la variante DINO-R50-4scale (<i>ResNet-50</i>).	82
4.3.36. Matriz de confusión de la variante DINO-R50-4scale (<i>ResNet-50</i>).	83
4.4.1. Inferencia comparativa entre las mejores variantes de cada modelo (<i>YOLOv5-s</i> , SSD512, DETR-R50 y DINO-R50-4scale). Se incluye el tiempo de latencia de procesamiento por variante y se muestra la leyenda para distinguir las cajas delimitadoras de fondo (reales) y las predichas.	84
4.4.2. Inferencia comparativa entre las mejores variantes de cada modelo (<i>YOLOv5-s</i> , SSD512, DETR-R50 y DINO-R50-4scale). Se incluye el tiempo de latencia de procesamiento por variante y se muestra la leyenda para distinguir las cajas delimitadoras de fondo (reales) y las predichas.	85

4.4.3.	Inferencia comparativa entre las mejores variantes de cada modelo (<i>YOLOv5-s</i> , SSD512, DETR-R50 y DINO-R50-4scale). Se incluye el tiempo de latencia de procesamiento por variante y se muestra la leyenda para distinguir las cajas delimitadoras de fondo (reales) y las predichas.	86
4.4.4.	Inferencia comparativa entre las mejores variantes de cada modelo (<i>YOLOv5-s</i> , SSD512, DETR-R50 y DINO-R50-4scale). Se incluye el tiempo de latencia de procesamiento por variante y se muestra la leyenda para distinguir las cajas delimitadoras de fondo (reales) y las predichas.	87
4.4.5.	Inferencia comparativa entre las mejores variantes de cada modelo (<i>YOLOv5-s</i> , SSD512, DETR-R50 y DINO-R50-4scale). Se incluye el tiempo de latencia de procesamiento por variante y se muestra la leyenda para distinguir las cajas delimitadoras de fondo (reales) y las predichas.	88
4.4.6.	Inferencia comparativa entre las mejores variantes de cada modelo (<i>YOLOv5-s</i> , SSD512, DETR-R50 y DINO-R50-4scale). Se incluye el tiempo de latencia de procesamiento por variante y se muestra la leyenda para distinguir las cajas delimitadoras de fondo (reales) y las predichas.	89
4.4.7.	Inferencia comparativa entre las mejores variantes de cada modelo (<i>YOLOv5-s</i> , SSD512, DETR-R50 y DINO-R50-4scale). Se incluye el tiempo de latencia de procesamiento por variante y se muestra la leyenda para distinguir las cajas delimitadoras de fondo (reales) y las predichas.	90

Capítulo 1

Introducción

Las correas transportadoras constituyen un componente crítico en la minería y en diversas industrias intensivas en el transporte de materiales a granel, al permitir el movimiento continuo y eficiente de grandes volúmenes de carga sobre largas distancias. Su operación sostenida se encuentra estrechamente vinculada a la productividad global de la planta, de modo que fallas inesperadas en la correa o en sus componentes asociados se traducen directamente en detenciones no programadas, sobrecostos de mantenimiento y potenciales riesgos para la seguridad de las personas y equipos. En este contexto, asegurar la disponibilidad y confiabilidad de las correas transportadoras es un requisito fundamental para la operación industrial moderna.

Durante su vida útil, las correas están expuestas a condiciones de servicio severas que favorecen la aparición de diversos modos de daño, tales como desgaste abrasivo, grietas o rasgaduras, roturas de empalme y fallas asociadas a desalineamientos prolongados. Estos defectos pueden evolucionar desde deterioros superficiales hasta fallas catastróficas, comprometiendo la integridad estructural de la correa y de la línea de transporte [1]. La gran longitud de los sistemas transportadores y la variabilidad de las condiciones ambientales dificultan su inspección exhaustiva, incrementando la probabilidad de que daños incipientes no sean detectados oportunamente.

Tradicionalmente, el monitoreo del estado de las correas transportadoras se ha basado en rondas de inspección visual realizadas por personal especializado, apoyadas ocasionalmente por dispositivos de medición puntual. Si bien este enfoque ha sido la práctica estándar en la industria, presenta limitaciones importantes: la cobertura espacial y temporal de las inspecciones es restringida, la detección de fallas depende fuertemente de la experiencia del inspector y, en muchos casos, es necesario detener o intervenir la correa para realizar revisiones detalladas, afectando la continuidad operacional. Estas restricciones han motivado el desarrollo de estrategias de monitoreo automatizado que reduzcan la carga operativa y mejoren la detección temprana de daños [2].

En paralelo, los avances en visión computacional y aprendizaje profundo (*Deep Learning*) han permitido desarrollar modelos capaces de identificar y localizar objetos o patrones específicos directamente a partir de imágenes, sin requerir una etapa manual de ingeniería de características. En particular, los detectores de una sola etapa (*one-stage detectors*), como las familias YOLO y SSD, han mostrado un compromiso favorable entre precisión y velocidad de inferencia en escenarios de detección en tiempo casi real, lo que los convierte en candidatos naturales para su aplicación en el monitoreo continuo de correas transportadoras [3, 4]. Adicionalmente, trabajos recientes han demostrado la viabilidad de utilizar arquitecturas basadas en YOLO para la detección de fallas o condiciones anómalas en sistemas transportadores, reforzando el potencial de estas técnicas en entornos industriales [2].

Considerando este contexto, la presente memoria aborda el problema de la identificación automática de fallas en correas transportadoras mediante modelos de detección de objetos basados en aprendizaje profundo, abarcando tanto detectores de una sola etapa como arquitecturas recientes basadas en *transformers*. El trabajo se centra en la implementación, entrenamiento y evaluación comparativa de estos enfoques sobre un conjunto de datos específico de defectos en correas, con el objetivo de analizar su capacidad para detectar distintos tipos de daño y su adecuación para ser integrados en sistemas de monitoreo y mantenimiento predictivo en un entorno cercano al operativo, según la eficiencia y exigencia de cada modelo.

1.1. Objetivo General

Implementar y evaluar algoritmos de detección de objetos, basados en inteligencia artificial, para la identificación automática de fallas en correas transportadoras.

1.2. Objetivos Específicos

Para lograr el objetivo general, se propone cumplir con los siguientes objetivos específicos:

1. Implementar modelos de detección de objetos basados en redes convolucionales, tales como YOLO (You Only Look Once) y sus variantes, y SSD (Single Shot Multibox Detector).
2. Implementar modelos de detección de objetos basados en redes neuronales con arquitectura tipo transformer, tales como DETR (DEtection TRansformer) y DINO.
3. Entrenar algoritmos de detección de objetos utilizando bases de datos con registros de fallas en correas transportadoras.
4. Comparar el desempeño de los modelos en la detección de distintos tipos de fallas mediante el uso de métricas de evaluación cuantitativas.

1.3. Alcances

- El entrenamiento de los algoritmos de detección se realizará exclusivamente con bases de datos públicas disponibles en la web que contengan registros de fallas en correas transportadoras.
- Considerar el uso de datos sin etiquetar, los cuales serán procesados mediante estrategias de etiquetado manual o mediante técnicas no supervisadas y/o auto-supervisadas, según su aplicabilidad.
- Evaluar el rendimiento de los modelos seleccionados (YOLO, SSD, DETR y DINO) en la detección y clasificación de fallas, utilizando una misma base de datos con partición en conjuntos de entrenamiento, validación y prueba. Posteriormente se evaluarán utilizando métricas estándar de evaluación.

Capítulo 2

Antecedentes

2.1. Estado del arte en la identificación de fallas en correas transportadoras

2.1.1. Fallas en correas transportadoras

Las correas transportadoras, máquinas de transporte de material continuo, operan en condiciones que las exponen a diversos tipos de daños, los cuales pueden generar costosos tiempos de inactividad. Para cada condición de falla, se estudiaron exhaustivamente sus tipos y causas, así como los métodos adecuados para su mitigación, contribuyendo a mejorar la confiabilidad del sistema [1].

Entre las fallas más comunes en correas transportadoras se identifican el *daño por impacto*, que puede provocar *perforaciones o roturas locales*; la *delaminación*, generada por esfuerzos cíclicos y fatiga; y la *desalineación de la banda*, que causa *desgaste anómalo en los bordes*. Además, se reporta *desgaste abrasivo* en la cubierta superior por efecto de partículas duras, *rotura de cables de acero* en correas reforzadas, y *cortes longitudinales y transversales*, todos los cuales afectan directamente la vida útil y seguridad operativa del sistema. Estas fallas se encuentran recopiladas en la Figura 2.1.1.

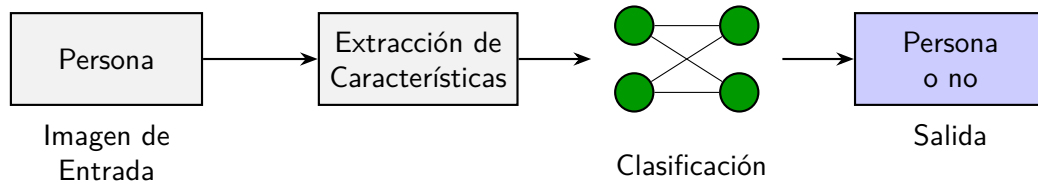


Figura 2.1.1: Tipos de falla comunes en correas transportadoras. Imágenes extraídas y adaptadas de [1].

2.1.2. Aprendizaje profundo (*Deep Learning*)

El aprendizaje profundo (*Deep Learning*) surge como una evolución de los enfoques clásicos de aprendizaje automático (*Machine Learning*), al introducir redes neuronales profundas capaces de aprender de forma automática representaciones jerárquicas directamente desde los datos. En los métodos tradicionales de *Machine Learning*, la etapa de ingeniería de características recae en el especialista, quien define manualmente descriptores (por ejemplo, basados en bordes, texturas o histogramas) y luego entrena un clasificador independiente sobre dichas representaciones. En contraste, en *Deep Learning* la misma red realiza de manera integrada la extracción de características y la decisión final, aprendiendo dichas representaciones de alto nivel a partir de grandes volúmenes de datos. Esto ha permitido superar de forma sistemática a los métodos convencionales en tareas de clasificación, segmentación y detección de objetos. En la Figura 2.1.2 se ilustra gráficamente la diferencia conceptual.

Machine Learning



Deep Learning

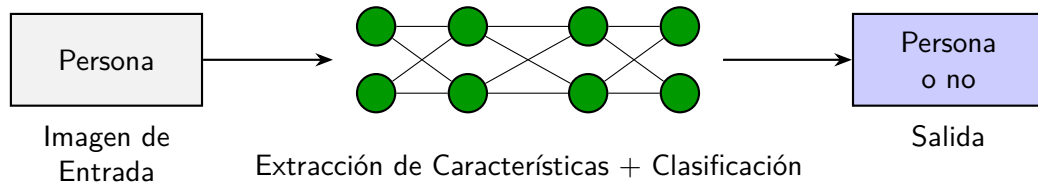


Figura 2.1.2: Machine Learning vs Deep Learning. Diagrama adaptado de [5].

Adicionalmente, el aprendizaje por transferencia (*transfer learning*) ha cobrado relevancia en escenarios con escasez de datos etiquetados, al reutilizar modelos profundos preentrenados en bases de datos extensas y ajustarlos a nuevas tareas de visión por computador, mejorando el rendimiento y reduciendo el costo computacional del entrenamiento en los nuevos modelos [5].

En trabajos recientes, una memoria de título desarrollada en la Universidad de Chile [2] implementa visión computacional y técnicas de aprendizaje profundo para la detección temprana de fallas en correas transportadoras, utilizando una de las versiones del modelo YOLO. El estudio describe detalladamente la implementación y calibración del modelo, además de proporcionar información técnica sobre redes neuronales convolucionales, métricas de evaluación, pruebas experimentales y resultados ilustrativos.

2.2. Detección de objetos en visión por computadora

La *detección de objetos* consiste en identificar automáticamente instancias presentes en una imagen o secuencia, localizándolas mediante cajas delimitadoras y asignándoles una clase (*clasificación*). En los modelos modernos, esta tarea se formula como un problema conjunto de clasificación y regresión: para cada región candidata de la imagen se predice, de manera simultánea, la probabilidad de pertenencia a una clase y las coordenadas de la caja delimitadora asociada. La Figura 2.2.1 ilustra ejemplos de detección simple y múltiple de objetos, representativos de este tipo de problemas [6].

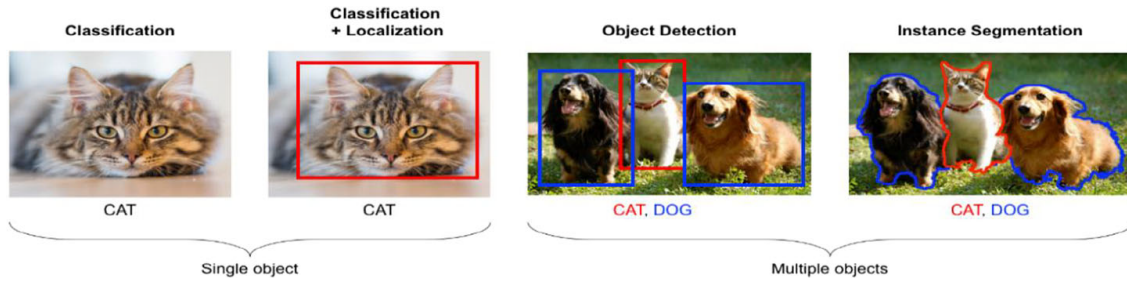


Figura 2.2.1: Ejemplos de detección de objetos simples y múltiples. Imagen extraída de [6].

La detección de objetos ha sido uno de los problemas centrales en visión por computadora, tradicionalmente abordado mediante arquitecturas basadas en redes neuronales convolucionales (CNN). Estos modelos dominaron el campo durante más de una década, impulsando desarrollos significativos en precisión y eficiencia. Sin embargo, alcanzar un balance óptimo entre velocidad y exactitud ha requerido explorar estructuras más allá del marco convencional de las CNN. Los modelos basados en transformers, originalmente desarrollados para procesamiento de lenguaje natural (Natural Language Processing), han demostrado ser altamente efectivos al capturar relaciones espaciales globales en imágenes, lo que los convierte en una propuesta a considerar para la implementación de nuevos modelos de detección de objetos [7].

Aplicado al sector industrial, la detección de fallas en correas transportadoras se aborda fundamentalmente como un problema de detección de objetos mediante aprendizaje supervisado. Existen dos enfoques principales actualmente usados: los modelos de dos etapas, como la familia R-CNN (red neuronal convolucional basada en regiones), que emplean redes de propuestas regionales seguidas de refinamiento, y los modelos de una etapa, como YOLO y SSD, que realizan predicciones por regresión directa, optimizados para aplicaciones en tiempo real. Un ejemplo de aplicación elaborado por Mengchao et al.[3], se muestra en la Figura 2.2.2.

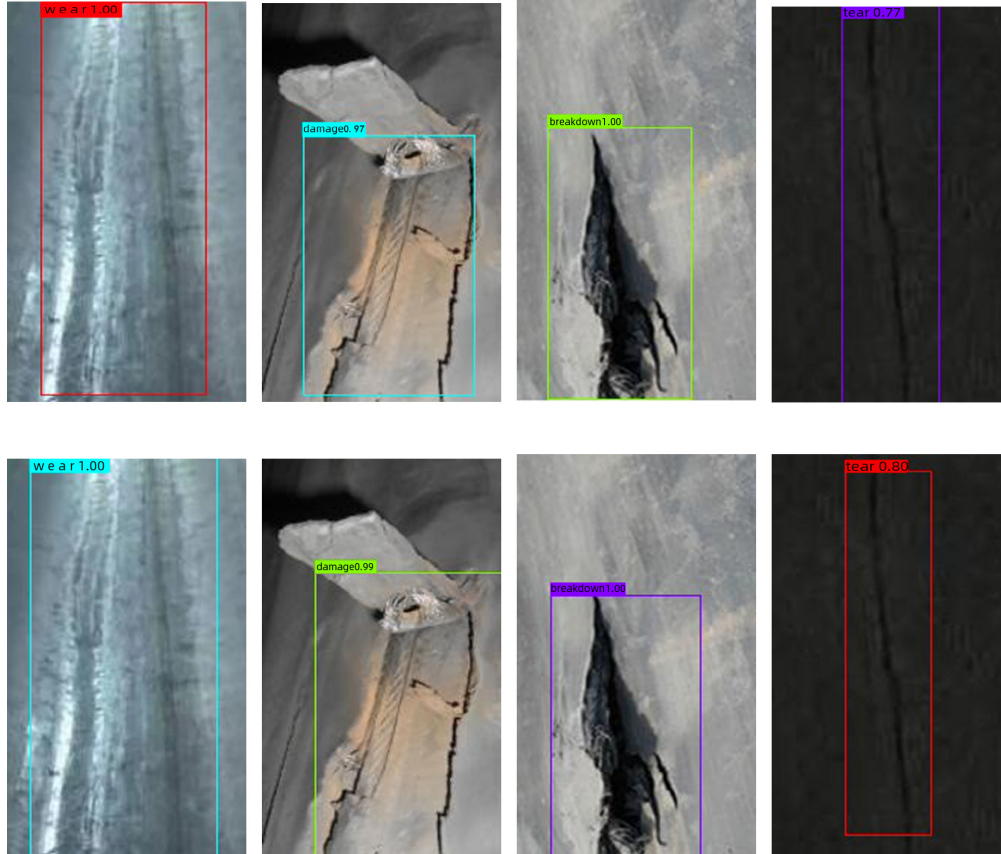


Figura 2.2.2: Ejemplo de detección de fallas utilizando detección de objetos en un caso industrial. Imagen extraída de [3].

2.3. Redes Neuronales Convolucionales (CNN)

2.3.1. Contextualización

Desde sus orígenes, las redes neuronales artificiales se han concebido como modelos inspirados en el funcionamiento del cerebro humano, en los que un conjunto de neuronas simples se conectan mediante pesos sinápticos para procesar información de manera distribuida. Las redes neuronales convolucionales (CNN, por sus siglas en inglés) profundizan en esta analogía al especializarse en el procesamiento del sistema visual: reciben la imagen como una matriz de píxeles y, a través de una secuencia estructurada de operaciones —capas de convolución, funciones de activación no lineales (por ejemplo, ReLU), etapas de *pooling*, aplanamiento y una capa final de clasificación basada en *softmax*— son capaces de reconocer dígitos, objetos, escenas o acciones con alta precisión [8].

En una CNN típica, las capas de convolución aplican filtros o núcleos (*kernels*) que se deslizan sobre la imagen, generando mapas de activación que condensan las características más representativas del contenido visual. Gracias a la conectividad local y al compartimiento de parámetros, estos filtros capturan patrones espaciales relevantes reduciendo el número total

de pesos a entrenar y otorgando cierta invarianza frente a traslaciones. Las capas de *pooling*, como el *max pooling*, reducen progresivamente la resolución espacial de los mapas de activación, reteniendo las respuestas más fuertes y contribuyendo simultáneamente a disminuir la complejidad computacional y el sobreajuste. Finalmente, una o varias capas totalmente conectadas (*fully connected*) integran la información extraída y producen la decisión final del modelo [8, 9].

2.3.2. Arquitectura

En este contexto, las redes neuronales convolucionales constituyen el núcleo de la mayoría de arquitecturas de visión por computadora. Una CNN está formada por una sucesión de capas de convolución, funciones de activación no lineales (como ReLU o variantes), capas de normalización y, en algunos casos, capas de reducción de dimensionalidad (*pooling*). Las convoluciones aplican filtros o *kernels* que se desplazan sobre la imagen, compartiendo pesos en el plano espacial. Este mecanismo reduce el número de parámetros, explota la estructura local de las imágenes y confiere invariancia aproximada a traslaciones [9].

Tal como se describe en revisiones recientes de aprendizaje profundo, las primeras capas de una CNN tienden a aprender características de bajo nivel (bordes, texturas, gradientes de intensidad), mientras que las capas intermedias y profundas capturan patrones de mayor complejidad (formas, partes de objetos y, finalmente, representaciones semánticas de alto nivel). Este aprendizaje jerárquico permite que una misma arquitectura pueda generalizar sobre grandes volúmenes de datos y adaptarse a distintas tareas visuales mediante el ajuste de sus parámetros [5, 9].

En las capas de reducción o *pooling*, como el *max pooling*, se disminuye la resolución espacial de los mapas de características conservando las activaciones más relevantes. Esta etapa reduce la carga computacional y actúa como un mecanismo de regularización, al atenuar pequeños desplazamientos o deformaciones locales en la imagen. Finalmente, uno o varios bloques de capas densas (*fully connected*) agregan la información extraída y realizan la clasificación o regresión final. Esto se muestra en la Figura 2.3.1.

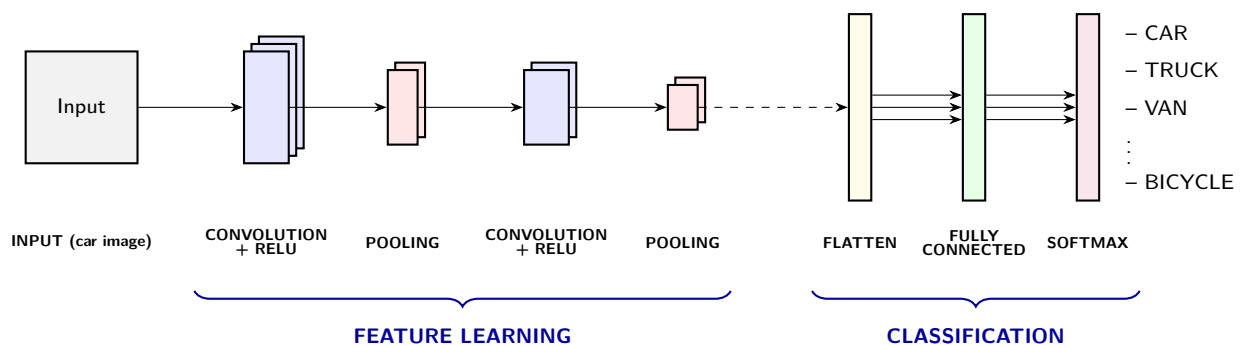


Figura 2.3.1: Arquitectura típica de Redes Neuronales Convolucionales. Diagrama adaptado de [5].

Para tareas de detección de objetos, las CNN se integran habitualmente en una arquitectura de tipo *backbone-neck-head*. El *backbone* (por ejemplo, variantes de VGG, Darknet, ResNet u otras arquitecturas optimizadas para eficiencia) extrae los mapas de características base. Sobre estos mapas, el *neck* construye representaciones multiescala mediante estructuras como pirámides de características (FPN, PAN), lo que permite detectar objetos de distintos tamaños. Finalmente, la *head* de detección proyecta esas características a predicciones de clases y cajas delimitadoras, como ocurre en los modelos de una sola etapa (*single-shot*) tipo YOLO y SSD, descritos en secciones posteriores [4]. Esta conceptualización tripartita se estandariza y extrapola también a las arquitecturas modernas basadas en *Transformers*, donde el *neck* es sustituido por un bloque *encoder-decoder* encargado de modelar dependencias globales mediante atención, acoplado a un *backbone* de extracción visual y a una *head* (típicamente redes prealimentadas simples o FFN) que ejecuta la predicción final [7].

2.3.3. Hiperparámetros en modelos de detección basados en CNN

Los hiperparámetros son valores que se definen externamente al modelo y controlan tanto su arquitectura como el proceso de aprendizaje. A diferencia de los parámetros internos de las redes neuronales (pesos y sesgos), que se ajustan automáticamente durante el entrenamiento, los hiperparámetros se fijan antes de entrenar y guían la optimización. Al finalizar el entrenamiento, el modelo contiene únicamente los parámetros ajustados, mientras que los hiperparámetros permanecen como configuraciones externas a la estructura aprendida [10].

En arquitecturas de detección de objetos basadas en CNN, como SSD, YOLO y variantes híbridas, la elección adecuada de estos hiperparámetros es crucial para lograr un equilibrio entre precisión, capacidad de generalización y costo computacional. Una configuración poco adecuada puede conducir a sobreajuste, subentrenamiento o inestabilidad numérica, mientras que combinaciones bien calibradas permiten aprovechar de forma más eficiente la capacidad representacional de la red [3, 5].

En la práctica, muchos hiperparámetros son compartidos entre diferentes modelos de detección, lo que permite establecer configuraciones iniciales razonables y luego refinarlas de manera iterativa en función de las métricas de evaluación empleadas en visión computacional. Este ajuste de hiperparámetros suele seguir un proceso experimental, apoyado en recomendaciones de la literatura y en guías de implementación como las proporcionadas por los desarrolladores de frameworks de código abierto [11]. La Tabla 2.3.1 resume algunos de los hiperparámetros más habituales, junto con rangos típicos orientados a modelos de detección de una etapa.

Tabla 2.3.1: Hiperparámetros comunes en modelos de detección basados en CNN

Hiperparámetro	Rango típico	Nombre
lr_0	$10^{-5} - 10^{-1}$	Tasa de aprendizaje inicial
lr_f	0,01 – 1,0	Factor de tasa de aprendizaje final
Número de <i>épocas</i>	200 – 400	<i>Épocas</i> de entrenamiento
<i>img_size</i>	640×640 píxeles	Resolución de entrada
<i>Batch_size</i>	2 / 4 / 16 imágenes	Tamaño de lote
<i>weight_decay</i>	0,0 – 0,001	Regularización L_2
<i>box_loss</i>	0,02 – 0,2	Peso pérdida de cajas
<i>cls_loss</i>	0,02 – 4,0	Peso pérdida de clasificación
<i>object_loss</i>	0,02 – 0,04	Peso pérdida de objeto
<i>warmup_epochs</i>	0 – 5	<i>Épocas</i> de calentamiento

En los esquemas de optimización empleados para el entrenamiento de redes convolucionales, la tasa de aprendizaje inicial (lr_0) juega un rol central al determinar la magnitud del paso de actualización paramétrica en cada iteración. Matemáticamente, si $\eta_0 = lr_0$, factor de aprendizaje inicial (η_0) y η_f es la tasa de aprendizaje final, el hiperparámetro de factor de tasa de aprendizaje final (lr_f) actúa como un factor de decaimiento tal que $\eta_f \approx lr_f \cdot \eta_0$. Adicionalmente, el número de *épocas* define el límite de pasadas completas sobre el conjunto de datos, mientras que el tamaño de lote (*batch size*) controla la cantidad de muestras procesadas simultáneamente para la estimación estocástica del gradiente. Por su parte, la resolución de entrada (denotada como ancho por alto), se encuentra estandarizada frecuentemente en 640×640 píxeles, establece el compromiso directo entre la retención de detalle espacial y la carga computacional [11].

Para mitigar el riesgo de sobreajuste, la regularización L_2 se modula a través del hiperparámetro *weight_decay*, el cual inyecta una penalización proporcional a la magnitud de los pesos en la función de pérdida. Simultáneamente, el balance del objetivo multi-tarea se ajusta mediante los coeficientes *box_loss*, *cls_loss* y *object_loss*, los cuales ponderan la importancia relativa de la regresión geométrica de las cajas, la clasificación de instancias y la confianza de presencia de objeto en cada ancla, respectivamente. Finalmente, para prevenir la divergencia de gradientes en las fases tempranas, el hiperparámetro *warmup_epochs* establece un período de calentamiento que escala gradualmente la tasa de aprendizaje desde una magnitud residual hasta su valor nominal operativo [11].

2.4. Mecanismo de atención y arquitectura Encoder-Decoder

2.4.1. Mecanismo de atención en Transformers

El núcleo del paradigma *Transformer* reside en el mecanismo de atención, el cual permite modelar dependencias globales entre los elementos de una secuencia o conjunto de datos sin recurrir a operaciones convolucionales o recurrentes [12]. La función de atención mapea una consulta (*Query*, Q) y un conjunto de pares clave-valor (*Key-Value*, K, V) hacia una salida, donde todos los elementos operan como vectores en un espacio de características.

La formulación algorítmica específica empleada se denomina atención escalada por producto punto (*Scaled Dot-Product Attention*). La salida se calcula como una suma ponderada de los valores (V), donde el peso asignado a cada valor es determinado por una función de compatibilidad entre la consulta y la clave correspondiente. Operando matricialmente sobre dimensiones d_k (para consultas y claves) y d_v (para valores), la operación se define rigurosamente como:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (2.4.1)$$

El factor de escalado $\frac{1}{\sqrt{d_k}}$ es crítico: previene que la magnitud de los productos punto escale excesivamente en dimensiones altas, lo cual saturaría la función *softmax* empujándola hacia regiones con gradientes infinitesimales, imposibilitando la convergencia durante el entrenamiento [12].

Para expandir la capacidad de representación del modelo, se implementa la atención multicabezal (*Multi-Head Attention*). En lugar de calcular una única función de atención sobre la dimensionalidad completa, el modelo proyecta linealmente Q , K y V un número h de veces hacia subespacios de menor dimensionalidad. Sobre cada una de estas proyecciones se aplica la atención de forma paralela, generando valores de salida de dimensión d_v . Finalmente, estas salidas se concatenan y se proyectan nuevamente. La operación matemática se expresa como:

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \\ \text{where head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \end{aligned} \quad (2.4.2)$$

Donde las proyecciones son matrices de parámetros de aprendizaje continuo $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ y $W^O \in \mathbb{R}^{h d_v \times d_{\text{model}}}$ [12]. Esta ramificación permite que el modelo atienda simultáneamente a diferentes aspectos representacionales y subespacios de características, optimizando la extracción de patrones complejos. Ambos mecanismos se ilustran en la Figura 2.4.1

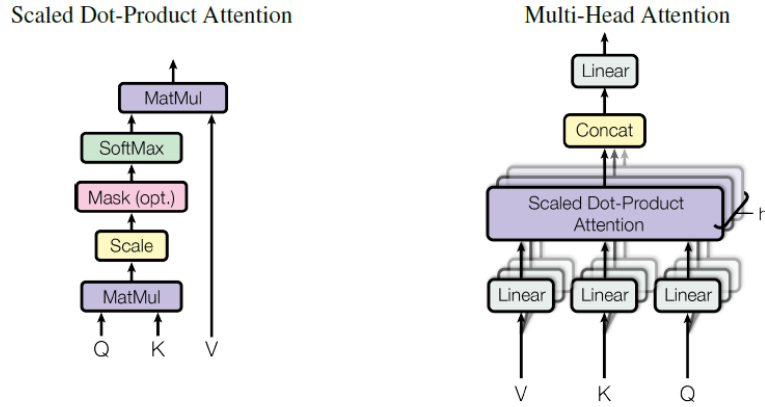


Figura 2.4.1: Atención escalada por producto punto (izquierda). Atención Multicabezal (derecha). Imagen extraída de [12].

2.4.2. Arquitectura Encoder-Decoder

La arquitectura fundamental de los Transformers se estructura en dos bloques principales compuestos por la repetición apilada de capas idénticas: el codificador (*Encoder*) y el decodificador (*Decoder*) [12].

El *codificador* asimila los datos de entrada y construye una representación de contexto global. Cada una de sus capas consta de dos submódulos principales: un mecanismo de autoatención multicabezal (*multi-head self-attention*) y una red neuronal totalmente conectada (*Feed-Forward Network*, FFN) aplicada posición por posición. La autoatención permite que cada elemento de la entrada evalúe su relación métrica con todos los demás elementos simultáneamente, capturando dependencias de largo alcance en un único paso computacional.

El *decodificador* transforma las representaciones procesadas por el codificador hacia el dominio de salida. Además de los dos submódulos presentes en el codificador, incorpora un tercer submódulo de *atención cruzada* (*cross-attention*). En este puente funcional, las consultas (Q , *queries*) provienen de la capa interna anterior del decodificador, mientras que las claves (K , *keys*) y los valores (V , *values*) se alimentan directamente desde la salida final del codificador. Esta interacción obliga al decodificador a focalizar su procesamiento en las regiones más relevantes de la entrada original al momento de generar predicciones.

Para garantizar la estabilidad numérica de la red profunda, ambos bloques integran conexiones residuales alrededor de cada submódulo, seguidas inmediatamente por una operación de normalización de capa (*Layer Normalization*), lo que facilita un flujo ininterrumpido de gradientes durante la fase de retropropagación.

De acuerdo a lo antes mencionado, la arquitectura transformer se muestra en la Figura 2.4.2.

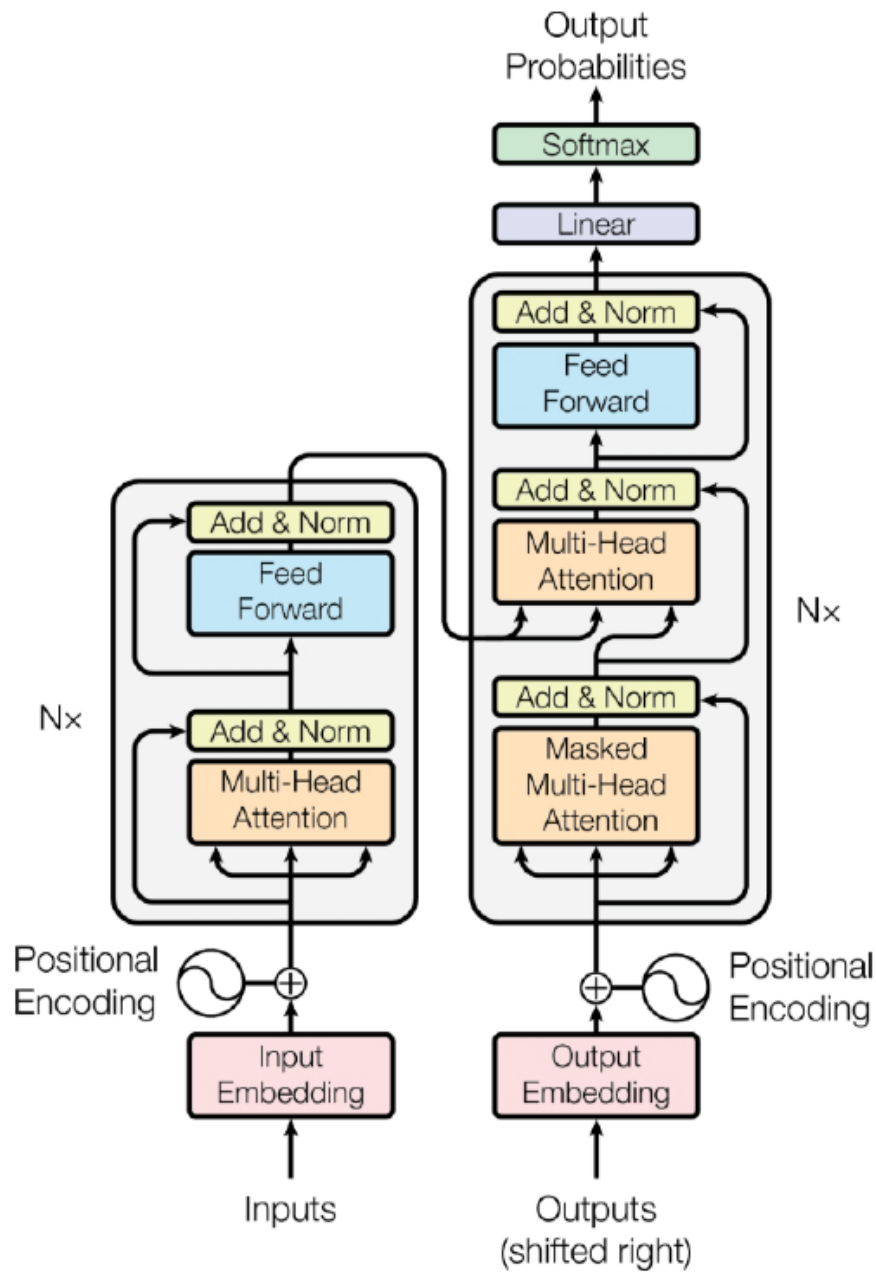


Figura 2.4.2: Arquitectura del Modelo Transformer. Imagen extraída de [12].

2.5. Modelos de detección de objetos de una sola etapa

2.5.1. Contextualización

YOLO y SSD pertenecen a la familia de modelos de detección de objetos de una sola etapa (*one-stage detectors*), en los cuales la localización de las cajas delimitadoras y la clasificación de las instancias se realizan en una única pasada por la red. Ambos se basan en redes neuronales convolucionales (CNN) y se enmarcan dentro de los enfoques de detección de objetos con aprendizaje profundo, donde la propia red efectúa de manera conjunta la extracción de características y la predicción final. En la Figura 2.5.1 se esquematiza la relación entre los esquemas tradicionales de detección de objetos basados en *Machine Learning*, los modelos de una etapa como YOLO y SSD —que serán objeto de estudio en esta memoria— y los modelos de dos etapas (*two-stage*), dentro de los cuales destaca la familia R-CNN, pionera en el uso de propuestas de regiones como base para la extracción de características y la detección posterior [6].

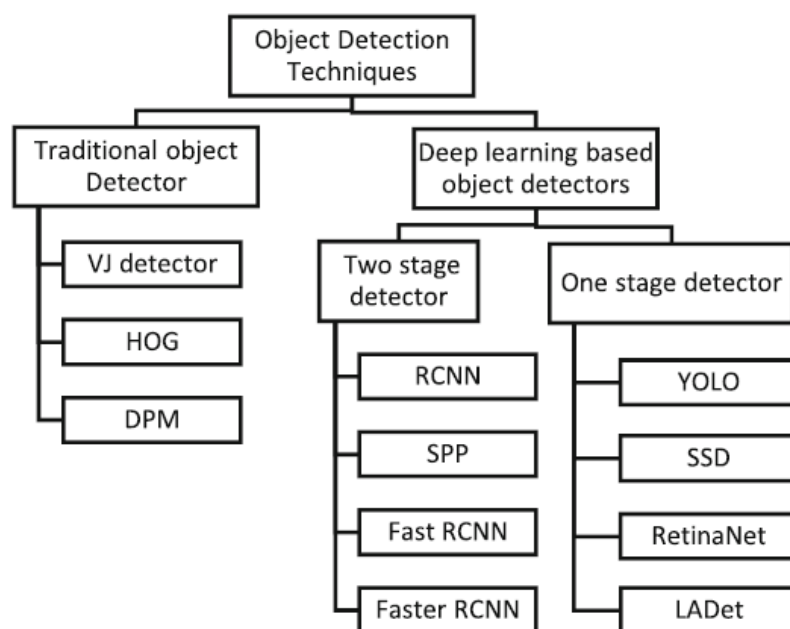


Figura 2.5.1: Clasificación de varias técnicas de detección de objetos. Imagen extraída de [6].

Dentro de los detectores de una sola etapa, una revisión exhaustiva de las arquitecturas de regresión propuestas en la literatura reciente identifica a las familias YOLO y SSD —incluyendo sus extensiones como DSSD— como los representantes más influyentes de este paradigma, al combinar elevados niveles de rendimiento con velocidades de inferencia adecuadas para aplicaciones en tiempo real [4]. En dicha revisión se analiza la evolución cronológica desde YOLOv1 y SSD-300 hasta variantes más recientes como YOLOv4 y YOLOv5, destacando el compromiso alcanzado entre exactitud, capacidad para detectar objetos pequeños y

eficiencia computacional en escenarios exigentes.

De forma general, estos modelos se apoyan en la idea de reemplazar las fases explícitas de generación de propuestas de regiones por una red totalmente convolucional que, a partir de mapas de características de distinta resolución, regresa directamente las coordenadas y puntuaciones de confianza de múltiples cajas ancla sobre la imagen de entrada. En el contexto de mantenimiento predictivo, este esquema permite instrumentar sistemas de monitoreo continuo basados en visión computacional, en los cuales las imágenes capturadas de la correa transportadora se procesan en tiempo casi real para detectar y clasificar defectos incipientes. La baja latencia y el alto grado de paralelismo que ofrecen estos detectores facilitan la integración de sus salidas con herramientas de pronóstico —por ejemplo, modelos que estiman la probabilidad de falla o la vida remanente en función de la frecuencia, severidad y evolución temporal de los daños—, habilitando decisiones proactivas sobre la intervención de equipos críticos antes de que se produzcan fallas catastróficas o detenciones no programadas.

2.5.2. Selección de modelos

La revisión de Zhang et al. [4] entrega un marco cuantitativo para esta decisión. En la Figura 2.5.2 de dicho trabajo se reportan, sobre el conjunto de datos MS COCO, métricas de precisión promedio (AP) y velocidad máxima en cuadros por segundo (FPS) para distintos detectores de una etapa. Allí se observa que modelos basados en SSD, como SSD-300 y DSSD-321, alcanzan AP del orden de 31–33,2 con velocidades cercanas a 11,8–16,4 FPS, mientras que las versiones recientes de YOLO (YOLOv3, YOLOv4 y YOLOv5) logran precisiones comparables o superiores junto con incrementos significativos en velocidad, llegando en el caso de YOLOv5 a valores cercanos a 200 FPS con AP en torno a 45,5 en MS COCO.

One-stage Detectors	Backbone	Max FPS	AP	Dataset	Performance analysis
YOLOv1-448 [4]	GoogleNet like	45	63.4	VOC2007+2012	Lightweight, high-speed, poor-accuracy, Insensitive to small goals.
YOLOv2-416 [11]	Darknet-19	67	78.6	VOC2007+2012	Borrowing from R-CNN's anchor box idea, higher-accuracy (compared to YOLOv1).
SSD-300 [5]	VGGNet-16	16.4	31	MSCOCO	Compared to the YOLOv1, SSD can handle objects with abnormal aspect ratio/configuration, but has poor detection capabilities for small objects.
DSSD-321 [6]	ResNet-101	11.8	33.2	MSCOCO	Improved the ability to detect small objects and improved accuracy, but the speed is slightly reduced.
YOLOv3-416 [13]	Darknet-53	34.5	33	MSCOCO	Deepen the depth of the darknet network, and the accuracy is improved again.
YOLOv4 [14]	CSPDarknet-53	125	45.8	MSCOCO	Introduce a series of strengthening measures to greatly improve the accuracy of target detection.
YOLOv5	CSPDarknet-53 like	200	45.5	MSCOCO	There are four models to choose from, which is more flexible than YOLOv4, but the performance is slightly worse.

Figura 2.5.2: Una revisión del rendimiento de los detectores de una sola etapa. Imagen extraída de [4].

Esta evidencia posiciona a YOLO como una alternativa particularmente atractiva cuando la restricción de tiempo de cómputo es crítica, como ocurre en la inspección continua de correas transportadoras en operación. No obstante lo anterior, la familia SSD mantiene un interés práctico como referencia clásica de detectores multi-escala. La arquitectura SSD fue concebida para superar las limitaciones de YOLOv1 frente a objetos pequeños y con relaciones de aspecto inusuales mediante el uso de múltiples mapas de características y conjuntos de cajas ancla de distintas escalas y proporciones [4]. Su comparación directa con un modelo YOLO moderno permite analizar, en un mismo dominio industrial, el rendimiento relativo de dos enfoques representativos: uno enfocado en una representación multi-escala explícita (SSD) y otro en una arquitectura optimizada y profundamente integrada que ha ido incorporando progresivamente mecanismos de fusión de características y mejoras en la función de pérdida (YOLOv5).

En consecuencia, en esta memoria se selecciona *YOLOv5* como representante de las arquitecturas YOLO de última generación (de acuerdo al artículo de Zhang et al.), dada su combinación favorable de velocidad y precisión y la disponibilidad de variantes escalables en tamaño de modelo (*nano*, *small*, *medium*, *large*, *extra large*). Además se adopta *SSD* como modelo de comparación base clásico de detectores de una etapa con énfasis en la detección multi-escala de objetos pequeños. Ambos modelos serán implementados y entrenados sobre un conjunto de datos específico de fallas en correas transportadoras, bajo un mismo protocolo experimental, con el propósito de comparar de forma controlada sus métricas de detección, su capacidad para identificar defectos y sus requerimientos computacionales en un escenario cercano al operativo.

2.5.3. YOLO (You Only Look Once)

YOLO es una familia de modelos de detección de objetos de una sola etapa (*one-stage detectors*) que destaca por su alta velocidad y eficiencia, al predecir directamente las clases y ubicaciones de objetos en una sola pasada por la red neuronal convolucional (CNN). A diferencia de los métodos de dos etapas (*two-stage detectors*), como R-CNN o Faster R-CNN, YOLO evita la generación explícita de propuestas de regiones (*region proposals*) y formula la detección como un problema de regresión directa sobre una rejilla espacial. Esta arquitectura unificada permite un compromiso favorable entre precisión y velocidad, lo que la hace especialmente adecuada para aplicaciones en tiempo real, como la inspección automática de correas transportadoras [13].

A lo largo de sus versiones, la familia YOLO ha incorporado mejoras progresivas en la arquitectura de la red, en las estrategias de entrenamiento y en los mecanismos de fusión multi-escala. Las primeras versiones (YOLOv1–YOLOv3) introdujeron el uso de cajas ancla (*anchor boxes*) y pirámides de características (*feature pyramids*), mientras que YOLOv4 integró la arquitectura CSPDarknet53 y técnicas avanzadas de entrenamiento [4]. Sobre esta base, YOLOv5 consolida una arquitectura moderna orientada a la implementación práctica,

optimizando el balance entre costo computacional y desempeño en métricas de detección, lo que explica su amplia adopción en aplicaciones industriales y embebidas [14].

2.5.3.1. Arquitectura del modelo YOLOv5

YOLOv5 adopta una arquitectura modular de tres componentes: espina dorsal (*backbone*), cuello de agregación de características (*neck*) y cabeza de detección (*detection head*), tal como se describe en revisiones recientes del modelo [14]. Esta descomposición permite separar claramente el aprendizaje de representaciones de bajo y alto nivel, la fusión de información multi-escala y la predicción final de cajas y clases. Ver Figura 2.5.3.

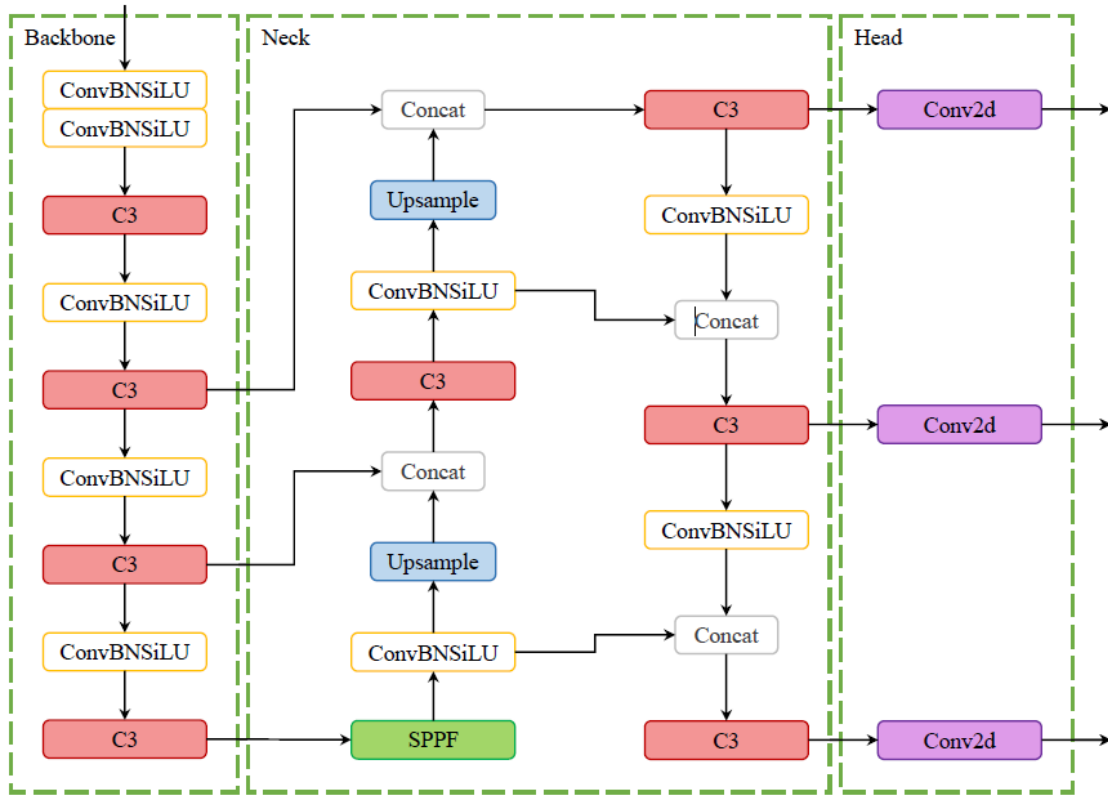


Figura 2.5.3: Arquitectura de YOLOv5, basada en una estructura de tipo *backbone-neck-head*. Imagen extraída de [14].

En términos generales, el *backbone* extrae características jerárquicas a partir de la imagen de entrada, el *neck* combina y refina dichas características en múltiples resoluciones, y la *head* produce, para cada nivel de resolución, las predicciones de cajas delimitadoras (*bounding boxes*), confianza de objeto (*objectness*) y probabilidades de clase. Esta organización sigue el esquema *backbone-neck-head* común en detectores de una etapa, pero YOLOv5 lo implementa mediante bloques convolucionales específicos (CSP, C3, SPPF) que mejoran la eficiencia del flujo de información y reducen redundancias en los gradientes [14]. A continuación se detalla cada una de las partes de la arquitectura de YOLOv5.

2.5.3.2. Bloques convolucionales: backbone, neck y head

Columna Vertebral (*backbone*)

La columna vertebral (*backbone*) de YOLOv5 se basa en una variante de CSPDarknet53, derivada de la arquitectura Darknet-53 con la incorporación de la estrategia de redes parciales por etapa (*Cross Stage Partial Network, CSP*). Esta estrategia consiste, de forma simplificada, en dividir el mapa de características en dos rutas paralelas, procesar una de ellas mediante bloques residuales y luego fusionarlas nuevamente. El objetivo es reducir la duplicación de gradientes y hacer más eficiente el uso de los canales de características.

En la práctica, YOLOv5 utiliza bloques C3, que son módulos convolucionales de tipo CSP simplificados. Cada bloque C3 agrupa varias capas convolucionales con conexiones de atajo (*shortcut connections*) internas, lo que facilita el aprendizaje de representaciones profundas sin degradar el gradiente. La espina dorsal, tras recibir imágenes de entrada de 640x640 píxeles (resolución de trabajo de YOLO), combina secuencias de bloques *ConvBNSiLU* (convolución + normalización por lotes + función de activación SiLU) y módulos C3, culminando en un bloque de agrupación piramidal espacial rápida (*Spatial Pyramid Pooling Fast, SPPF*). Este último expande el campo receptivo (*receptive field*) mediante operaciones de *max-pooling* con diferentes tamaños de ventana, sin incrementar la resolución de los mapas de características [14].

Cuello de agregación (*neck*)

El cuello (*neck*) de YOLOv5 implementa una red de agregación de rutas (*Path Aggregation Network, PANet*) combinada con la filosofía CSP, conocida como CSP-PAN. Su función es fusionar características provenientes de diferentes niveles de resolución del *backbone*, de modo que las capas profundas (con alto contenido semántico) se combinen con capas más superficiales (con mejor resolución espacial) [14].

Esta fusión se realiza mediante una secuencia de operaciones de muestreo ascendente (*up-sampling*), concatenación de mapas de características y bloques C3 adicionales. De este modo, el *neck* genera un conjunto de mapas a distintas escalas que preservan tanto la localización precisa de los objetos como la información contextual, lo que es especialmente relevante en escenas industriales donde defectos pequeños y grandes coexisten en una misma imagen.

Cabeza de detección (*detection head*)

La cabeza (*head*) de YOLOv5 es una cabeza de detección convolucional multi-escala, conceptualmente similar a la introducida en YOLOv3. Para cada escala de salida (por ejemplo, mapas con baja, media y alta resolución), la cabeza predice, en cada celda de la rejilla y para cada caja ancla (*anchor box*), un vector que incluye: desplazamientos de la caja delimitadora (posición y tamaño), una puntuación de presencia de objeto (*objectness score*) y una distri-

bución de probabilidades sobre las clases.

Estas predicciones se obtienen mediante capas convolucionales finales de tamaño 1×1 , que proyectan los mapas de características del *cuello* hacia el espacio de parámetros de detección. La formulación permite que la red aprenda, de forma conjunta, la localización y clasificación de múltiples objetos en una sola etapa, manteniendo el tiempo de inferencia bajo, un requisito clave para la detección de fallas en correas transportadoras en tiempo casi real.

2.5.3.3. Funciones de activación y funciones de salida en YOLOv5

Funciones de activación internas

En las capas internas, YOLOv5 utiliza principalmente la función de activación SiLU (*Sigmoid Linear Unit*), también conocida como Swish. La función SiLU combina una parte lineal con una función sigmoide (*sigmoid*) y se expresa, de forma simplificada, como

$$\text{SiLU}(x) = x \cdot \sigma(x),$$

donde $\sigma(x)$ es la función sigmoide estándar. En comparación con funciones más simples como ReLU (*Rectified Linear Unit*), SiLU entrega transiciones más suaves y mantiene gradientes no nulos en un rango más amplio de valores, lo que favorece la optimización en redes profundas.

La combinación *ConvBNSiLU* (convolución + normalización por lotes + SiLU) se utiliza de forma repetida en los bloques C3 y en otras partes del *backbone* y el *neck*, contribuyendo a estabilizar el entrenamiento y mejorar la capacidad de generalización del modelo sobre conjuntos de datos complejos.

Funciones de salida para detección y clasificación

En la capa de salida, YOLOv5 emplea funciones de activación específicas para cada tipo de predicción:

- *Confianza de objeto (objectness)*: la probabilidad de que exista un objeto en una celda de la rejilla se modela mediante una función sigmoide (*sigmoid*), que asigna valores en el rango $[0, 1]$. Esta probabilidad se interpreta como el grado de confianza en que la caja propuesta contiene un objeto de interés.
- *Probabilidades de clase (class probabilities)*: para cada clase, YOLOv5 utiliza igualmente funciones sigmoides independientes, lo que permite un esquema de clasificación de tipo multi-etiqueta (*multi-label*), en lugar de una normalización *softmax* estricta. De este modo, una misma caja podría, en teoría, activar más de una clase, aunque en la práctica suele seleccionarse la clase con mayor probabilidad.
- *Parámetros de la caja delimitadora (bounding box regression)*: las coordenadas centrales y dimensiones de la caja se parametrizan respecto de las cajas ancla y de la celda de

la rejilla. A través de funciones no lineales (incluyendo sigmoides y transformaciones exponenciales), el modelo aprende a ajustar el tamaño y posición de las cajas para aproximarse lo mejor posible a las anotaciones reales.

Estas funciones de salida se combinan con funciones de pérdida (*loss functions*) específicas, típicamente basadas en el índice de similitud de intersección sobre unión (*Intersection over Union, IoU*) para las cajas y en la entropía cruzada (*cross-entropy*) para las probabilidades, lo que permite entrenar el modelo de forma robusta frente a solapamientos entre objetos y desbalances entre clases.

2.5.3.4. Diferencias clave de YOLOv5 respecto a versiones anteriores

En síntesis, y de acuerdo con revisiones recientes de la literatura [14], las principales diferencias de YOLOv5 frente a versiones previas de la familia se pueden resumir en los siguientes puntos:

- Uso sistemático de la estrategia CSP (*Cross Stage Partial*) en el *backbone* y el *neck*, mediante los módulos C3, lo que mejora la eficiencia del flujo de gradientes y reduce redundancias.
- Incorporación del bloque SPPF (*Spatial Pyramid Pooling Fast*), que extiende el campo receptivo con menor costo computacional que los esquemas de pirámide espacial tradicionales.
- Adopción de la función de activación SiLU (*Sigmoid Linear Unit*) en los bloques convolucionales internos, en lugar de activaciones más simples como ReLU o Mish, mejorando la suavidad del paisaje de optimización.
- Definición de múltiples variantes de tamaño (por ejemplo, YOLOv5n, YOLOv5s, YOLOv5m, YOLOv5l, YOLOv5x), que permiten ajustar el equilibrio entre precisión y velocidad de inferencia según las restricciones de hardware y los requisitos de la aplicación.

2.5.4. SSD (Single Shot Multibox Detector)

SSD es un detector de objetos de una sola etapa (*single-shot detector*) que, al igual que YOLO, evita el uso explícito de propuestas de regiones y realiza simultáneamente la clasificación y regresión de cajas delimitadoras en una única pasada de la red. Su diseño se basa en una red convolucional profunda preentrenada para clasificación (VGG-16), sobre la cual se añaden capas convolucionales adicionales que producen mapas de características a diferentes escalas. En cada uno de estos mapas, SSD evalúa un conjunto de cajas predeterminadas (*default boxes*) de múltiples escalas y razones de aspecto —estrategia de la cual deriva el término *multibox*—, generando para cada una puntuaciones de clase y desplazamientos de caja [15].

De este modo, SSD modela la detección como un problema de regresión y clasificación conjunta sobre un conjunto fijo de cajas ancla distribuidas de manera densa en varios mapas de características. Esta formulación permite alcanzar un compromiso favorable entre precisión y velocidad, evitando la etapa de *region proposals* y el reprocesamiento de píxeles o características, logrando así detección en tiempo casi real con una precisión comparable a métodos de dos etapas basados en Faster R-CNN [15]. La arquitectura general (que contempla sus dos versiones principales, SSD300 y SSD512), se muestra en la Figura 2.5.4.

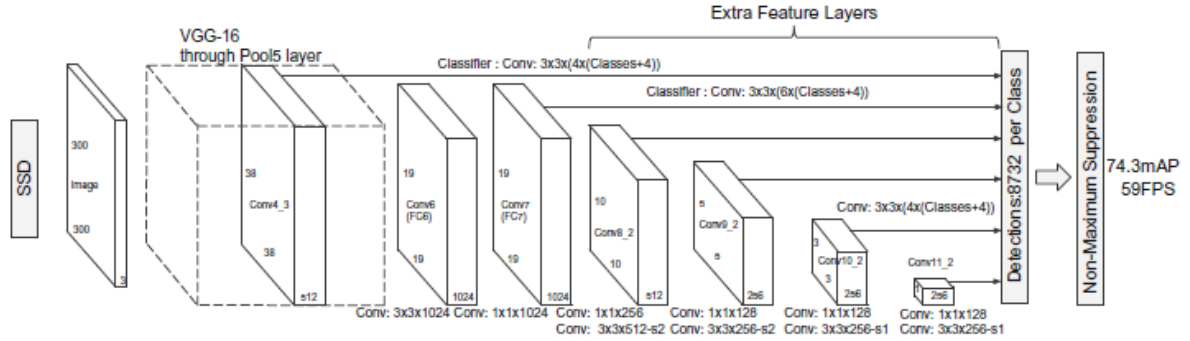


Figura 2.5.4: Arquitectura general de SSD. Imagen extraída y procesada de [15].

2.5.4.1. Arquitectura del modelo SSD

Conceptualmente, SSD se describe como una arquitectura basada en un *backbone* con capas de características adicionales y cabezas de predicción convolucionales, estructurada en múltiples niveles de resolución. A partir de una imagen de entrada (típicamente de 300×300 o 512×512 píxeles), la red VGG-16 preentrenada extrae representaciones jerárquicas hasta una profundidad específica, tras lo cual se anexan capas convolucionales extra que reducen progresivamente la resolución espacial de los mapas de características.

Sobre un conjunto seleccionado de mapas de salida, se definen para cada celda varias cajas predeterminadas (*default boxes*) de distintas escalas y razones de aspecto. Para cada caja candidata, filtros convolucionales compactos de 3×3 computan en una sola operación los desplazamientos espaciales y las puntuaciones de clase asociadas. El tensor global de salidas comprende miles de predicciones por imagen, las cuales son depuradas posteriormente mediante umbrales de confianza y supresión de no-máximos (*Non-Maximum Suppression*, *NMS*) para consolidar las detecciones finales [15]. A continuación se detallan los módulos estructurales del modelo.

2.5.4.2. Bloques convolucionales: backbone, capas adicionales y cabezas de predicción

Columna Vertebral (*backbone*)

El *backbone* de SSD hereda la arquitectura VGG-16 truncada hasta la capa *pool5*, descartando las capas densas terminales y los módulos de *dropout*. Las capas *fc6* y *fc7* se transforman en capas convolucionales (*conv6* y *conv7*). Simultáneamente, la operación de *pool5* se modifica y se implementan convoluciones dilatadas (*atrous convolutions*) para expandir el campo receptivo sin degradar la resolución de los mapas de características [15].

Entre los mapas extraídos, la capa *conv4_3* resulta crítica al conservar una alta resolución espacial, empleándose directamente para la detección de instancias de menor escala. Para alinear su magnitud de activaciones con las capas más profundas, SSD aplica una normalización L2 por ubicación sobre *conv4_3*, introduciendo un factor de escala aprendible durante la optimización. Este mecanismo garantiza la integración estable de características superficiales y profundas durante la propagación [15].

Capas de características adicionales (*extra feature layers*)

Sobre la salida de *conv7*, se acoplan redes convolucionales en cascada que generan mapas de resolución decreciente (por ejemplo, *conv8_2* hasta *conv11_2* en SSD300, añadiendo *conv12_2* en SSD512). Estos bloques operan típicamente mediante pares de convoluciones de reducción de dimensionalidad de canales (1×1) seguidas de convoluciones espaciales (3×3) con *stride* mayor a uno para ejecutar la submuestreación.

El objetivo de esta extensión es construir una pirámide de características explícita, donde la detección de objetos pequeños se delega a los mapas de alta resolución, y los objetos de gran escala se capturan en las capas terminales más gruesas. Este diseño emula el comportamiento de una *feature pyramid* de manera eficiente y directa sobre el flujo convolucional de una sola etapa [15].

Cabezas de predicción (*prediction heads*)

Para cada mapa seleccionado, SSD proyecta una cabeza de predicción que opera de forma densa sobre la topología espacial. En un mapa de dimensiones $m \times n$ con p canales, se asocian k *default boxes* por celda. La proyección convolucional aplica filtros de $3 \times 3 \times p$, emitiendo c puntuaciones de clase (incluyendo la clase de fondo) y 4 coordenadas de regresión geométrica por cada caja.

La arquitectura genera un total de $(c + 4)kmn$ salidas por mapa. Evaluado globalmente, SSD300 produce del orden de 8732 cajas candidatas por imagen, volumen que se incrementa sustancialmente en SSD512 debido a su mayor resolución de entrada [15]. Durante el post-procesamiento, este conjunto masivo se filtra empleando NMS por clase para aislar las detecciones óptimas.

2.5.4.3. Funciones de activación y funciones de salida en SSD

Funciones de activación internas

En las capas extractoras, SSD conserva la función de activación no lineal ReLU (*Rectified Linear Unit*) estándar de VGG-16 aplicada tras las convoluciones. Esta función proporciona una representación computacionalmente eficiente y mitiga la saturación del gradiente en capas profundas. Adicionalmente, la normalización L2 específica de la capa *conv4_3* actúa como un regularizador estabilizador para la varianza de las activaciones tempranas empleadas en la detección.

Funciones de salida para detección y clasificación

La capa de inferencia emplea funciones diferenciadas según el objetivo matemático de la rama:

- *Regresión de cajas delimitadoras (bounding box regression)*: Los parámetros geométricos se modelan como desplazamientos residuales (centroide y dimensiones) respecto a la *default box*. La optimización emplea una métrica de pérdida Smooth L1 (tipo Huber), la cual ofrece mayor robustez frente a valores atípicos espaciales y estabiliza el entrenamiento [15].
- *Puntuaciones de clase (class confidences)*: La distribución probabilística sobre las c clases se aproxima mediante la función *softmax*, penalizada a través de la entropía cruzada multiclase (*softmax loss*). El fondo se procesa explícitamente como una categoría independiente [15].

El objetivo total de entrenamiento se formula como una combinación ponderada de la pérdida de localización (Smooth L1) y la pérdida de confianza (*softmax loss*), normalizada por el número de cajas positivas asociadas a objetos reales. Este enfoque, junto con estrategias de minería de negativos difíciles (*hard negative mining*) y de aumento de datos, permite entrenar SSD de manera estable incluso con un gran número de cajas ancla negativas por imagen [15].

2.6. Modelos de detección de objetos basados en Transformers

2.6.1. Contextualización

El éxito consolidado de las Redes Neuronales Convolucionales (CNN) en visión computacional se fundamenta en su capacidad para explotar la invariancia a la traslación y la localidad espacial mediante el uso de filtros deslizantes. Sin embargo, esta naturaleza local limita inherentemente la captura de dependencias de largo alcance, requiriendo arquitecturas profundamente estratificadas o mecanismos complejos de fusión multiescala (como las pirámides de características) para asimilar el contexto global de una escena.

En este escenario, el paradigma de los *Transformers* —originalmente concebido para el procesamiento de lenguaje natural— redefine la formulación de tareas visuales al reemplazar por completo las operaciones de convolución por mecanismos de autoatención global [12]. En el contexto de la detección de objetos, esta transición metodológica permite modelar explícitamente todas las interacciones de emparejamiento entre los elementos de una imagen. Al tratar la detección como un problema directo de predicción de conjuntos, estos modelos eliminan de manera nativa la necesidad de componentes heurísticos y procesos manuales críticamente dependientes del diseño del especialista, tales como la generación de cajas ancla espaciales (*anchor boxes*) y los algoritmos de supresión de no máximos (NMS) encargados de colapsar predicciones duplicadas [16].

2.6.2. Arquitectura general

A nivel general, mientras que los detectores convolucionales de una sola etapa se estructuran bajo la topología clásica de columna vertebral-cuello-cabeza (*backbone-neck-head*), los detectores basados en Transformers reconfiguran estos componentes sustituyendo el *neck* por un bloque de codificación y decodificación (*Encoder-Decoder*), manteniendo una cabeza de predicción optimizada para la proyección directa de conjuntos paralelos.

El flujo de información dentro de esta topología unificada se desglosa en los siguientes bloques funcionales:

- *Columna Vertebral (backbone)*: Una red convolucional convencional preentrenada (ej. ResNet, SwinL) recibe la imagen de entrada y genera un mapa de características de baja resolución espacial y alta densidad de canales, condensando la información visual representativa.
- *Mecanismo de aplanado y codificación posicional (Positional Encoding)*: Dado que los módulos de atención interna procesan secuencias unidimensionales y son simétricos respecto a permutaciones, las dimensiones espaciales del mapa de características se colapsan

a una dimensión. Posteriormente, se incorporan vectores de codificación posicional estática (*positional encodings*) para inyectar de forma explícita la estructura espacial perdida durante el aplanado.

- *Codificador contextual (Transformer Encoder)*: Formado por una sucesión de capas de autoatención multifrecuencia y redes prealimentadas (FFN). El codificador analiza de forma global las relaciones de proximidad y contexto entre todos los píxeles/parches del mapa, separando y desenredando las instancias de los objetos directamente en el espacio de características.
- *Decodificador de consultas (Transformer Decoder)*: Procesa en paralelo un conjunto fijo de vectores de aprendizaje continuo denominados consultas de objetos (*object queries*). A través de capas alternas de autoatención (interacción consulta-consulta) y atención cruzada (*cross-attention* consulta-características del codificador), el decodificador reestructura los vectores query mapeándolos hacia las zonas con mayor relevancia de la escena visual.
- *Cabezas de predicción (Prediction Heads)*: Un bloque final constituido por redes neuronales completamente conectadas (FFN) toma los tensores resultantes del decodificador y proyecta, de forma paralela e independiente, las coordenadas normalizadas de las cajas y las probabilidades de clase correspondientes para cada ranura (*slot*).

2.6.3. DETR (DEtection TRansformer)

2.6.3.1. Arquitectura general de DETR

El modelo DETR [16] constituye el hito fundamental en la transición hacia detectores de extremo a extremo libres de componentes manuales. Arquitectónicamente, se concibe como una implementación elegante que acopla un extractor convolucional ResNet-50 con un bloque Transformer estándar de procesamiento no autorregresivo. El objetivo central de DETR es inferir simultáneamente un conjunto de tamaño fijo de N predicciones, donde N se define para ser significativamente mayor que el número real de objetos habitualmente presentes en una imagen de la base de datos operativa. El modelo y su arquitectura se ilustran en la Figura 2.6.1.

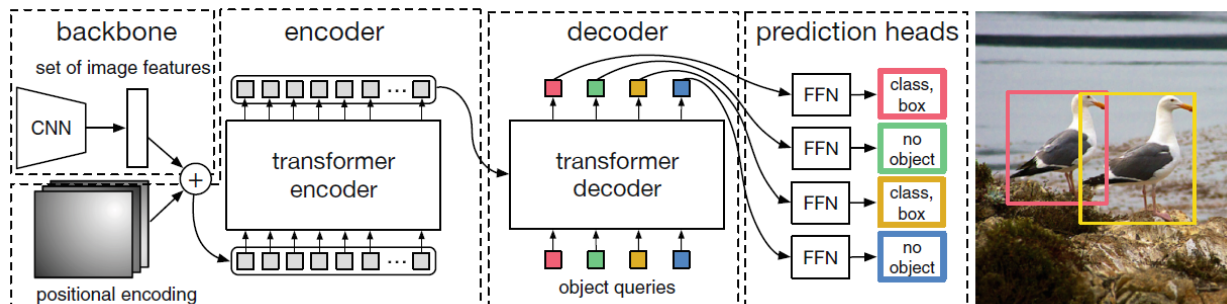


Figura 2.6.1: Arquitectura general de DETR. Imagen extraída de [16].

2.6.3.2. Bloques fundamentales: Backbone, Encoder, Decoder y FFN

Columna vertebral (*Backbone*)

Conectando con la estructura unificada definida previamente en la Sección 2.6.2, el *backbone* actúa como el extractor visual base. En la configuración por defecto de DETR, el mapa de características extraído reduce la resolución espacial por un factor de 32 ($H, W = H_0/32, W_0/32$). Para transicionar al dominio de atención, una convolución de 1×1 comprime la dimensionalidad de canales de 2048 a un espacio latente de tamaño $d = 256$, colapsando posteriormente sus dimensiones espaciales a una secuencia unidimensional.

Codificador (*Encoder*)

Como mecanismo de codificación contextual, el codificador asimila la secuencia aplanada junto con su codificación posicional. Posee 6 capas apiladas, compuestas estructuralmente por bloques de autoatención multicabezal (*8-headed self-attention*) y una red prealimentada (FFN) con dimensión oculta de 2048, modelando las interacciones globales entre todos los parches de la imagen.

Decodificador (*Decoder*) y Cabezas de Predicción (*FFN*)

Actuando como decodificador de consultas, este bloque implementa 6 capas de procesamiento en paralelo. A diferencia de los Transformers autorregresivos, DETR decodifica simultáneamente un conjunto fijo de N consultas. Estas *object queries* operan como filtros abstractos que asimilan el contexto global a través de la atención cruzada con el codificador. Finalmente, la cabeza de predicción proyecta las salidas: un perceptrón multicapa (MLP) de 3 capas densas predice las coordenadas geométricas, mientras una proyección lineal ejecuta la clasificación mediante *softmax*. Si una ranura no empareja con un objeto real, el modelo predice la clase de fondo ($\emptyset$).

2.6.3.3. Pérdida de emparejamiento bipartito y el Algoritmo Húngaro

La naturaleza post-procesamiento libre de DETR radica en su formulación de la función de pérdida como un problema de optimización global de conjuntos. Para puntuar las predic-

ciones con respecto a la verdad de terreno, se requiere realizar un emparejamiento biunívoco (uno a uno) entre el conjunto de etiquetas reales y (completado con elementos vacíos $\emptyset$ hasta alcanzar el tamaño N) y el conjunto de predicciones $\hat{y} = \{\hat{y}_i\}_{i=1}^N$.

Para hallar la permutación óptima del espacio de permutaciones $\mathfrak{S}_N$ con el menor costo global, se resuelve el problema de asignación mediante el método formalizado en la Ecuación 2.6.1:

$$\hat{\sigma} = \arg \min_{\sigma \in \mathfrak{S}_N} \sum_{i=1}^N \mathcal{L}_{match}(y_i, \hat{y}_{\sigma(i)}) \quad (2.6.1)$$

Donde $\mathcal{L}_{match}(y_i, \hat{y}_{\sigma(i)})$ representa el costo de emparejamiento por parejas entre la anotación real $y_i = (c_i, b_i)$ (donde c_i es la clase y b_i el vector de coordenadas espaciales $[x, y, w, h]$ relativo al tamaño de la imagen) y la predicción indexada bajo la permutación $\sigma(i)$. Este costo integra de forma equilibrada la verosimilitud de la clase y la calidad geométrica de la caja delimitadora, expresándose matemáticamente según la Ecuación 2.6.2:

$$\mathcal{L}_{match}(y_i, \hat{y}_{\sigma(i)}) = -\mathbb{I}_{\{c_i \neq \emptyset\}} \hat{p}_{\sigma(i)}(c_i) + \mathbb{I}_{\{c_i \neq \emptyset\}} \mathcal{L}_{box}(b_i, \hat{b}_{\sigma(i)}) \quad (2.6.2)$$

Donde $\hat{p}_{\sigma(i)}(c_i)$ indica la probabilidad predicha para la clase verdadera del objeto objetivo, y $\mathcal{L}_{box}$ es la penalización de regresión. Esta asignación óptima se calcula eficientemente mediante el algoritmo Húngaro [16]. Una vez determinada la asignación óptima $\hat{\sigma}$, se computa la función de pérdida final del modelo, denominada pérdida Húngara (*Hungarian Loss*), definida por la Ecuación 2.6.3:

$$\mathcal{L}_{Hungarian}(y, \hat{y}) = \sum_{i=1}^N \left[-\log \hat{p}_{\hat{\sigma}(i)}(c_i) + \mathbb{I}_{\{c_i \neq \emptyset\}} \mathcal{L}_{box}(b_i, \hat{b}_{\hat{\sigma}(i)}) \right] \quad (2.6.3)$$

Para contrarrestar el desbalance de clases provocado por la alta cantidad de ranuras vacías, el término de log-probabilidad correspondiente a $c_i = \emptyset$ se pondera a la baja por un factor de escala de 10. Para garantizar la invariancia de escala de la pérdida geométrica frente a la coexistencia de objetos de dimensiones heterogéneas, la pérdida de caja $\mathcal{L}_{box}$ combina linealmente una norma L_1 espacial y una pérdida de Intersección sobre la Unión Generalizada (GIoU), calibrada por dos hiperparámetros de ponderación fijos (λ_{L1} y λ_{iou}):

$$\mathcal{L}_{box}(b_i, \hat{b}_{\hat{\sigma}(i)}) = \lambda_{iou} \mathcal{L}_{iou}(b_i, \hat{b}_{\hat{\sigma}(i)}) + \lambda_{L1} \|b_i - \hat{b}_{\hat{\sigma}(i)}\|_1 \quad (2.6.4)$$

2.6.4. DINO (DETR with Improved DeNoising Anchor Boxes)

2.6.4.1. Evolución y arquitectura general de DINO

A pesar de las ventajas conceptuales de DETR, las implementaciones iniciales presentaban dos limitaciones de carácter crítico en entornos operativos: una velocidad de convergencia extremadamente lenta durante la optimización —atribuible a la inestabilidad inherente de las asignaciones dinámicas del emparejamiento bipartito— y deficiencias notorias en el rendi-

miento de detección sobre objetos con dimensiones espaciales reducidas. El modelo DINO [17] surge para mitigar de forma conjunta estas ineficiencias, consolidándose como un detector Transformer de extremo a extremo capaz de alcanzar el estado del arte competitivo mediante innovaciones metodológicas explícitas.

DINO preserva la topología jerárquica elemental *backbone–encoder–decoder*, pero reconfigura el flujo incorporando el mecanismo de atención deformable multiescala heredado de Deformable DETR [17]. Esta operación concentra el cómputo de atención restringiendo el escaneo a un conjunto pequeño y acotado de puntos de muestreo distribuidos clave de referencia espacial, optimizando la complejidad computacional. Asimismo, DINO modifica sustancialmente la naturaleza matemática de las *object queries* decodificadoras, formulándolas como cajas de anclaje dinámicas cuatridimensionales (4D) de la forma (x, y, w, h) , las cuales son refinadas secuencialmente de manera iterativa a lo largo de cada una de las capas constitutivas del decodificador. Esto se ilustra en la Figura 2.6.2.

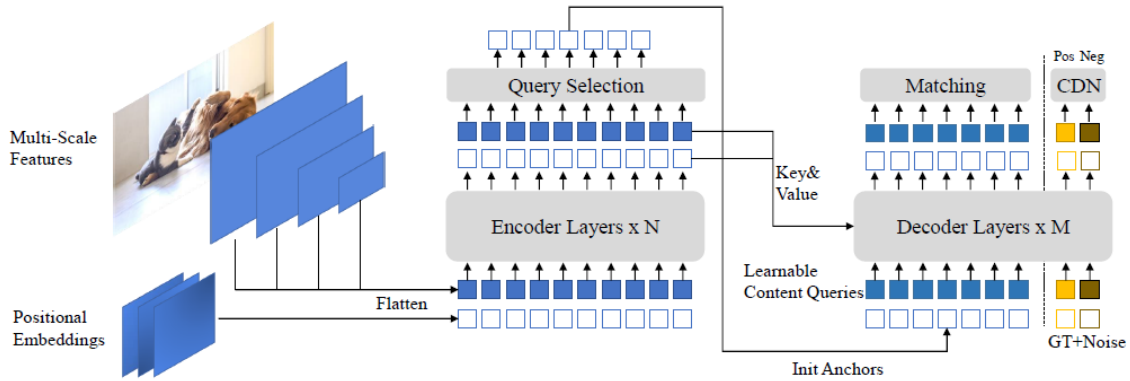


Figura 2.6.2: Arquitectura DINO. Imagen extraída de [17].

2.6.4.2. Eliminación de ruido contrastiva (*Contrastive DeNoising*)

La inestabilidad en el entrenamiento de detectores basados en asignaciones biunívocas se manifiesta cuando múltiples cajas candidatas convergen espacialmente sobre un mismo objeto; pequeñas perturbaciones numéricas provocan saltos drásticos en las permutaciones del algoritmo Húngaro, desestabilizando los gradientes. Aunque modelos previos como DN-DETR paliaron este efecto inyectando ruido moderado para forzar la reconstrucción geométrica (*denoising loss*), carecían de robustez ante falsos positivos cercanos. DINO evoluciona orgánicamente este concepto mediante el entrenamiento por eliminación de ruido contrastivo (*Contrastive DeNoising*, CDN) [17]. Este enfoque expone explícitamente al modelo ante muestras negativas difíciles (*hard negative samples*) definiendo dos escalas geométricas estrictas ($\lambda_1 < \lambda_2$), inyectando en paralelo dos tipos de consultas en un esquema concéntrico:

- *Consultas Positivas (CDN positivas)*: Ubicadas en el contorno interior (λ_1), obligando

al modelo a reconstruir con exactitud el objeto real y estimar su clase.

- *Consultas Negativas (CDN negativas)*: Confinadas en la franja espacial intermedia entre λ_1 y λ_2 . Al operar como anclas confusas, el modelo es penalizado si no las clasifica estrictamente como "sin objeto" ($\emptyset$ o fondo).

Este régimen dual de entrenamiento enseña a la red a discernir desviaciones milimétricas de localización, suprimiendo de raíz las predicciones redundantes alrededor de un mismo defecto físico.

2.6.4.3. Selección mixta de consultas (*mixed query selection*)

La inicialización de las consultas en el decodificador ha transitado históricamente desde tensores estáticos (en el DETR clásico) hasta selecciones dinámicas basadas en las mejores activaciones del codificador (*pure query selection*). Sin embargo, transferir directamente las características semánticas del codificador arrastra ambigüedad contextual, ya que un único parche convolucional frecuentemente encapsula información parcial o múltiples instancias superpuestas [17]. Para depurar esta entrada, DINO implementa una estrategia de selección mixta de consultas (*mixed query selection*). Este mecanismo desacopla la inicialización: extrae de los mejores K mapas del codificador únicamente las coordenadas espaciales para inicializar dinámicamente las anclas posicionales, mientras que las consultas de contenido asociadas se mantienen aisladas como parámetros de aprendizaje independientes de la imagen. Esta separación modular orienta la atención inicial del decodificador estrictamente hacia la topología espacial, evitando la contaminación semántica prematura.

2.6.4.4. Proyección anticipada doble (*look forward twice*)

El refinamiento iterativo de cajas de anclaje suele padecer de un desacople en el flujo de gradientes. En arquitecturas tradicionales (*look forward once*), la propagación matemática se interrumpe intencionalmente mediante una desconexión de gradiente (*gradient detach*) entre capas sucesivas para evitar inestabilidad. No obstante, esto impide que los parámetros de una capa temprana se ajusten basándose en el éxito o fracaso del refinamiento geométrico ocurrido en capas más profundas. Para restaurar esta comunicación sin sacrificar la estabilidad numérica, DINO introduce la proyección anticipada doble (*look forward twice*) [17]. Bajo este esquema bidireccional, los parámetros estructurales de la capa i se optimizan asimilando simultáneamente la pérdida local de su predicción y el gradiente retropropagado desde la capa adyacente ($i + 1$). Esta lógica algorítmica de actualización unificada se rige formalmente por el sistema de ecuaciones diferenciales y de asignación estructurado en la Ecuación 2.6.5:

$$\begin{aligned}
\Delta b_i &= \text{Layer}_i(b_{i-1}) \\
b'_i &= \text{Update}(b_{i-1}, \Delta b_i) \\
b_i &= \text{Detach}(b'_i) \\
b_i^{(\text{pred})} &= \text{Update}(b'_{i-1}, \Delta b_i)
\end{aligned} \tag{2.6.5}$$

Donde la función de actualización *Update* refina la geometría espacial operando con sumas algebraicas aplicadas en el dominio matemático transformado tras la evaluación de una función sigmoide inversa. Mediante el flujo de *Look Forward Twice*, el gradiente residual proveniente del término diferencial Δb_{i+1} penetra la barrera de la capa adyacente para optimizar de forma unificada tanto la calidad geométrica de la caja inicial de referencia b_{i-1} como la precisión intrínseca del estimador de desplazamiento local.

2.6.5. Hiperparámetros en modelos de detección basados en Transformers

La calibración de hiperparámetros en arquitecturas basadas en mecanismos de atención es fundamental para garantizar la convergencia estable de los optimizadores analíticos de segundo orden y regular el balance matemático de las predicciones de conjuntos. A diferencia de las CNN convolucionales optimizadas típicamente mediante algoritmos SGD clásicos, los Transformers de visión requieren configuraciones de optimización adaptativa con decaimiento de peso explícito y programas estrictos de modulación de tasa de aprendizaje. La Tabla 2.6.1 sistematiza los hiperparámetros de control característicos para la arquitectura general de detectores Transformer.

Tabla 2.6.1: Hiperparámetros representativos en arquitecturas Transformer

Hiperparámetros	Descripción Técnica
lr	Tasa de aprendizaje global del modelo.
lr_{backbone}	Tasa de aprendizaje reducida para el extractor visual.
<code>weight_decay</code>	Regularización L2 para mitigar el sobreajuste.
<code>enc_layers</code>	Número de capas operativas en el Codificador.
<code>dec_layers</code>	Número de capas operativas en el Decodificador.
<code>hidden_dim</code> (d)	Dimensión del espacio latente en los bloques de atención.
<code>dim_feedforward</code>	Dimensión oculta en las redes prealimentadas (FFN).
<code>nheads</code>	Cantidad de cabezas en la atención multicabezal.
<code>num_queries</code>	Ranuras fijas procesadas en paralelo por imagen.
α_{focal}	Factor de balanceo de clases en la pérdida Focal.
γ_{focal}	Penalizador predicciones de fondo en la pérdida Focal.

2.6.6. Selección de modelos

La adopción de arquitecturas basadas en atención para la detección de fallas requiere balancear la complejidad paramétrica ($Params$) y el costo computacional ($GFLOPs$) con el rendimiento empírico. El marco cuantitativo establecido por las investigaciones seminales de DETR [16] y DINO [17] evidencia la superioridad progresiva de estos modelos sobre el conjunto de datos estándar MS-COCO. La Tabla 2.6.2 sintetiza la evolución del rendimiento (AP y AP₅₀) frente a los requerimientos de cómputo dentro de la misma familia de detectores.

Tabla 2.6.2: Comparativa de rendimiento y complejidad evolutiva para detectores *Transformer* en MS-COCO. Datos extraídos y consolidados de [16] y [17].

Modelo	Backbone	Épocas	AP (mAP global)	AP ₅₀	GFLOPs	FPS Máx.	Parámetros
DETR	<i>ResNet-50</i>	500	42.0	62.4	86	28	41M
DETR-DC5	<i>ResNet-50</i>	500	43.3	63.1	187	12	41M
DETR-R101	<i>ResNet-101</i>	500	43.5	63.8	152	20	60M
DETR-DC5-R101	<i>ResNet-101</i>	500	44.9	64.7	253	10	60M
DINO (4-scale)	<i>ResNet-50</i>	36	50.9	69.0	279	24	47M
DINO (5-scale)	<i>ResNet-50</i>	36	51.2	69.0	860	10	47M
DINO (Ours)	<i>Swin-L</i>	36	63.2	-	-	-	218M

Los resultados tabulados demuestran empíricamente que las mejoras arquitectónicas integradas en DINO optimizan radicalmente su eficiencia de convergencia y rendimiento final respecto a la arquitectura seminal DETR. Específicamente, DINO supera los 50.9 AP en apenas 36 épocas (frente a los 42.0 AP requeridos por el modelo base DETR tras 500 épocas), y escala masivamente hasta los 63.2 AP al implementar un extractor *Swin-L*, estableciendo el estado del arte en la literatura.

En consecuencia, esta memoria establece a *DETR* —abarcando sus variantes *ResNet-50*, *ResNet-101* y modificaciones *DC5*— como la arquitectura de referencia (línea base) del paradigma de atención, justificado por su valor histórico como pionero en la detección libre de anclas. Como contraparte evolutiva, se selecciona a *DINO* (mediante sus configuraciones *ResNet-50* de 4 y 5 escalas, y *Swin-L*) para cuantificar el límite superior de rendimiento empírico y aceleración de convergencia derivado de sus optimizaciones frente al modelo original.

2.7. Métricas

Con el fin de validar la implementación de los modelos de detección de objetos para la clasificación de fallas, se emplean métricas cuantitativas estándar en visión por computador. El uso de estas métricas otorgan las herramientas esenciales para evaluar críticamente y de forma fiable los modelos implementados [18].

2.7.1. Precisión, Exhaustividad y F-Score

La *precisión* (*precision*) mide la proporción de verdaderos positivos entre todas las predicciones positivas. La *exhaustividad* (*recall*) mide la proporción de verdaderos positivos detectados sobre los positivos reales.

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN}$$

Tabla 2.7.1: Definiciones generales *Machine Learning* [18]

Abreviación	Significado
T	Total
P	Positivos
N	Negativos
TP	Verdaderos Positivos
TN	Verdaderos Negativos
FP	Falsos Negativos
n	Número de Clases

El *F-Score* (F_β) es la media ponderada entre precisión y recall, ajustada por el parámetro β . Un caso particular es el *F1-Score* (F_1), donde $\beta = 1$.

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}} \quad F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

2.7.2. Intersección sobre la unión (IoU)

La intersección sobre la unión (*Intersection over Union*) ó IoU, evalúa el solapamiento entre la predicción y la verdad de terreno:

$$\text{IoU} = \frac{\text{Área de Intersección}}{\text{Área de Unión}}$$

En la Figura 2.7.1, es posible ver una representación mediante conjuntos de la definición de intersección sobre la unión.

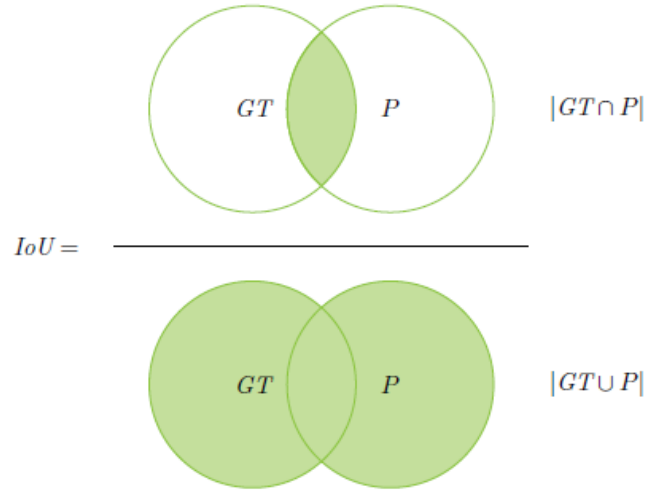


Figure 8: Intersection over union (IoU).

$$IoU = \frac{|GT \cap P|}{|GT \cup P|}$$

Figura 2.7.1: Representación visual de Intersección sobre la Unión. Imagen extraída de [18].

Esta métrica es aplicada en la detección de objetos para ver qué tan acertada es la caja delimitadora de predicción respecto a la caja delimitadora real.

2.7.3. Precisión promedio (Average Precision, AP) y precisión media promedio (mean Average Precision, mAP)

La *precisión promedio* (*Average Precision*) o AP, cuantifica el desempeño del modelo para una clase específica como el área bajo la curva precisión–exhaustividad (*precision–recall*). En términos prácticos, el AP resume en un único valor cómo varía la precisión al recorrer distintos umbrales de confianza, integrando el compromiso entre detectar la mayor cantidad de verdaderos positivos posible y mantener bajo el número de falsos positivos para dicha clase.

La *precisión promedio media* (*mean Average Precision*) o mAP, se define como el promedio del AP calculado sobre todas las clases del problema. Esta métrica entrega una visión global del rendimiento del modelo de detección, evitando que el análisis se concentre únicamente en las clases mayoritarias. En la práctica, se emplean variantes estandarizadas como *mAP@50*, que utiliza un umbral IoU fijo de 0,5 para considerar una predicción como correcta, y *mAP@50-95*, que promedia el AP sobre umbrales de IoU entre 0,5 y 0,95 (con paso de 0,05), imponiendo un criterio más estricto sobre la calidad geométrica de las cajas predichas.

2.7.4. Matriz de confusión (Confusion Matrix)

La *matriz de confusión* (*Confusion Matrix*) representa de forma tabular los aciertos y errores del modelo al comparar sus predicciones con la verdad de terreno. En su forma más simple (binaria), recoge los conteos de verdaderos positivos (TP), verdaderos negativos (TN), falsos positivos (FP) y falsos negativos (FN); en el caso multiclase, se extiende a una matriz de dimensión $n \times n$, donde cada fila corresponde a la clase verdadera y cada columna a la clase predicha.

Esta representación permite analizar en detalle el comportamiento del clasificador, identificando clases que se confunden sistemáticamente entre sí, posibles desequilibrios en el rendimiento (por ejemplo, clases con alta tasa de falsos negativos) y patrones de error que no son evidentes a partir de métricas agregadas. En el contexto de detección de objetos, la matriz de confusión se construye a partir de las detecciones aceptadas según un umbral de confianza e IoU, y resulta especialmente útil para interpretar los resultados por tipo de falla o categoría de interés.

Capítulo 3

Metodología

3.1. Identificación de fallas

De acuerdo al Capítulo 2, sección 2.1, las fallas en correas transportadoras se encuentran enmarcadas de acuerdo a [1] y para la implementación de modelos de reconocimiento de objetos que identifiquen estas fallas, se consideraron las siguientes categorías:

- *Agujero (Hole)*: pérdida completa de material a través del espesor de la banda, de tamaño apreciable, originada por eventos de daño severo (impactos altos, atrapamiento de material o elementos extraños).
- *Daño por impacto (Impact Damage)*: deformaciones, cortes o abrasiones localizadas en la zona de carga, producidas por el impacto repetitivo o puntual del material transportado.
- *Perforación (Puncture)*: daño puntual en el que un elemento punzante atraviesa la banda generando una abertura de menor extensión superficial que un agujero, usualmente de carácter incipiente.
- *Desgarro (Tear)*: separación longitudinal o transversal de la banda, asociada a sobrecargas, material atascado o concentraciones de tensión, y que puede propagarse a lo largo de la correa.
- *Abrasión (Wear)*: desgaste progresivo de la superficie de la correa debido a fricción, que se manifiesta como adelgazamiento del recubrimiento y patrones característicos de pérdida de material.

De esta forma, las fallas antes mencionadas se convierten en *clases* para los modelos de detección de objetos. Los casos donde no se presenten fallas, se consideran clases de fondo (*background*), las cuales corresponderán a casos sin falla.

3.2. Base de datos

3.2.1. Base de datos elaborada

El entrenamiento de modelos basados en redes neuronales (convolucionales o Transformers) requiere disponer de un conjunto de datos suficientemente representativo, anotado y particionado en subconjuntos mutuamente excluyentes:

- *Entrenamiento (train)*: conjunto utilizado para ajustar los parámetros del modelo mediante el proceso de optimización. Se utilizó un 70 % del total de muestras disponibles.
- *Validación (valid)*: conjunto empleado durante el entrenamiento para calcular métricas intermedias, seleccionar hiperparámetros y monitorizar el sobreajuste. Sus ejemplos no se usan para actualizar los pesos del modelo y, por lo general, representan una fracción menor que el conjunto de entrenamiento. Se utilizó un 20 % del total de muestras disponibles.
- *Prueba (test)*: conjunto reservado para la evaluación final e independiente del modelo, una vez completado el ajuste y la selección de hiperparámetros. Permite estimar el desempeño esperado en condiciones de uso real. Se utilizó el 10 % del total de muestras disponibles.

3.2.2. Recopilación y limpieza

Mediante el sitio web *Roboflow* y repositorios académicos, se recopilaron diversas bases de datos relacionadas con fallas en correas transportadoras. Cada conjunto original presentaba su propia organización de clases, resolución y formato de anotación, exigiendo una homologación estricta.

El proceso de depuración operó en tres fases secuenciales. En primer lugar, se purgaron imágenes irrelevantes (ausencia de fallas) o con ruido severo (desenfoque, baja resolución). En segundo lugar, se normalizaron las anotaciones corrigiendo etiquetas inconsistentes y reajustando cajas delimitadoras geométrica o posicionalmente defectuosas. Finalmente, se homogeneizó la nomenclatura de clases según lo descrito en la Sección 3.1. Este protocolo iterativo permitió consolidar un corpus definitivo de 2312 imágenes útiles, escaladas a una resolución matricial uniforme de 640×640 píxeles.

3.2.3. Estructura topológica y formato de anotación

Para facilitar la evaluación y separar los regímenes de entrenamiento, validación y prueba, la base de datos se dividió físicamente. La organización de los datos, gestionada mediante un archivo de configuración maestro (*data.yaml*) que mapea rutas y clases, sigue el estándar

Código 3.1: Estructura topológica del conjunto de datos estandarizado. Formato extraído de [19]

```

dataset/
|
|-- train/
|   |-- images/
|       |-- 0001.jpg
|       |-- 0002.jpg
|       `-- ...
|   |-- labels/
|       |-- 0001.txt
|       |-- 0002.txt
|       `-- ...
|-- valid/
|   |-- images/
|       |-- 1622.jpg
|       |-- 1623.jpg
|       `-- ...
|   |-- labels/
|       |-- 1622.txt
|       |-- 1623.txt
|       `-- ...
|-- test/
|   |-- images/
|       |-- 2083.jpg
|       |-- 2084.jpg
|       `-- ...
|   |-- labels/
|       |-- 2083.txt
|       |-- 2084.txt
|       `-- ...
|-- data.yaml

```

jerárquico expuesto en el Código 3.1.

Dentro de la estructuración mencionada, la parametrización de las etiquetas espaciales se estandarizó universalmente bajo el formato YOLO. Cada imagen posee un archivo de texto (*label.txt*) homónimo, donde cada línea representa una instancia detectada mediante el vector acoplado a su clase. De esta forma, el nombre de cada una de las imágenes en la carpeta *images/*, en la base de datos (*dataset/*), comparte el mismo nombre de su archivo de etiqueta en la carpeta *labels/*, con el fin de facilitar el procesamiento y localización. El vector presente en cada etiqueta se muestra en la Ecuación 3.2.1.

$$v = [c, x, y, w, h] \quad (3.2.1)$$

Donde $c \in \{0, 1, 2, 3, 4\}$ es el identificador de clase de cada falla y se corresponde con el siguiente vector: $\{Hole, ImpactDamage, Puncture, Tear, Wear\}$. Para garantizar la invariancia frente a futuras transformaciones de resolución matricial, las coordenadas del centroide (x, y) y las dimensiones de la caja (w, h) se normalizan en el intervalo continuo $[0, 1]$ respecto a la anchura (W) y altura (H) absoluta de la imagen original, rigiéndose por el sistema de Ecuaciones 3.2.2:

$$\begin{aligned} x &= \frac{x_{\min} + x_{\max}}{2W}, & y &= \frac{y_{\min} + y_{\max}}{2H} \\ w &= \frac{x_{\max} - x_{\min}}{W}, & h &= \frac{y_{\max} - y_{\min}}{H} \end{aligned} \quad (3.2.2)$$

En la Figura 3.2.1 se muestra una representación de las coordenadas en el vector de etiquetado.

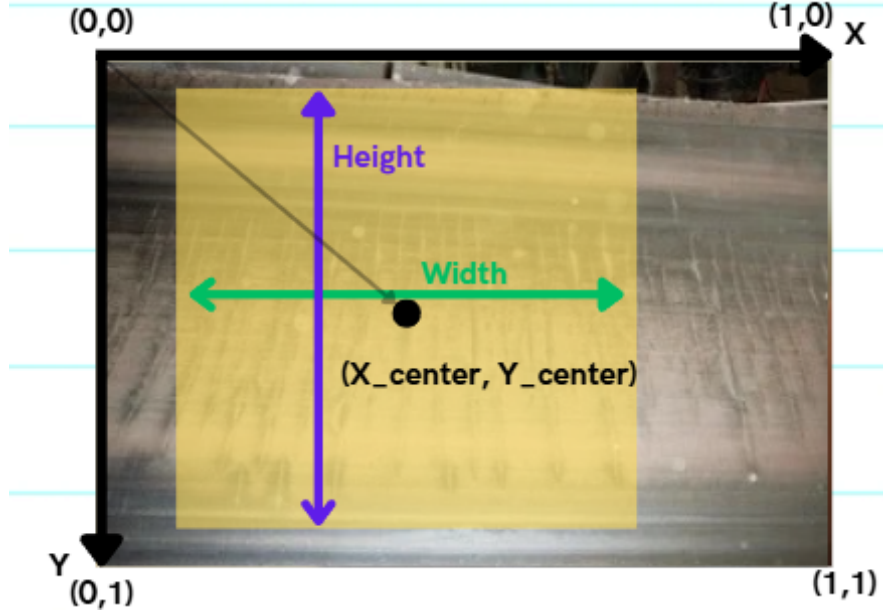


Figura 3.2.1: Representación visual de las coordenadas espaciales del vector de formato de etiquetas. Elaboración Propia.

3.2.4. Adaptación de etiquetas

Si bien la normalización escalar de YOLO es altamente eficiente para el almacenamiento en disco, las arquitecturas convolucionales como SSD exigen tensores con coordenadas absolutas (formato Pascal VOC), mientras que los detectores basados en atención (DETR, DINO) requieren anotaciones jerárquicas bajo el estándar MS-COCO (*JSON*).

Para resolver esta discrepancia geométrica sin la ineficiencia de duplicar físicamente los archivos, se implementó el módulo auxiliar *data_loader.py*. Operando como un intérprete dinámico en la implementación de utilidades, este algoritmo lee los tensores normalizados de YOLO durante la fase de carga en memoria de los datos y aplica las transformaciones algebraicas inversas necesarias. De este modo, adapta las coordenadas, las mallas espaciales y la resolución a las restricciones topológicas exactas del modelo requerido.

3.3. Entorno virtual y arquitectura de la implementación

Con el fin de asegurar la reproducibilidad de los experimentos y aislar las dependencias del proyecto, se implementó un entorno virtual basado en *Python* en la raíz del repositorio oficial [20]. Todas las librerías requeridas se gestionan mediante un archivo de requerimientos (*requirements.txt*), que agrupa las dependencias y paquetes de ROCm (debido al uso de Arquitectura AMD para esta memoria), librerías de Pytorch compatibles, scripts de análisis de datos y librerías específicas para cada modelo. Para garantizar la estandarización, la gestión del código fuente se estructuró mediante un sistema topológico de ramas (*branches*) a través del controlador de versiones GIT.

El núcleo de procesamiento (*engine*), las configuraciones globales y los algoritmos utilitarios operan como una capa base transversal (*wrapper*). En este ecosistema, cada arquitectura de detección se aísla de forma modular en una rama independiente del repositorio. Debido a restricciones físicas de memoria en el hardware (VRAM), el motor de cada rama opera con un diseño de inserción única (*single-socket*): solo una arquitectura puede acoplarse y entrenarse a la vez. Los códigos (*scripts*) principales actúan como orquestadores que encienden el flujo: entrenamiento (*train*), validación (*valid*) y prueba (*test*). Estos orquestadores cargan bases de datos y configuraciones en memoria, entrenando y validando el modelo, para finalmente exportar los resultados estandarizados, tal como se esquematiza en la Figura 3.3.1.

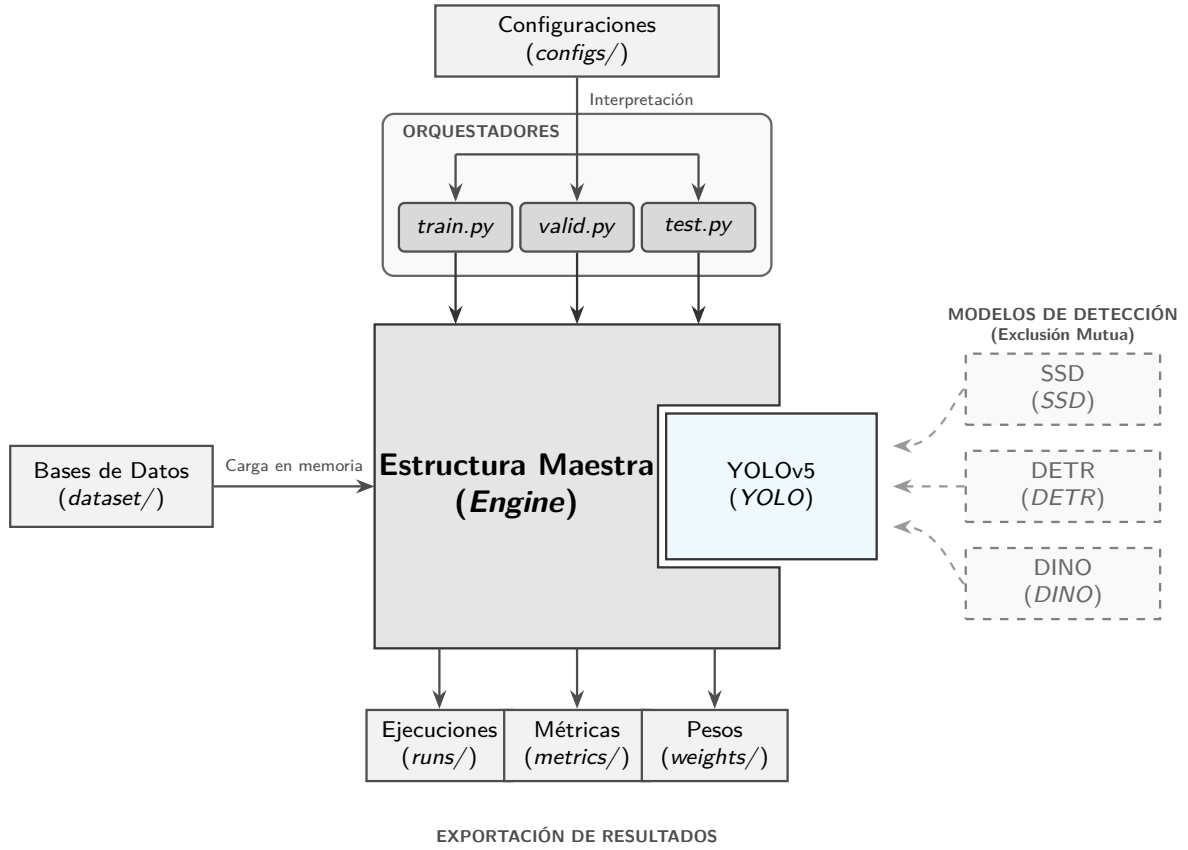


Figura 3.3.1: Topología modular *single-socket* implementada. Los orquestadores interpretan las configuraciones y dictan las directrices operativas a la estructura maestra, cargando las Bases de Datos y entregando los resultados requeridos tras pasar por las etapas internas de entrenamiento y validación. Sólo una arquitectura (o rama) se acopla a la vez.

3.3.1. Anatomía de la Estructura Maestra

La Estructura Maestra o Motor (*engine*) se compone de diversas partes internas que permiten la vinculación entre el modelo de detección de objetos que se entrenará y utilizará, con las configuraciones y base de datos de fallas en correas transportadoras. Esta arquitectura se implementa en todos los modelos de detección de objetos que resuelven el problema a abordar para el presente trabajo de memoria: *detección de fallas en correas transportadoras*. En la Figura 3.3.2 se muestra el esquema detallado y los diversos bloques y componentes esenciales para la implementación de los modelos de detección de objetos.

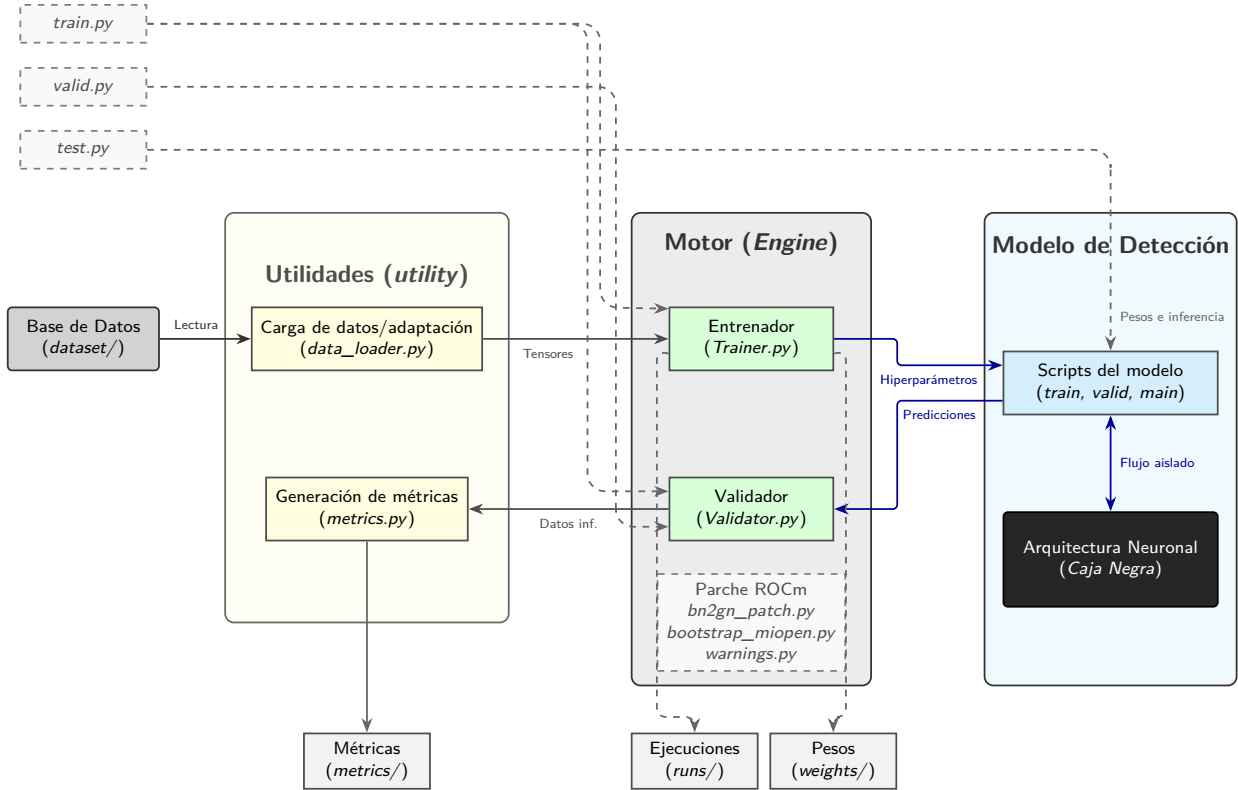


Figura 3.3.2: Vista detallada (*zoom*) de la interacción interna. La arquitectura neuronal opera como una caja negra intocable (no se modifica). Los 3 módulos principales se conectan mediante los dos ejecutores principales: Entrenador (*Trainer*) y Validador (*Validator*)

A continuación, se expone el flujo resumido desde el accionamiento de la rutina (entrenamiento, validación o prueba), la interacción entre las diversas partes del esquema y la obtención de métricas y resultados:

1. Primero, se acciona el Orquestador Principal, según el tipo de rutina a ejecutar (entrenamiento, validación o pruebas).
2. En el caso de entrenar por primera vez:
 - a) El orquestador de entrenamiento (*train.py*) carga las configuraciones (*configs/*), las cuales contienen las variantes del modelo, los hiperparámetros asociados, rutas de descarga de resultados y las rutas principales hacia los códigos del modelo (todo esto guardado en el archivo de configuración *train.yaml*).
 - b) El motor (*engine*), en particular la componente Entrenador (*Trainer.py*), mediante la utilidad de Carga de datos y/o adaptación (*data_loader.py*), carga en memoria la Base de datos de fallas en correas transportadoras con las etiquetas y formato solicitados por el modelo específico (*dataset.yaml*). Se utiliza la partición de entrenamiento y validación de la Base de Datos.

- c) Tras cargar los datos e hiperparámetros necesarios, invoca las clases y funciones de entrenamiento del código principal del modelo original (*train, main, etc*) y coloca en memoria los datos introducidos. Según el tamaño de los lotes definidos (*batch_size*), este traspaso de imágenes a la arquitectura neuronal será secuencial y con una fracción de imágenes (con el fin de evitar colapso en la memoria).
- d) Dado que el modelo no ha sido modificado como tal, la arquitectura neuronal asociada al modelo de detección de objetos está intacta y solamente es modificada externamente mediante los hiperparámetros introducidos, definiendo así la variante del modelo. Estos datos de ejecución al iniciar el modelo se guardarán en la carpeta de ejecuciones (*runs*), mediante un archivo de hiperparámetros (*hyp.yaml*), el cual contiene un resumen de todas las configuraciones usadas en el entrenamiento actual del modelo.
- e) El modelo comienza a entrenar y cada época lograda, las predicciones del modelo son derivadas a la componente Validador (*Validator.py*), quien valida mediante el uso de la partición de validación de la Base de Datos, donde se obtienen solamente métricas preliminares de seguimiento para obtener las pérdidas de entrenamiento y validación. Esto permite un análisis de convergencia y evaluar si existe sobreajuste (*overfitting*) ó bajoajuste (*underfitting*), lo cual permite establecer si el modelo está aprendiendo/memorizando o no.
- f) Tras entrenar todas las épocas definidas en los hiperparámetros, se entregarán las métricas finales de entrenamiento en la carpeta de métricas (*metrics/*). Así mismo, guardará en la carpeta de pesos *weights/*, los parámetros de entrenamiento asociados a los pesos y sesgos del modelo de aprendizaje.

3. En el caso de validar un modelo entrenado:

- a) El orquestador de validación (*valid.py*) carga las configuraciones (*configs/*), similarmente a el entrenamiento, pero usando un archivo diferente de configuración (*valid.yaml*).
- b) El motor (*engine*), en particular la componente Validador (*Validator.py*), mediante la utilidad de Carga de datos y/o adaptación (*data_loader.py*), carga en memoria la Base de datos de fallas en correas transportadoras con las etiquetas y formato solicitados por el modelo específico (*dataset.yaml*). Se utiliza exclusivamente la partición de validación de la Base de Datos.
- c) Tras cargar los datos e hiperparámetros necesarios, invoca las clases y funciones de entrenamiento del código principal del modelo original (*valid, main, etc*) y coloca en memoria los datos introducidos. Se utiliza el mismo tamaño de lotes que en entrenamiento.
- d) El modelo comienza a validarse durante una sola época.
- e) Tras validarse, guarda solamente en la carpeta de métricas (*metrics/*) las métricas de validación finales asociadas a la confianza y diversos resultados estandarizados para

comparar posteriormente con otras variantes u modelos. Además, toma 50 imágenes de muestra de la partición de validación y aplica inferencia en el modelo (dibuja las cajas delimitadoras reales y predichas), entregando muestras de predicción para análisis manual.

4. En caso de querer probar el modelo, mediante una interfaz personalizada, ya sea en vivo ó como simulación:
 - a) El orquestador de pruebas (*test.py*) carga las configuraciones (*configs/*), usando el archivo de configuración (*valid.yaml*).
 - b) En este caso, no se requiere obtener métricas, solo usar los pesos del modelo entrenado y validado que se encuentran en la carpeta de pesos (*weights/*) para realizar solo inferencia en imágenes que se pasen al modelo de detección de objetos directamente. Además, este orquestador posee una interfaz personalizada para visualizar clases detectadas, cambiar imagen de forma interactiva y ocultar/mostrar inferencias en tiempo real.
5. Como función adicional, la utilidad de métricas (*metrics.py*) permite evaluar las métricas de múltiples variantes del modelo actual y fusionarlas (*merge*) para tener resultados comparativos directos y más completos, incluyendo además métricas de cómputo matemático, numero de millones de parámetros de cada modelo y valores máximos de métricas obtenidas.

Por último, cabe destacar que cada una de las ramas independientes de trabajo asociadas a cada modelo: *YOLO*, *SSD*, *DETR*, *DINO*, poseen la última versión de trabajo en la rama principal *main*.

3.4. Implementación de modelos One-Stage (YOLOv5 y SSD)

La implementación de los modelos YOLOv5 y SSD se rigen de acuerdo a la *Estructura Maestra* definida en la Sección 3.3. Cabe destacar que cada modelo se trabaja en diferentes ramas del repositorio [20], particularmente las ramas *YOLO* y *SSD*. Los hiperparámetros comunes utilizados para estandarizar entrenamiento y métricas entre los dos modelos mencionados se muestran en la Tabla 3.4.1 .

Tabla 3.4.1: Hiperparámetros base estandarizados para el entrenamiento comparativo de los modelos YOLOv5 y SSD. Ambos incorporan el optimizador de Descenso de Gradiente Estocástico (*Stochastic Gradient Descent*).

<i>Hiperparámetro</i>	<i>Valor</i>	<i>Descripción</i>
Épocas (<i>epochs</i>)	300	Ciclos totales de entrenamiento sobre el dataset.
Lote (<i>batch_size</i>)	16	Muestras procesadas simultáneamente por iteración.
Optimizador (<i>optimizer</i>)	SGD	Algoritmo de minimización de pérdida.

Los hiperparámetros utilizados para cada una de las variantes del modelo de detección de objetos YOLOv5, se encuentran en Anexos 6.1, en la Tabla 6.1.1. El modelo cuenta con 5 variantes asociadas al tamaño del modelo de acuerdo a lo descrito en la Sección 2.5.3. De esta forma, para cambiar de variante, se ingresó como hiperparámetro: *variant -n* (ejemplo para el caso de la variante nano); de forma análoga se declaran el resto de variantes.

Los hiperparámetros utilizados para cada una de las variantes del modelo de detección de objetos SSD, se encuentran en Anexos 6.1, en la Tabla 6.1.2. El modelo cuenta con 2 variantes asociadas al tamaño de imagen de entrada de acuerdo a lo descrito en la Sección 2.5.4. De esta forma, para cambiar de variante, se ingresó como hiperparámetro: *variant -ssd300* (ejemplo para el caso de la variante SSD300); de forma análoga se declaran el resto de variantes.

3.5. Implementación de modelos basados en Transformers (DETR y DINO)

La implementación de los modelos DETR y DINO se rigen de acuerdo a la *Estructura Maestra* definida en la Sección 3.3. Cabe destacar que cada modelo se trabaja en diferentes ramas del repositorio [20], particularmente las ramas *DETR* y *DINO*. Los hiperparámetros comunes utilizados para estandarizar entrenamiento y métricas entre los dos modelos mencionados se muestran en la Tabla 3.5.1 .

Tabla 3.5.1: Hiperparámetros base estandarizados compartidos por las arquitecturas Transformer (DETR y DINO).

<i>Hiperparámetro</i>	<i>Valor</i>	<i>Descripción</i>
Épocas (<i>epochs</i>)	300	Ciclos totales de entrenamiento sobre el dataset.
Lote (<i>batch_size</i>)	4 ó 2	Muestras procesadas simultáneamente por iteración.
Optimizador	AdamW	Minimización con decaimiento adaptativo.
Tasa de aprendizaje (<i>lr</i>)	0.0001	Tasa base del bloque Transformer.
Tasa de aprendizaje del extractor (<i>lr_backbone</i>)	1.0e-05	Tasa reducida para el extractor visual.
Dcaimiento (<i>lr_drop_epochs</i>)	[150, 250]	Épocas de reducción escalonada de tasa.
Dimensión oculta (<i>hidden_dim</i>)	256	Canales de representación del espacio latente.
Capas Encoder-Decoder (<i>enc_layers / dec_layers</i>)	6 / 6	Profundidad de codificador y decodificador.
Consultas (<i>num_queries</i>)	100	Ranuras de predicción procesadas en paralelo.

Los hiperparámetros utilizados para cada una de las variantes del modelo de detección de objetos DETR se encuentran en Anexos 6.1, en la Tabla 6.1.3. El modelo cuenta con 4 variantes asociadas a la profundidad del extractor convolucional (*backbone*) y la implementación de convoluciones dilatadas (DC5), de acuerdo a lo descrito en la Sección 2.6. De esta forma, para cambiar de variante, se ingresó como hiperparámetro: *variant -r50* (ejemplo para el caso de la variante estándar con ResNet-50); de forma análoga se declaran el resto de variantes.

Los hiperparámetros específicos para cada una de las variantes del modelo de detección de objetos DINO se detallan en Anexos 6.1, en la Tabla 6.1.4. El modelo despliega 3 variantes asociadas a la cantidad de niveles de características (4 o 5 escalas) y a la topología del *backbone* (ResNet o Swin Transformer). De esta forma, para cambiar de variante, se ingresó como hiperparámetro: *variant -r50_4scale* (ejemplo para la variante ResNet-50 de 4 escalas); de forma análoga se declaran las arquitecturas restantes.

Capítulo 4

Resultados y Discusión

4.1. Base de datos implementada

A continuación, se exponen un resumen de las propiedades de la base de datos confeccionada, lo cual se muestra en la Tabla 4.1.1.

Tabla 4.1.1: Resumen de las propiedades y distribución del conjunto de datos.

Subconjunto	Total Imágenes	Total Etiquetas
Entrenamiento (Train)	1621	3755
Validación (Valid)	461	1124
Prueba (Test)	230	491
Total	2312	5370

A partir del análisis cuantitativo de la base de datos, se generaron dos representaciones gráficas para caracterizar su composición estructural. La Figura 4.1.1 ilustra la distribución de frecuencia de las distintas clases de fallas, evidenciando un desbalance entre categorías predominantes y minoritarias. Este comportamiento responde principalmente a la disponibilidad de registros en las fuentes originales, los cuales ya poseen aumento de datos (*data augmentation*). Cabe destacar que se optó por no aplicar técnicas de submuestreo adicionales para forzar un balance artificial, dado que, ante la presencia de cruce multiclase (múltiples tipos de fallas coexistiendo en una misma imagen), la eliminación de muestras para equilibrar una categoría habría implicado una pérdida involuntaria de información valiosa para otras clases.

Por su parte, la Figura 4.1.2 detalla la organización de las particiones del conjunto de datos (Entrenamiento, Validación y Prueba), distinguiendo la proporción de imágenes que contienen anotaciones respecto a las imágenes de fondo (*background*) utilizadas como control negativo. Por último, en la Figura 4.1.3 se muestran algunas fallas representativas debidamente etiquetadas de la base de datos confeccionada.

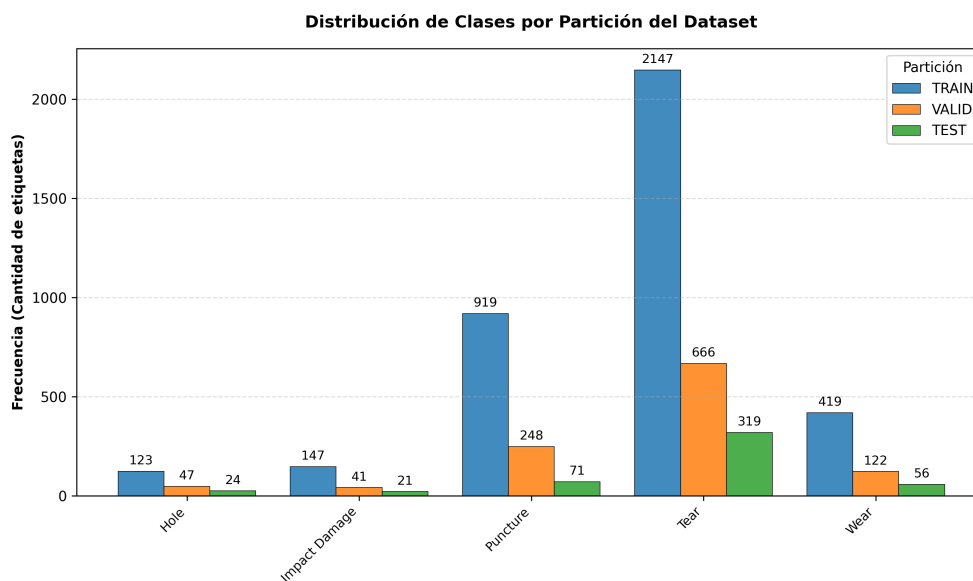


Figura 4.1.1: Distribución de frecuencia de las clases de fallas a través de las particiones del conjunto de datos.

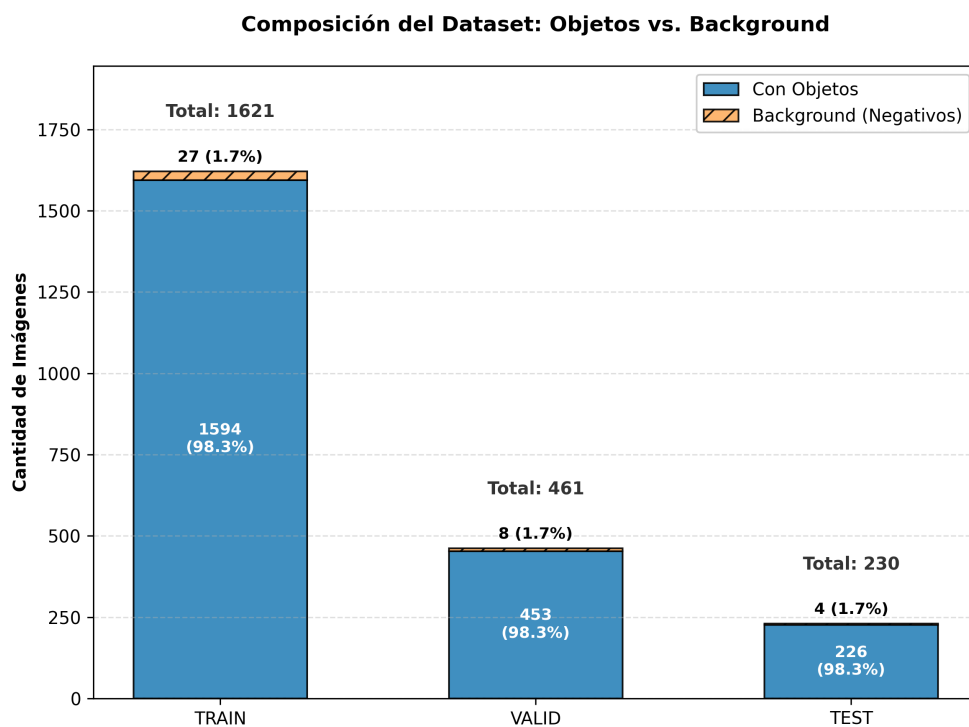
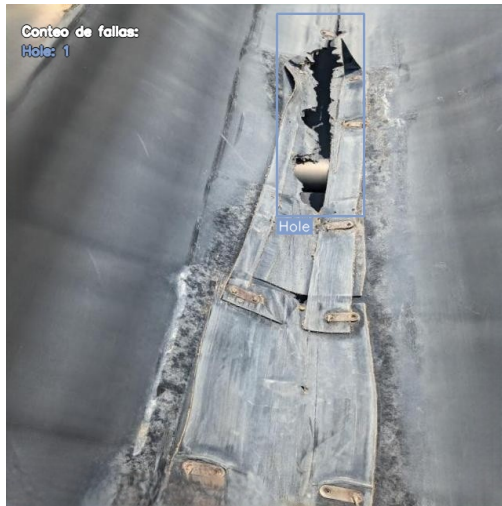


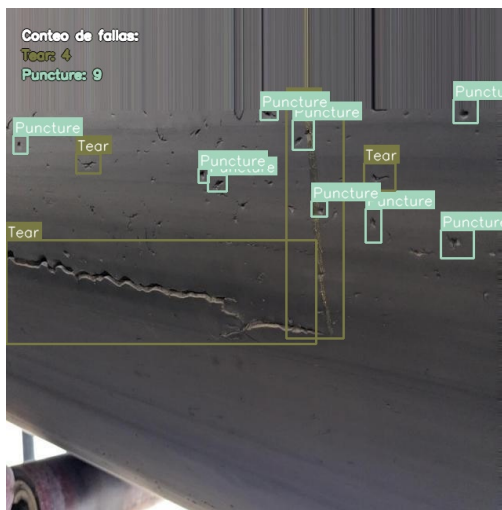
Figura 4.1.2: Composición detallada de las particiones del conjunto de datos, diferenciando entre imágenes con objetos y casos de fondo.



(a) Agujero (*Hole*)



(b) Daño por impacto (*Impact Damage*)



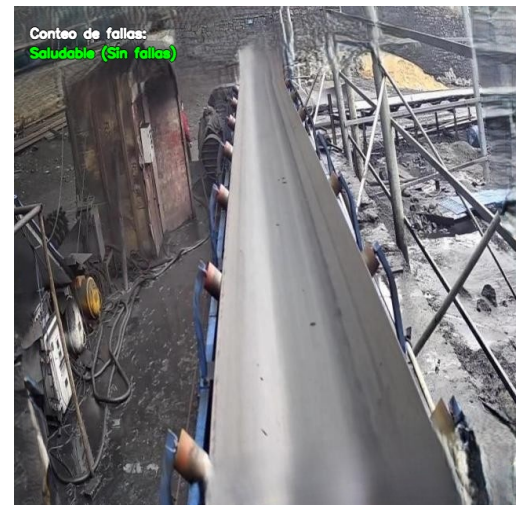
(c) Perforaciones (*Punctures*)



(d) Desgarro (*Tear*)



(e) Abrasión (*Wear*)



(f) Saludable (*Healthy*)

Figura 4.1.3: Muestras representativas de la base de datos confeccionada. Se exhiben las 5 fallas y el caso saludable con sus respectivas cajas delimitadoras y conteo de etiquetas.

4.2. Tiempos y rendimiento del entrenamiento

A continuación, se presenta un resumen del rendimiento computacional extraído durante la fase de entrenamiento. La Tabla 4.2.1 detalla la tasa de procesamiento por hora para cada variante arquitectónica y proyecta el tiempo total estimado para completar el régimen estandarizado de 300 épocas.

Tabla 4.2.1: Rendimiento computacional y estimación de tiempo total de entrenamiento (300 épocas) por variante.

Arquitectura	Variante	Rend. [épocas/h]	Tiempo Estimado [h]
YOLOv5	Nano (n)	80	3.75
	Small (s)	65	4.62
	Medium (m)	55	5.45
	Large (l)	40	7.50
	Extra Large (x)	25	12.00
SSD	SSD300	36	8.33
	SSD512	15	20.00
DETR	R50	22	13.64
	R50-DC5	18	16.67
	R101	12	25.00
	R101-DC5	7	42.86
DINO	R50-4scale	10	30.00
	R50-5scale	1	300.00
	Swin_L	2	150.00

Cabe destacar que las estimaciones de tiempo total presentadas no obedecen a un comportamiento estrictamente lineal. Estos valores fueron extrapolados a partir de observaciones de rendimiento periódicas muestreadas durante ventanas de varias horas de ejecución. Esta escalada en los tiempos de procesamiento representa el comportamiento teóricamente esperado: la transición desde arquitecturas convolucionales livianas de una sola etapa (*One-Stage*) hacia modelos basados en *Transformers* demanda un incremento sustancial en el consumo secuencial de GFLOPs, inherente a la complejidad de los mecanismos de atención.

A partir de los resultados expuestos en la Tabla 4.2.1, se evidencia una degradación drástica en el rendimiento computacional al transitar desde las variantes de DETR hacia las de DINO. Esta caída se atribuye a la saturación de la memoria de video dedicada (VRAM), lo que forzó al hardware a recurrir a la memoria RAM compartida del sistema (mecanismo de paginación o *swapping*).

4.3. Métricas y resultados de los modelos implementados

4.3.1. Resumen de resultados

A continuación, se muestra un resumen de resultados en la Tabla 4.3.1 de todos los modelos implementados, los cuales serán posteriormente discutidos y complementados con otras métricas.

Tabla 4.3.1: Resumen de métricas de rendimiento, complejidad paramétrica y costo computacional para los modelos YOLOv5, SSD, DETR y DINO.

Variante	mAP@0.5	mAP@0.5:0.95	F1-Score	Parám. (M)	GFLOPs
YOLOv5					
YOLOv5-n	0.8735	0.5089	0.8744	1.9	4.2
YOLOv5-s	0.8978	0.5517	0.8897	7.2	15.8
YOLOv5-m	0.8984	0.5480	0.8937	21.2	48.0
YOLOv5-l	0.8958	0.5359	0.8937	46.5	108.3
YOLOv5-x	0.9012	0.5435	0.8910	86.7	204.7
SSD					
SSD300	0.6856	0.3995	0.6954	26.3	31.4
SSD512	0.7394	0.4277	0.7424	27.1	90.8
DETR					
DETR-R50	0.8067	0.5033	0.6959	41.0	86.0
DETR-R50-DC5	0.7784	0.4713	0.6845	41.0	187.0
DETR-R101	0.7984	0.4914	0.6887	60.0	152.0
DETR-R101-DC5	0.7960	0.4878	0.6922	60.0	253.0
DINO					
DINO-R50-4scale	0.8809	0.5298	0.7550	47.0	279.0
DINO-R50-5scale	0.0123	0.0064	0.0225	47.0	860.0
DINO-Swin-L	0.8710	0.5240	0.7554	218.0	1040.0

4.3.2. YOLOv5 (*You Only Look Once*)

A continuación se muestran todos los resultados relevantes obtenidos para el modelo YOLOv5 y sus 5 variantes. Estos fueron obtenidos aplicando la metodología explicada en la Sección 3. Los mejores resultados numéricos fueron mostrados en la Tabla 4.3.1.

Pérdida Total vs Época (*Total Loss vs Epoch*)

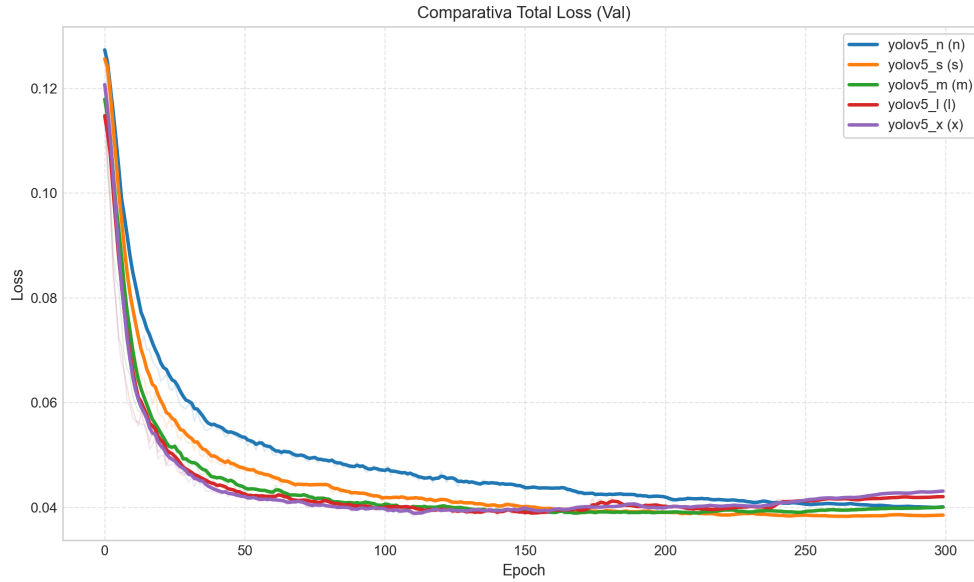


Figura 4.3.1: Pérdida total promedio vs época de la validación en cada variante del modelo YOLOv5.

En la Figura 4.3.1, la gráfica evidencia una convergencia estable para todas las variantes antes de la época 150. La variante *small* (s) converge hacia la mayor pérdida final de validación, además, se destaca al alcanzar y sostener el mínimo de pérdida global frente a las variantes paramétricamente más densas (*m*, *l*, *x*), las cuales muestran pérdidas ligeramente más altas en magnitud. Aun así, el comportamiento de convergencia temprana para las variantes más densas es mejor en términos gráficos.

Precisión Media Promedio vs Época (*Mean Average Precision vs Epoch*)

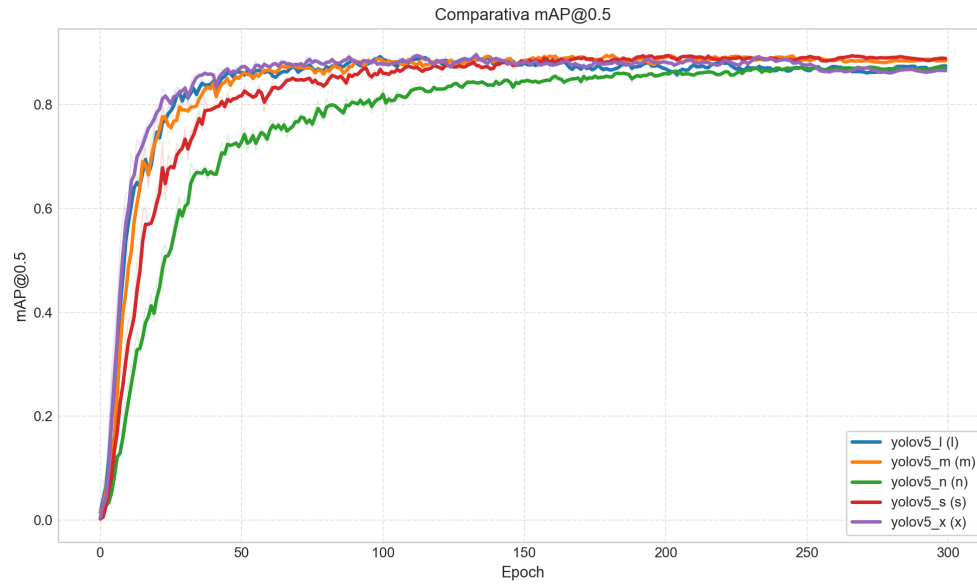


Figura 4.3.2: Precisión Media Promedio (mAP) vs época con umbral IoU 0,5 para todas las variantes del modelo YOLOv5.

A partir de la Figura 4.3.2, se observa un ascenso abrupto de la métrica durante las primeras 50 épocas. La variante *nano* exhibe un menor crecimiento, estabilizándose en un techo de 0.8735 y por debajo de las otras variantes en la convergencia temprana. El resto de las arquitecturas se agrupan asintóticamente en el límite superior, donde las variantes *small* (0.8978) y *medium* (0.8984) logran igualar el desempeño de la variante más densa *extra large*, la cual establece el máximo absoluto en 0.9012. Este comportamiento promedio es el esperado al considerar modelos más densos paramétricamente en términos de convergencia.

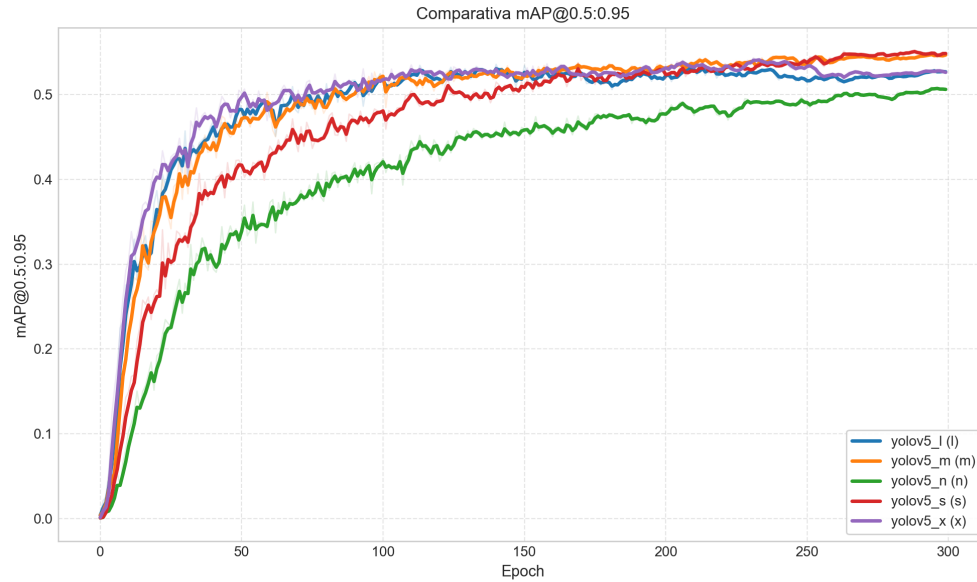


Figura 4.3.3: Precisión Media Promedio (mAP) vs época con umbral IoU desde 0,5 hasta 0,95 para todas las variantes del modelo YOLOv5.

Al analizar la Figura 4.3.3 bajo este umbral métrico más estricto, la separación arquitectónica es más pronunciada. Notablemente, la variante *small* (s) escala de manera continua a lo largo del entrenamiento, logrando superar el rendimiento final de las variantes *large* (0.5359) y *extra-large* (0.5435) hacia la época 300, consolidando el máximo óptimo entre las variantes con un valor de 0.5517.

F1-Score vs Época

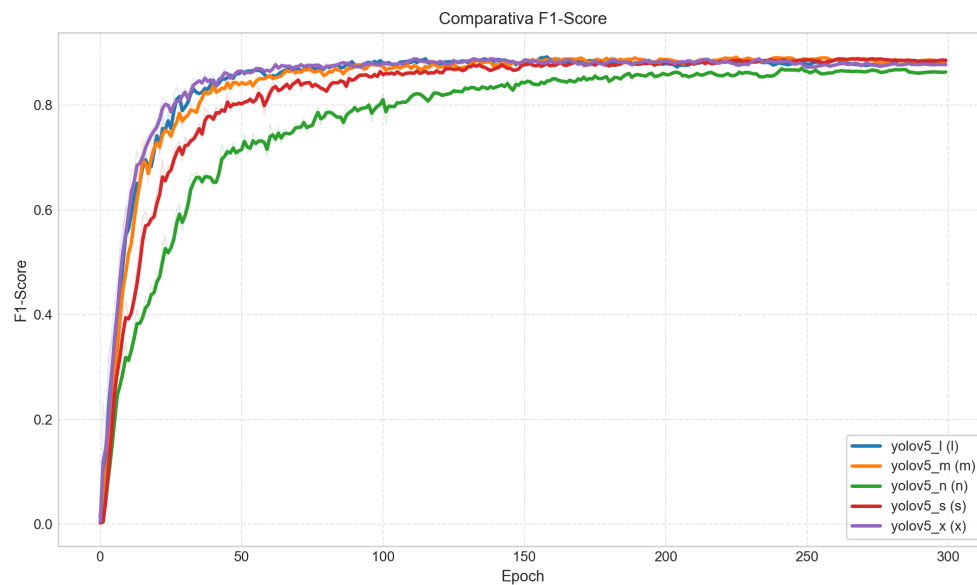


Figura 4.3.4: F1-Score vs época para todas las variantes del modelo YOLOv5.

La curva de la Figura 4.3.4 muestra el comportamiento entre precisión y exhaustividad (de acuerdo a la definición de la métrica), confirmando el límite de capacidad de la variante *nano* (0.8744). Las variantes *s*, *m*, *l* y *x* operan con rendimientos ajustados dentro de un estrecho margen, alcanzando el máximo de la métrica en la variante *medium* con un F1-Score de 0.8937.

Trade-off Performance vs Costo Computacional

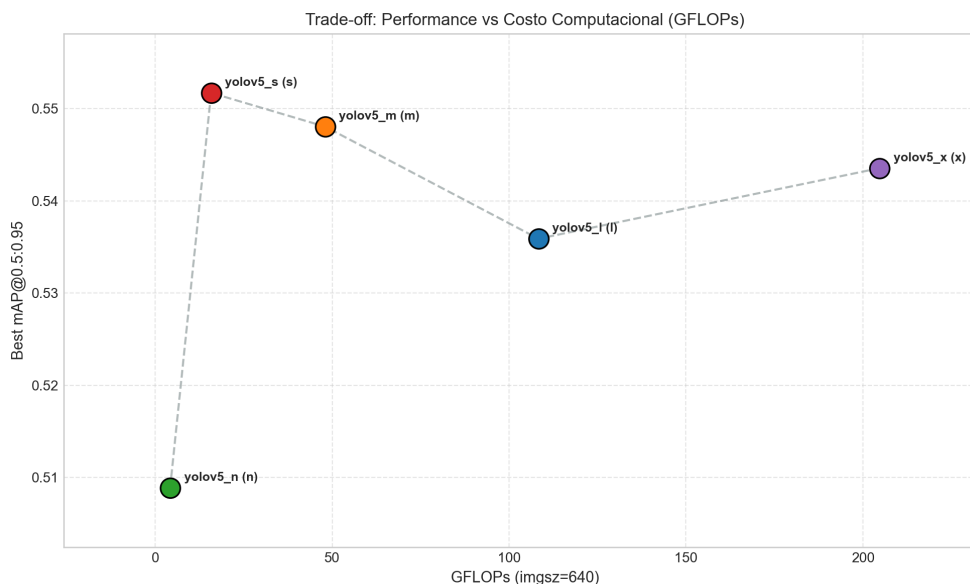


Figura 4.3.5: Trade-off de rendimiento entre todas las variantes del modelo YOLOv5. Muestra el rendimiento mAP@0.5:0.95 en función de la cantidad de GFlops de cómputo utilizado por la variante.

La Figura 4.3.5 determina gráficamente la viabilidad operativa de los modelos. En ella, la variante *small* (s) destaca al maximizar la eficiencia con una métrica mAP@0.5:0.95 de 0.5517 requiriendo un costo computacional marginal de solo 15.8 GFLOPs y 7.2M de parámetros. Escalar la red hacia la variante *extra large* (x) implica multiplicar drásticamente la carga de procesamiento matricial a 204.7 GFLOPs y 86.7M de parámetros. Si bien los resultados son similares, es posible discernir entre la variante más óptima considerando el cómputo y la cantidad de millones de parámetros, donde claramente se ve que la variante *small* (s), del modelo YOLOv5, es más eficiente. Por lo tanto, esta variante será considerada como representante para ser evaluada con otros modelos.

Resultados específicos mejor variante (*YOLOv5-s*)

Curva de F1-Score vs Confianza

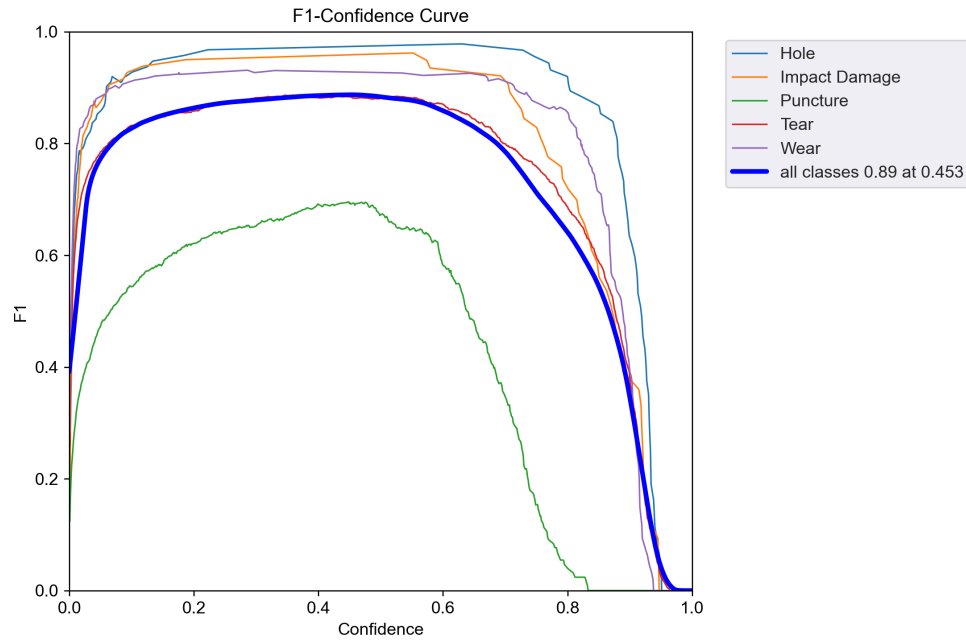


Figura 4.3.6: Curvas de F1-Score en función de la confianza (*confidence*) de detección de la variante *small* (s) de YOLOv5.

Como se observa en la Figura 4.3.6, el modelo global alcanza un pico óptimo de 0.89 bajo un umbral de confianza de 0.5 aproximadamente. Existe una brecha de rendimiento evidente: mientras las clases principales operan con alta eficiencia, la clase de perforación (*Puncture*) restringe la métrica global, exhibiendo un máximo deficiente cercano a 0.65. Si bien la clase no presenta un desbalance de acuerdo a lo discutido en la Sección 4.1, según la Figura 4.1.1, el tamaño de la falla y su multiplicidad puede ser la causa de una detección infructuosa y por ende la explicación de la métrica deficiente en torno a esa clase que está muy por debajo del promedio (ver curva azul en la Figura 4.3.6).

Curva de Precisión vs Confianza (*Precision Curve vs Confidence*)

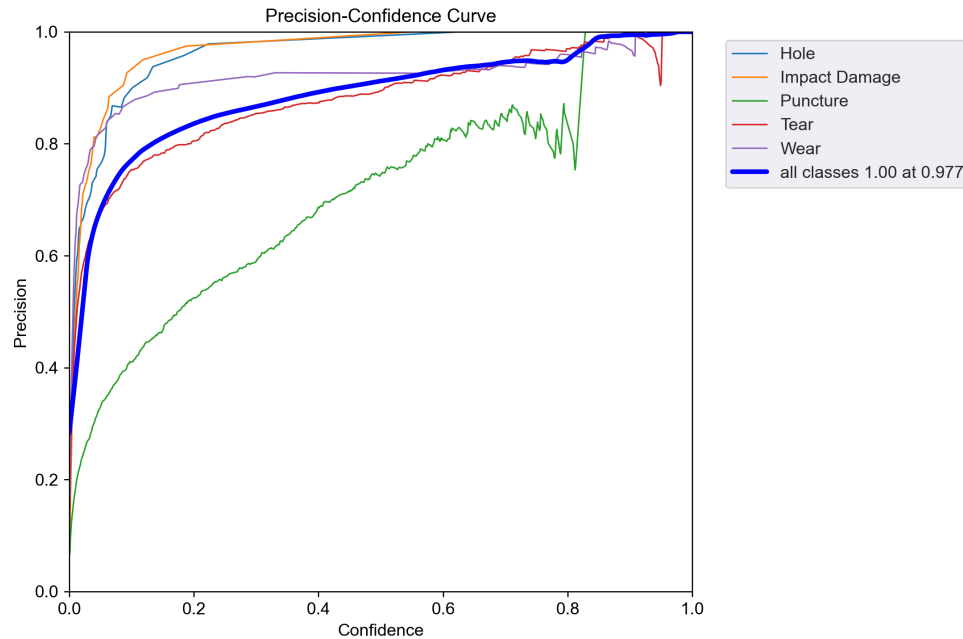


Figura 4.3.7: Curvas de Precisión en función de la confianza (*confidence*) de detección de la variante *small* (s) de YOLOv5.

De acuerdo a la Figura 4.3.7, la precisión global de la red converge al 100 % de asertividad recién en umbrales muy estrictos (por encima de 0.95). La clase de perforación (*Puncture*) evidencia gran inestabilidad en confianzas superiores a 0.7 (con un desplome de magnitud repentino hasta la confianza 0.8). Cabe destacar que la clase desgarró (*tear*) posee una precisión similar al promedio de todas las clases, a diferencia de las otras fallas que están por encima de este valor.

Curva de Exhaustividad vs Confianza (*Recall Curve vs Confidence*)

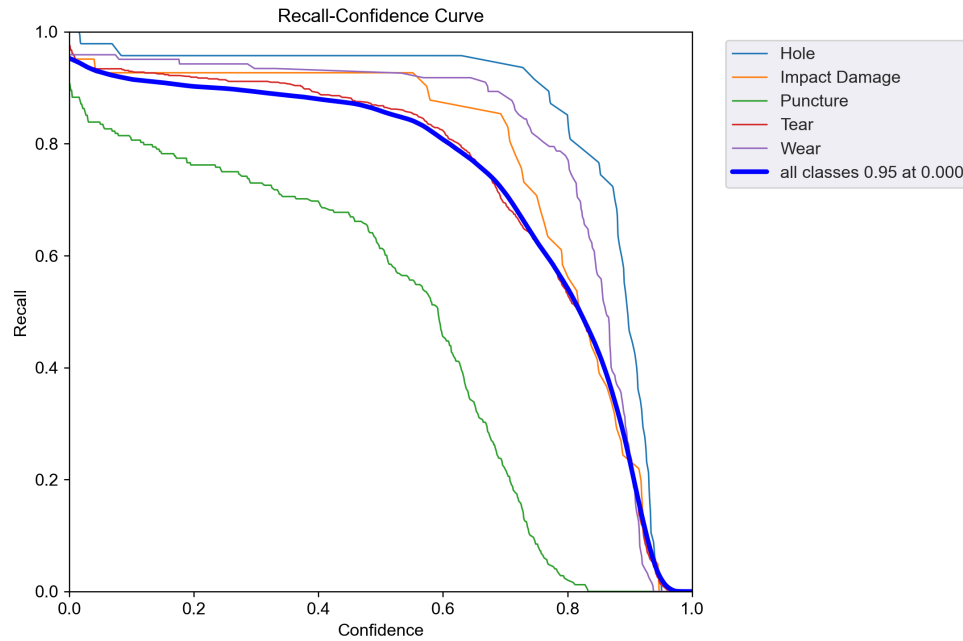


Figura 4.3.8: Curvas de Exhaustividad en función de la confianza (*confidence*) de detección de la variante *small* (s) de YOLOv5.

En la Figura 4.3.8, la exhaustividad (0.95 promedio) decae de forma progresiva. Notoriamente, la clase *Puncture* experimenta un desplome precipitado desde valores de confianza bajos, indicando severas dificultades. Esto puede atribuirse a la detección de falsos positivos en esta clase (lo cual es más probable debido al tamaño de la falla), lo que ocasiona detección del fondo (*background*) en vez de las perforaciones (*punctures*).

Matriz de confusión (*Confusion Matrix*)

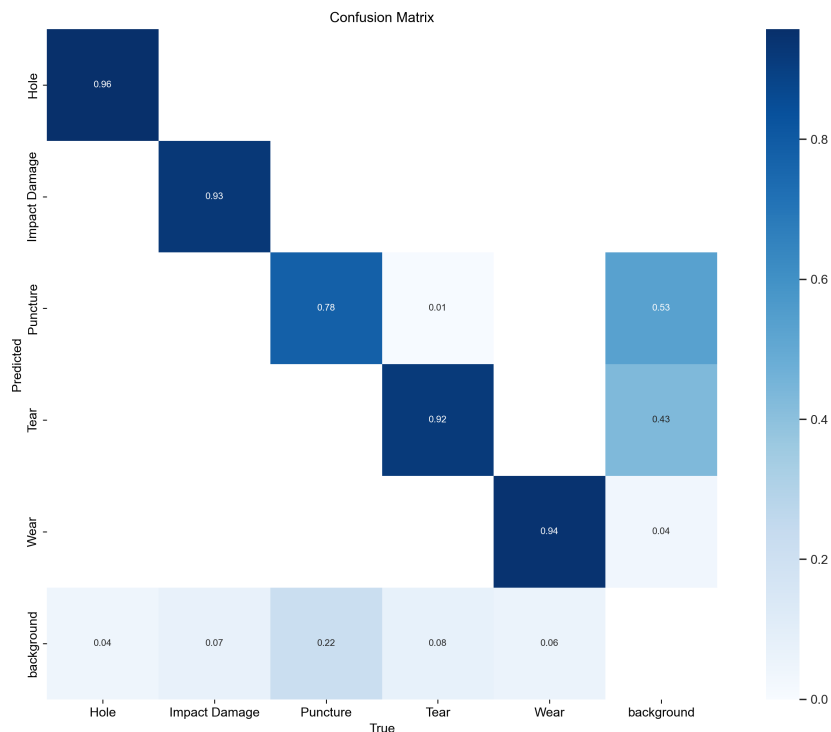


Figura 4.3.9: Matriz de confusión de la variante *small* (s) de YOLOv5.

La matriz presentada en la Figura 4.3.9 ratifica analíticamente la tendencia: el modelo extrae con alta fiabilidad las clases: agujero (*hole*) con un valor de 0.96; daño por impacto (*impact damage*) con un valor de 0.93; desgarró (*tear*) con un valor de 0.92; abrasión (*wear*) con un valor de 0.94. Al segmentar características con la clase perforación (*puncture*) se logra apenas un 0.78 de aciertos de muestras, mientras que el resto (0.22) son confundidas y absorbidas por el fondo (*background*).

4.3.3. SSD (*Single Shot Multibox Detector*)

A continuación se muestran todos los resultados relevantes obtenidos para el modelo SSD y sus 2 variantes. Estos fueron obtenidos aplicando la metodología explicada en la Sección 3. Los mejores resultados numéricos fueron mostrados en la Tabla 4.3.1.

Pérdida Total vs Época (*Total Loss vs Epoch*)

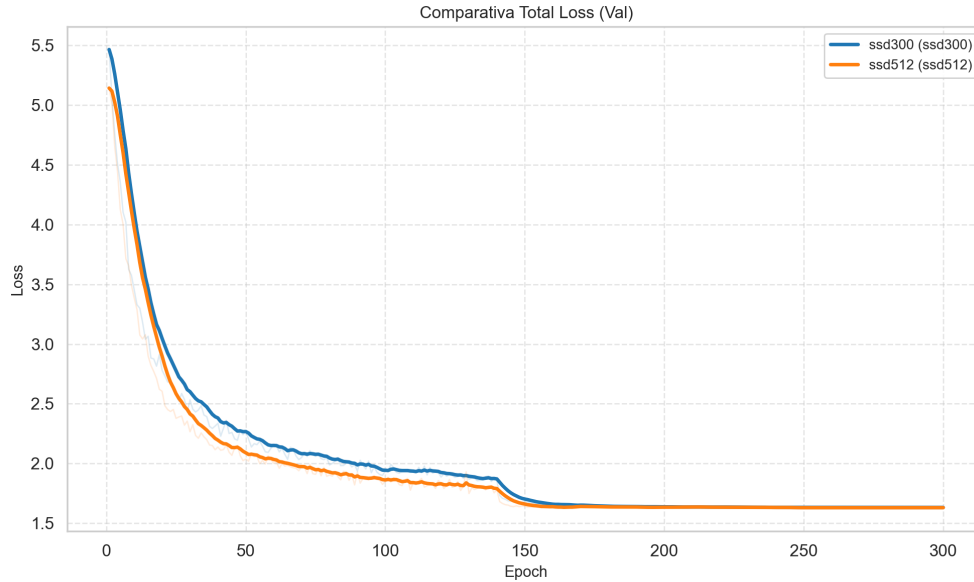


Figura 4.3.10: Pérdida total promedio vs época de la validación en cada variante del modelo SSD.

En la Figura 4.3.10, se observa una convergencia estable y acoplada para ambas arquitecturas. Se distingue un escalón descendente pronunciado cercano a la época 150, atribuible al decaimiento programado de la tasa de aprendizaje (*learning rate step*). La variante de mayor resolución (SSD512) logra mantener una pérdida de validación marginalmente inferior durante todo el proceso, aunque se vuelve indistinguible debido al solapamiento de ambas curvas tras la época 150.

Precisión Media Promedio vs Época (*Mean Average Precision vs Epoch*)

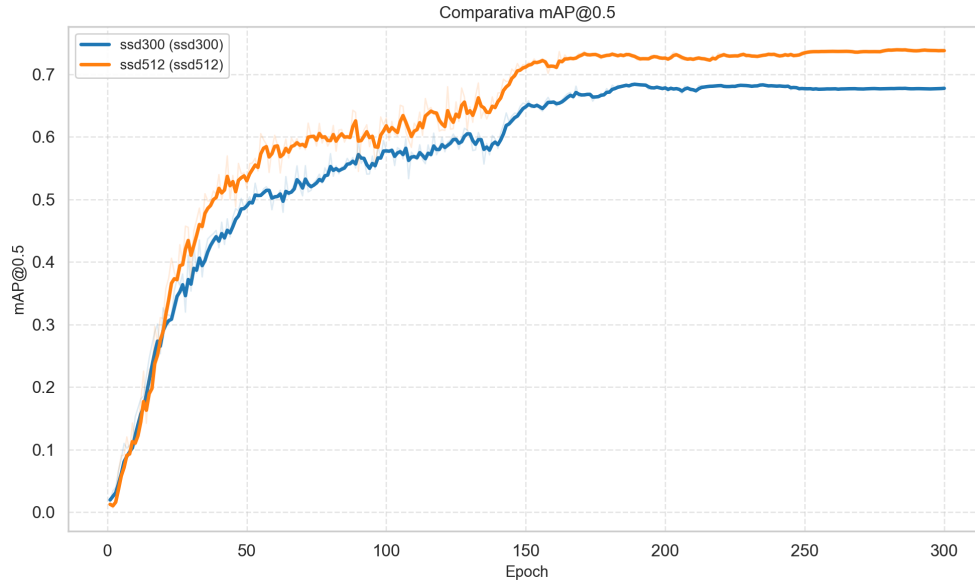


Figura 4.3.11: Precisión Media Promedio (mAP) vs época con umbral IoU 0,5 para todas las variantes del modelo SSD.

A partir de la Figura 4.3.11, es evidente la superioridad extractiva de la resolución ampliada de la variante SSD512. Al momento de superar la época 50 y después del reajuste del optimizador en la época 150, la curva de pérdida de la variante SSD512 supera a la curva de pérdida de la variante SSD300 en todo el entrenamiento, estableciendo un máximo de 0.7394, mientras que SSD300 se estanca prematuramente en un máximo de 0.6856.

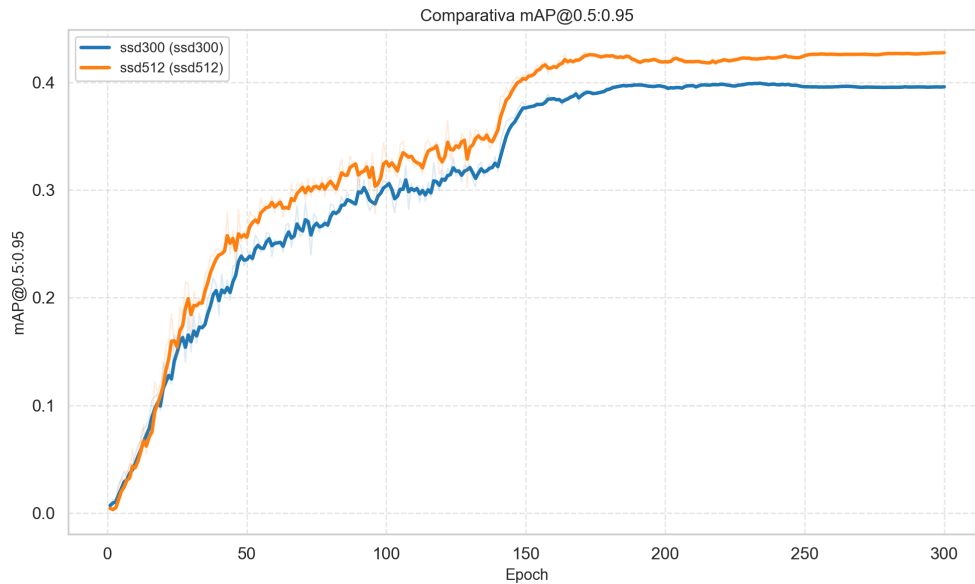


Figura 4.3.12: Precisión Media Promedio (mAP) vs época con umbral IoU desde 0,5 hasta 0,95 para todas las variantes del modelo SSD.

En la Figura 4.3.12, considerando el umbral IoU más estricto, el patrón de dominancia se repite respecto a la Figura 4.3.11. La variante SSD512 alcanza un óptimo absoluto de 0.4277, superando de forma constante el límite de 0.3995, lo cual es probablemente debido a las restricciones espaciales de la variante SSD300. Esta brecha de rendimiento se fundamenta en las restricciones espaciales de la arquitectura base: al operar con un tensor de entrada de menor resolución (300×300), las sucesivas operaciones convolucionales generan mapas de características menos refinados a diferencia de una entrada con mayor resolución (512×512).

F1-Score vs Época

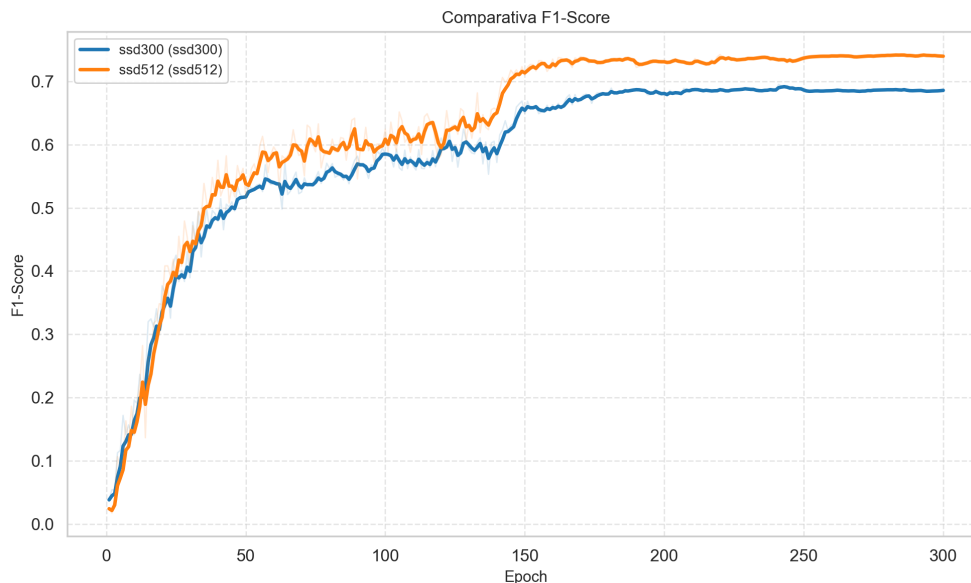


Figura 4.3.13: F1-Score vs época para todas las variantes del modelo SSD.

La curva de la Figura 4.3.13 confirma la variante superior del modelo SSD trabajando a mayor resolución. SSD512 logra un buen balance entre precisión y exhaustividad con un valor máximo de F1-score de 0.7424, manteniendo una brecha visible y constante respecto al 0.6954 obtenido por SSD300 (similar al comportamiento visto en la precisión media promedio).

Trade-off Performance vs Costo Computacional

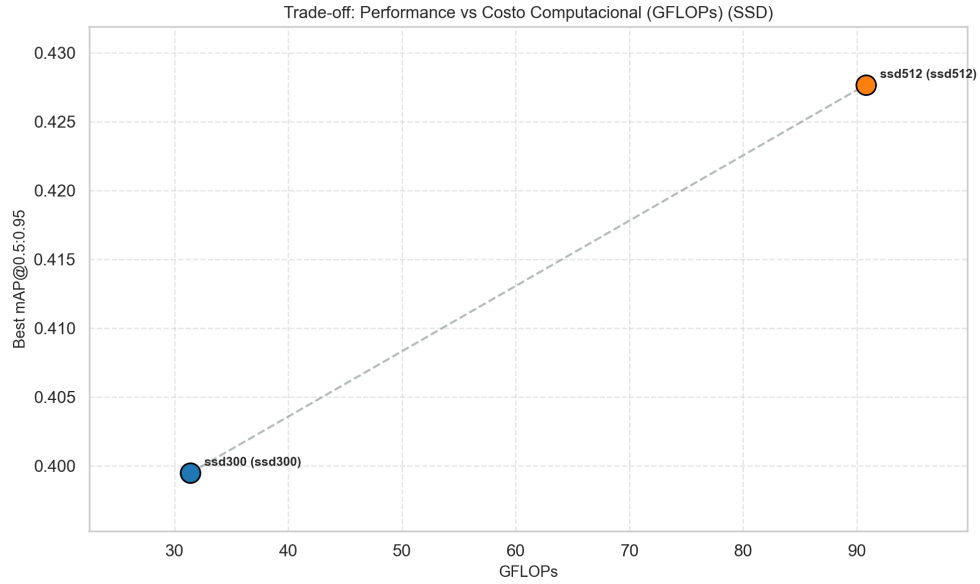


Figura 4.3.14: Tradeoff de rendimiento entre todas las variantes del modelo SSD. Muestra el rendimiento mAP@0.5:0.95 en función de la cantidad de GFlops de cómputo utilizado por la variante.

La Figura 4.3.14 ilustra el costo físico del incremento de rendimiento respecto a YOLOv5. Aunque la carga paramétrica es virtualmente idéntica de 26.3M frente a 27.1M (ver Tabla 4.3.1), procesar tensores de 512×512 exige triplicar la carga convolucional, elevando el costo computacional de 31.4 GFLOPs (SSD300) a 90.8 GFLOPs (SSD512) para asegurar el salto en precisión. Aun así, la variante SSD512 será considerada para analizar su desempeño frente a otros modelos.

Resultados específicos mejor variante (*SSD512*)

Curva de F1-Score vs Confianza

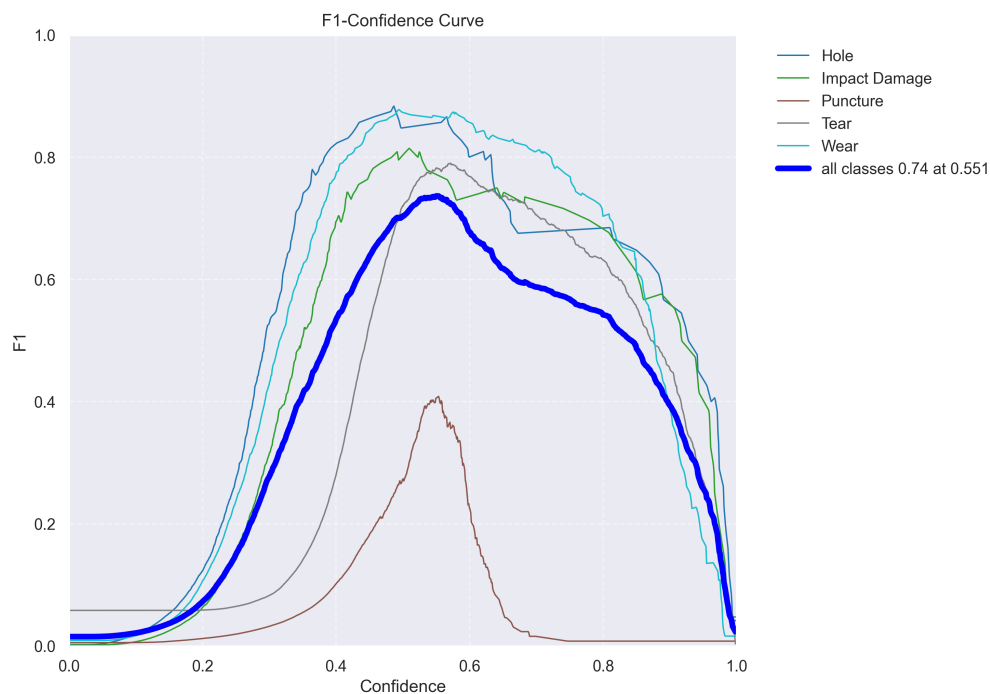


Figura 4.3.15: Curvas de F1-Score en función de la confianza (*confidence*) de detección de la variante SSD512.

Como se observa en la Figura 4.3.15, el modelo global alcanza su máximo (0.74) bajo un umbral de confianza promedio (0.55 aproximadamente). Existe una notoria disparidad por clase: las fallas prominentes dominan en magnitud de F1-score, mientras que la clase perforación (*Puncture*) arrastra la métrica global hacia abajo, exhibiendo un pico máximo deficiente cercano a 0.4 en el umbral de confianza promedio.

Curva de Precisión vs Confianza (*Precision Curve vs Confidence*)

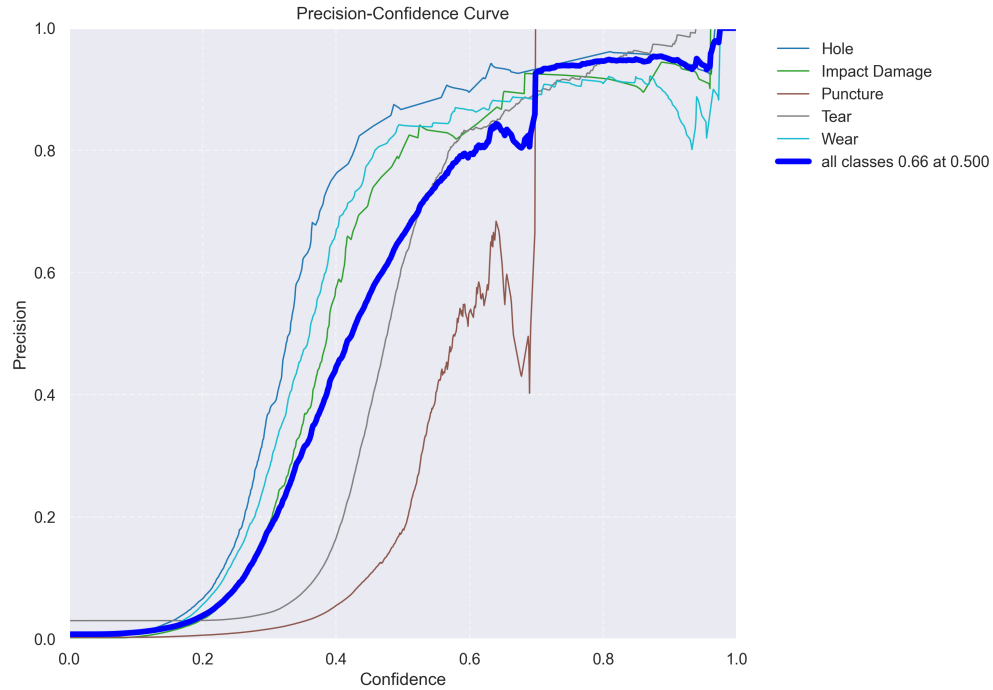


Figura 4.3.16: Curvas de Precision en función de la confianza (*confidence*) de detección de la variante SSD512.

De acuerdo a la Figura 4.3.16, la arquitectura experimenta severas dificultades para suprimir falsos positivos, requiriendo umbrales de confianza extremos (> 0.95) para alcanzar el 100 % de asertividad global. La clase perforación (*puncture*) exhibe una oscilación sumamente inestable a lo largo de toda la distribución, alterando el promedio global.

Curva de Exhaustividad vs Confianza (*Recall Curve vs Confidence*)

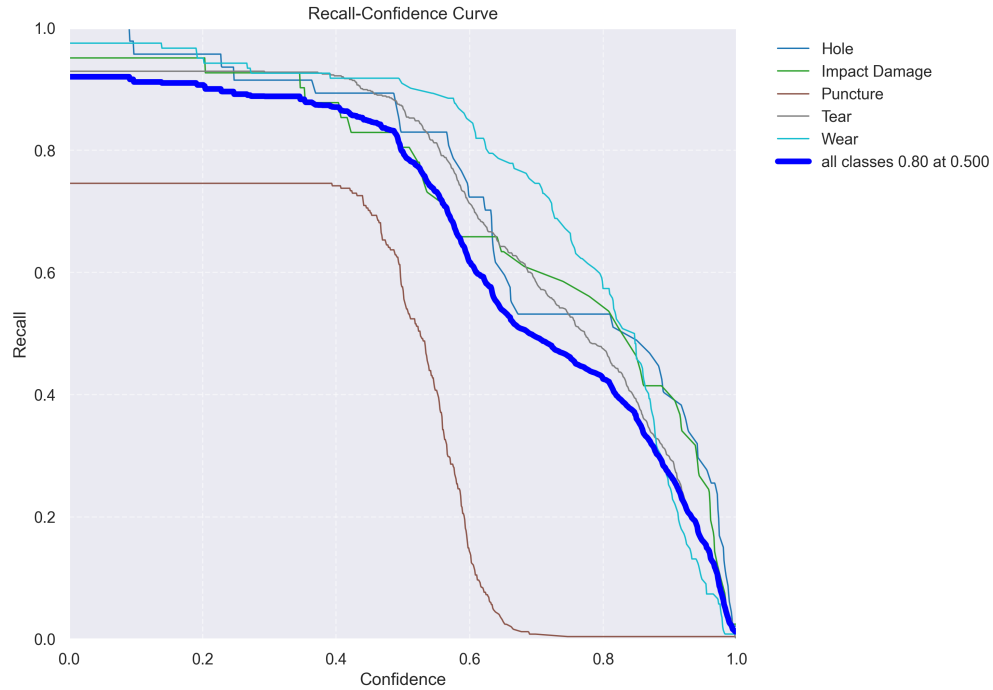


Figura 4.3.17: Curvas de Exhaustividad en función de la confianza (*confidence*) de detección de la variante SSD512.

En la Figura 4.3.17, los falsos positivos respecto a la clase perforación (*puncture*) se evidencian claramente.. La red pierde detecciones verdaderas al incrementar la confianza, con la clase perforación (*puncture*) desplomándose radicalmente a partir del umbral de confianza 0.4. El resto de clases muestran un comportamiento acorde a lo visto en otros modelos.

Matriz de confusión (*Confusion Matrix*)

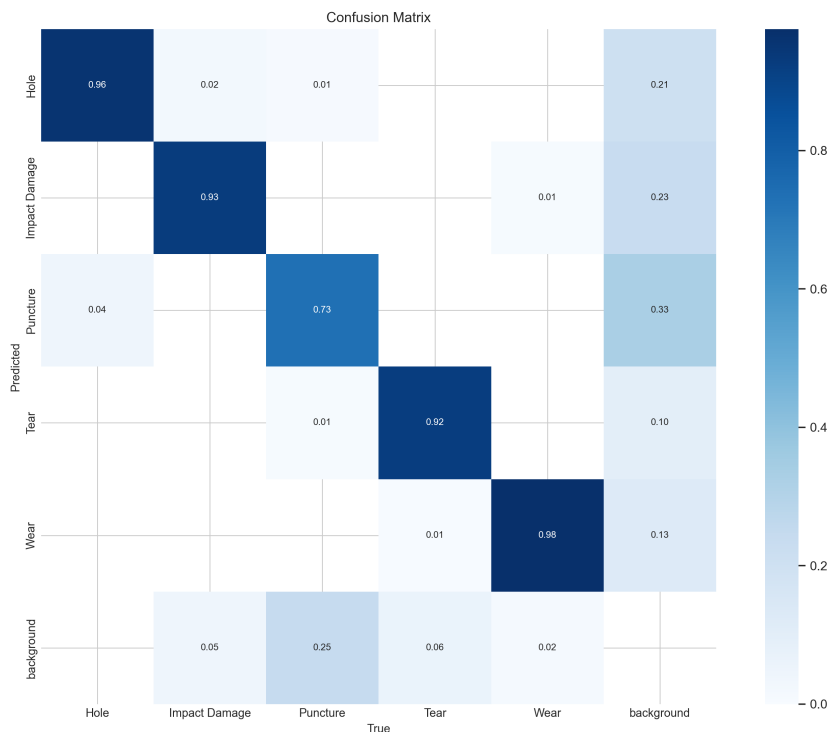


Figura 4.3.18: Matriz de confusión de la variante SSD512.

La matriz presentada en la Figura 4.3.18 ratifica la tendencia: el modelo posee buenos valores de predicción para las clases: agujero (*hole*) con un valor de 0.96; daño por impacto (*impact damage*) con un valor de 0.93; desgarró (*tear*) con un valor de 0.92; abrasión (*wear*) con un valor de 0.98. Sin embargo, falla drásticamente al segmentar características sutiles, con la clase perforación (*puncture*) logrando apenas un valor de 0.73, mientras que el resto (0.27) de muestras fueron absorbidas por la clase fondo (*background*). Por ende, el modelo obtuvo una mayor cantidad de falsos positivos respecto a YOLOv5 y su variante *small* (s).

4.3.4. DETR (*DE*tecti*on TR*ansformer)

A continuación se muestran todos los resultados relevantes obtenidos para el modelo DETR y sus 4 variantes. Estos fueron obtenidos aplicando la metodología explicada en la Sección 3. Los mejores resultados numéricos fueron mostrados en la Tabla 4.3.1.

Pérdida Total vs Época (*Total Loss vs Epoch*)

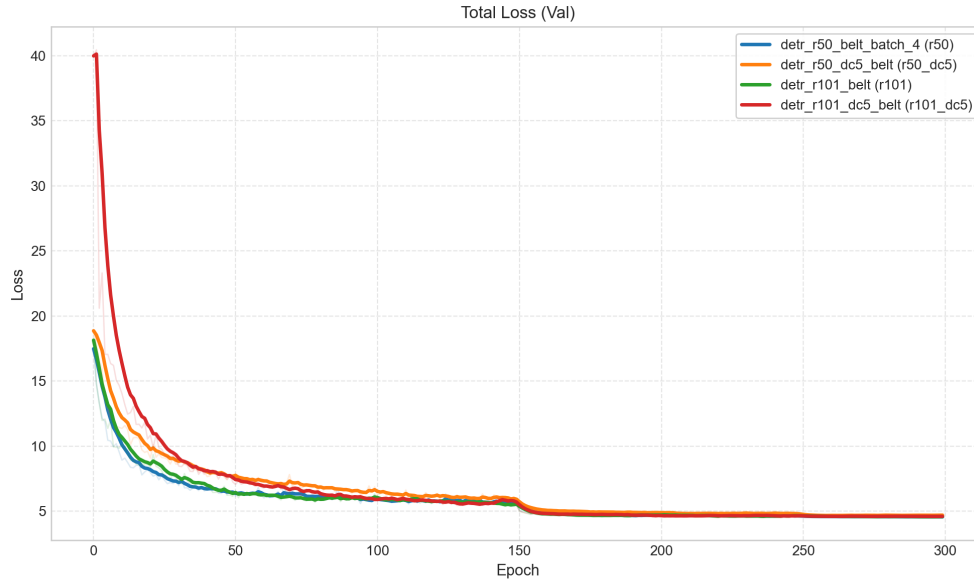


Figura 4.3.19: Pérdida total promedio vs época de la validación en cada variante del modelo DETR.

En la Figura 4.3.19, se aprecia un descenso drástico inicial seguido de una convergencia progresiva. Se evidencia un escalón de optimización pronunciado en la época 150, correspondiente al decaimiento programado de la tasa de aprendizaje (*lr drop*). La variante base *ResNet-50* (R50) logra estabilizarse en el nivel de pérdida más bajo en el entrenamiento temprano, aunque posteriormente todas las variantes se estabilizan con pérdidas bajas similares en torno a una magnitud de pérdida total menor a 5. Se puede observar también que la variante *ResNet-101-DC5* (R101-DC5) comienza con una pérdida muy elevada respecto a las otras variantes, lo cual es debido al uso de una mayor cantidad de parámetros (60.0M) y el uso de la modificación DC5, provocando una menor convergencia temprana de gradientes.

Precisión Media Promedio vs Época (*Mean Average Precision vs Epoch*)



Figura 4.3.20: Precisión Media Promedio (mAP) vs época con umbral IoU 0,5 para todas las variantes del modelo DETR.

A partir de la Figura 4.3.20, se puede ver que la escalada del rendimiento tras el ajuste del optimizador (época 150) es notoria (la curva se suaviza y se estabiliza). La variante estándar *ResNet-50* (R50) y la variante *ResNet-101* (R101) dominan la métrica, alcanzando una magnitud pico de 0.8067 y de 0.7984 respectivamente. La inclusión de la modificación (DC5) a las variantes base mencionadas antes, muestran un mal desempeño inicial e intermedio, a pesar de una convergencia pareja al final del entrenamiento junto a las otras variantes. Todas las variantes presentan una tendencia de inestabilidad al comienzo del entrenamiento, la cual se reduce drásticamente cuando llegan a la época 150, debido al decaimiento programado de la tasa de aprendizaje.

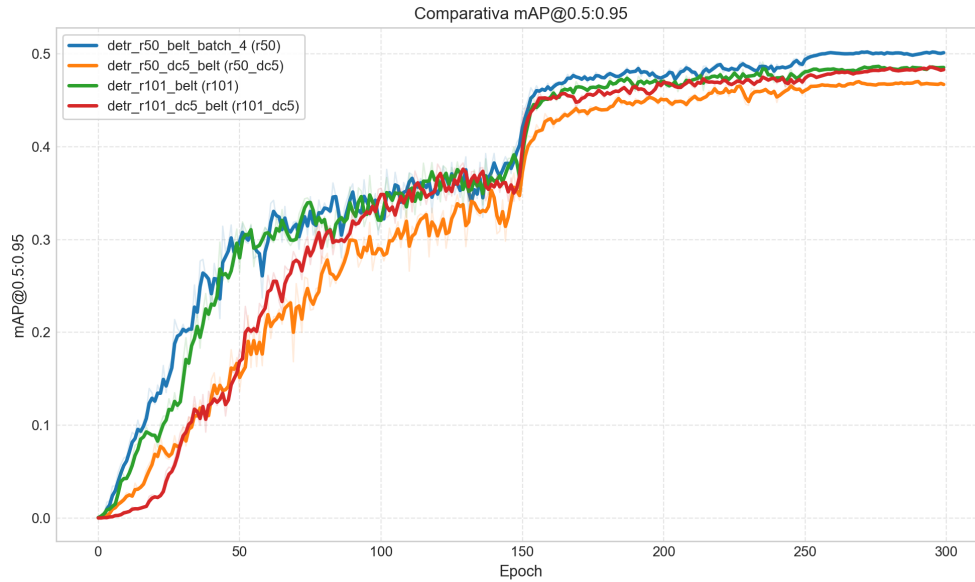


Figura 4.3.21: Precisión Media Promedio (mAP) vs época con umbral IoU desde 0,5 hasta 0,95 para todas las variantes del modelo DETR.

Al analizar la Figura 4.3.21 bajo un criterio de IoU más estricto, la variante *ResNet-50* (R50) obtiene el máximo global respecto a las otras variantes con un valor de 0.5033. Las otras variantes y/o modificaciones fracasan en superar el valor de 0.5. Al igual que el gráfico mostrado anteriormente, las variantes presentan una tendencia de inestabilidad al comienzo del entrenamiento, la cual se reduce drásticamente cuando llegan a la época 150, debido al decaimiento programado de la tasa de aprendizaje.

F1-Score vs Época

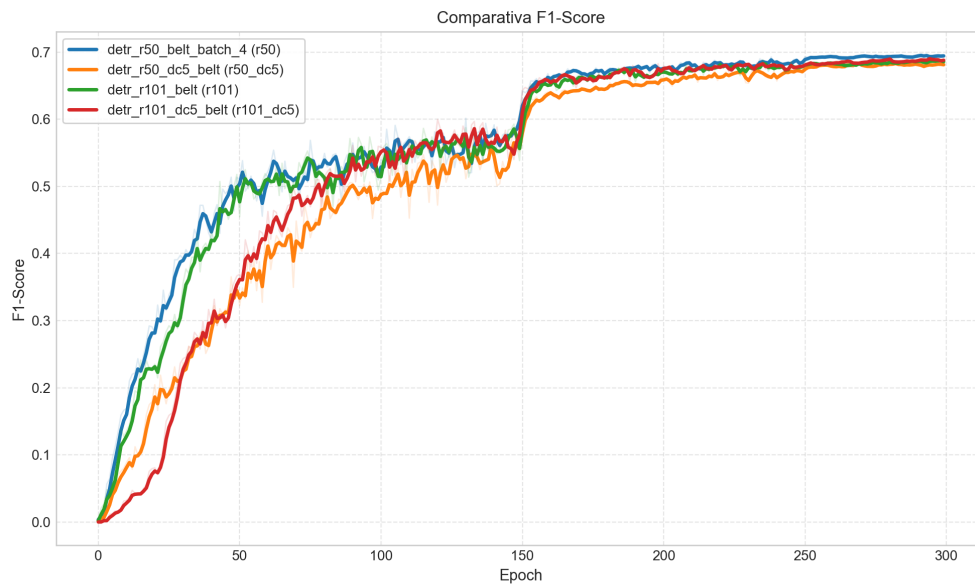


Figura 4.3.22: F1-Score vs época para todas las variantes del modelo DETR.

La curva de la Figura 4.3.22 confirma la tendencia global. El comportamiento entre precisión y exhaustividad alcanza un mayor valor con la variante *ResNet-50* (R50), con un valor de F1-Score máximo de 0.6959, manteniendo una ligera pero consistente superioridad sobre el resto de las variantes y/o modificaciones.

Trade-off Performance vs Costo Computacional

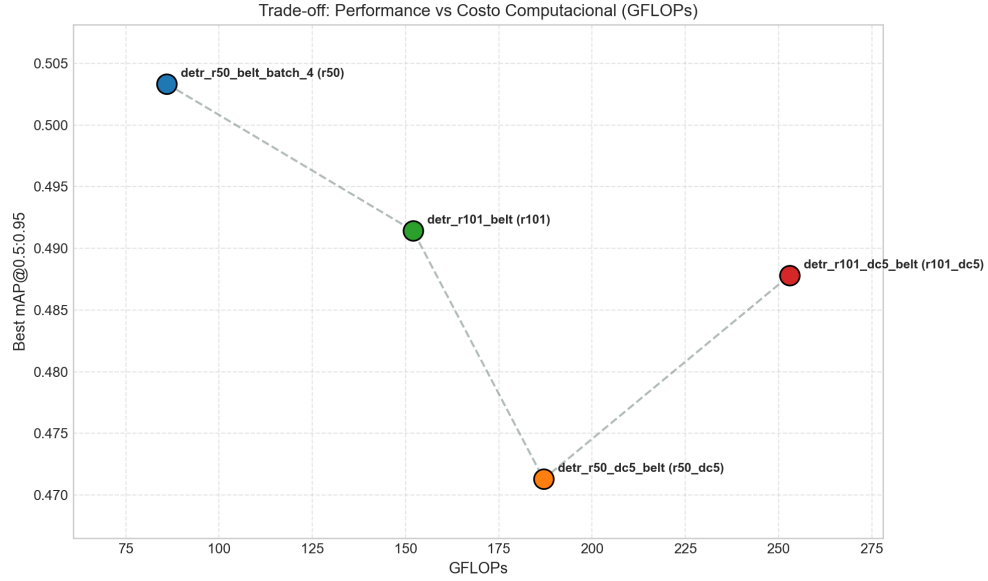


Figura 4.3.23: Tradeoff de rendimiento entre todas las variantes del modelo DETR. Muestra el rendimiento mAP@0.5:0.95 en función de la cantidad de GFlops de cómputo utilizado por la variante.

La Figura 4.3.23 es claro destacar que la eficiencia de los modelos presenta un comportamiento muy diferente con el modelo DETR, respecto a los modelos de detección de una sola etapa (*one stage detectors*). La variante *ResNet-50* (R50) maximiza el rendimiento (0.5033 mAP) consumiendo únicamente 86.0 GFLOPs. Por el contrario, la variante *ResNet-101-DC5* (R101-DC5) dispara el costo a 253.0 GFLOPs (un incremento de casi 300%) resultando en una reducción de la magnitud del rendimiento (0.4878 mAP). Para ambos casos, el rendimiento de las variantes que incluyen la modificación DC5, es menor respecto a las mismas variantes sin el uso de la modificación, obteniendo valores de métricas menores de 0.7784 para la variante *ResNet-50-DC5* (R50-DC5) y 0.7960 para la variante *ResNet-101-DC5* (R101-DC5). De acuerdo a esto, se considerará la variante *ResNet-50* (R50), del modelo DETR, para analizar su desempeño frente a otros modelos.

Resultados específicos mejor variante (*DETR_R50*)

Curva de F1-Score vs Confianza

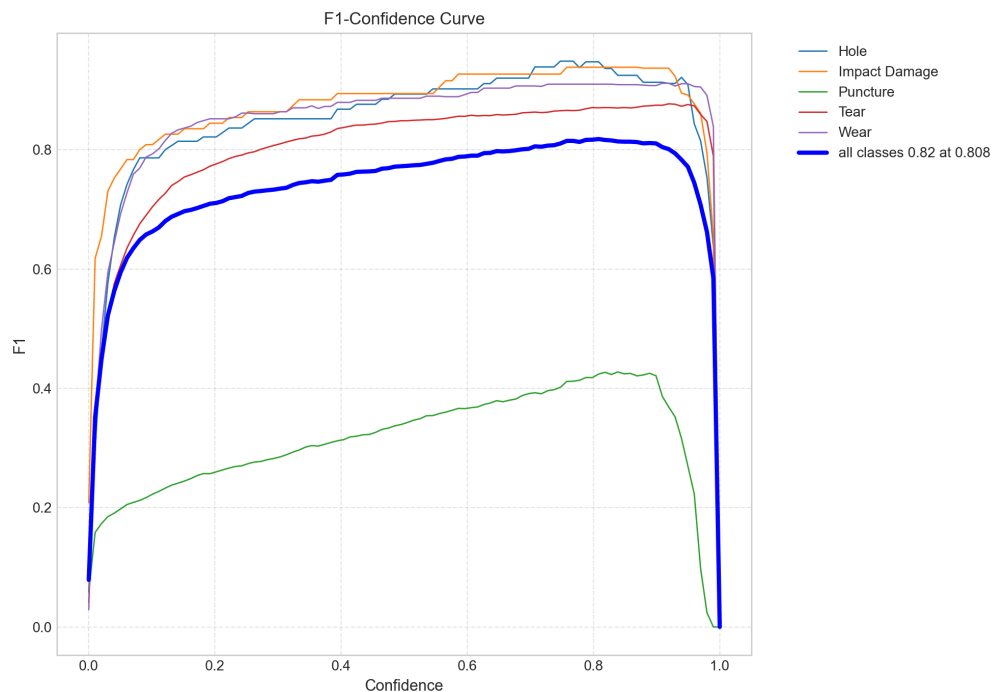


Figura 4.3.24: Curvas de F1-Score en función de la confianza (*confidence*) de detección de la variante DETR-R50 (*ResNet-50*).

Como se observa en la Figura 4.3.24, la métrica global del modelo alcanza su valor máximo bajo un umbral de confianza elevado con respecto a los modelos de detección de una sola etapa (*one stage detectors*). Las clases con mejores métricas, al igual que en los modelos anteriores, mantienen un desempeño aceptable, pero la clase de perforación (*puncture*), mantiene valores bajo el promedio global, pero con una confianza mayor (cerca al umbral de 0.9).

Curva de Precisión vs Confianza (*Precision Curve vs Confidence*)

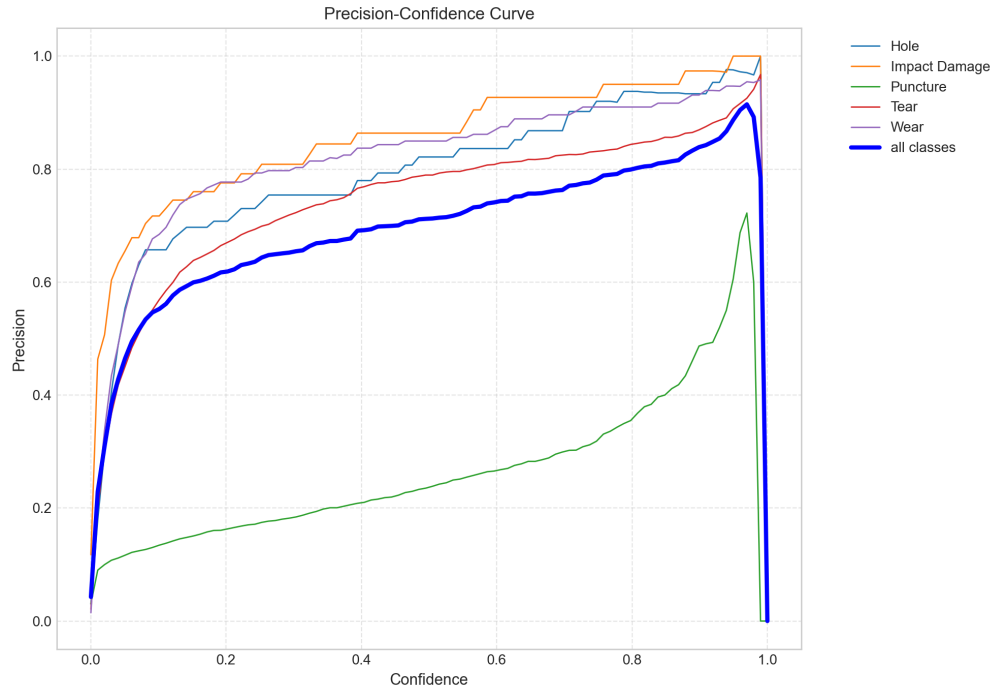


Figura 4.3.25: Curvas de Precision en función de la confianza (*confidence*) de detección de la variante DETR-R50 (*ResNet-50*).

De acuerdo a la Figura 4.3.25, la variante *ResNet-50* (R50) basada en transformers, presenta un mejor comportamiento de precisión al considerar umbrales de confianza más altos, mitigando de manera efectiva falsos positivos respecto a otros modelos. Esto puede explicarse debido a la arquitectura transformer y su mecanismo de atención y consultas.

Curva de Exhaustividad vs Confianza (*Recall Curve vs Confidence*)

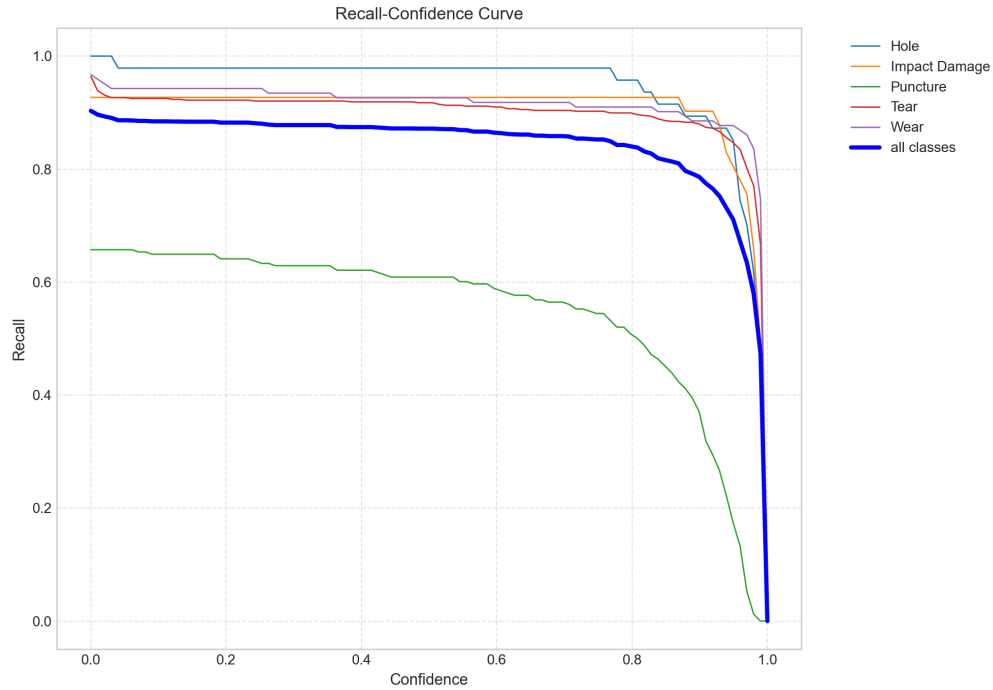


Figura 4.3.26: Curvas de Exhaustividad en función de la confianza (*confidence*) de detección de la variante DETR-R50 (*ResNet-50*).

En la Figura 4.3.26, el modelo evidencia una mejor detección y retención de verdaderos positivos, logrando altos valores de exhaustividad con un umbral de confianza elevado antes de colapsar la curva. Al igual que antes, el uso de consultas y atención, permite eliminar falsas predicciones (falsos positivos) de forma efectiva.

Matriz de confusión (*Confusion Matrix*)



Figura 4.3.27: Matriz de confusión de la variante DETR-R50 (*ResNet-50*).

La matriz presentada en la Figura 4.3.27 permite confirmar el comportamiento previamente discutido. El modelo posee buenos valores de predicción para las clases: agujero (hole) con un valor de 0.98; daño por impacto (impact damage) con un valor de 0.93; desgarró (tear) con un valor de 0.92; abrasión (wear) con un valor de 0.95. Sin embargo, la clase perforación (puncture) presenta un valor similar a la variante SSD512 (0.73), además, el valor de clase predecida de perforación y que en realidad resultaba ser fondo es mayor respecto a los modelos de detección de una sola etapa (*one stage detectors*), con un valor más alto de 0.72, lo que corresponde a una mayor cantidad de falsos positivos para los modelos basados en transformers.

4.3.5. DINO (*DETR with Improved DeNoising Anchor Boxes*)

A continuación se muestran los resultados preliminares obtenidos para el modelo DINO. Estos fueron obtenidos aplicando la metodología explicada en la Sección 3. Los mejores resultados numéricos fueron mostrados en la Tabla 4.3.1.

Pérdida Total vs Época (*Total Loss vs Epoch*)

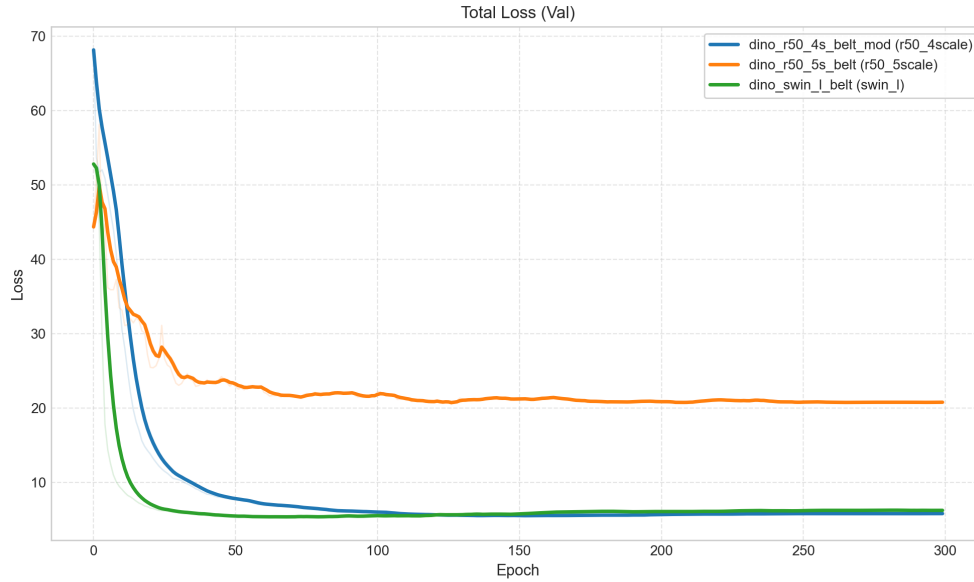


Figura 4.3.28: Pérdida total promedio vs época de la validación en cada variante del modelo DINO.

En la Figura 4.3.28, se observa que las variantes *ResNet-50-4scale* (R50-4s) y *Swin-L* (Swin-L) logran una convergencia rápida y estable en torno a una pérdida cercana al rango entre 2.0 y 3.0. La variante con *backbone* Swin-L exhibe un descenso inicial más pronunciado, pero finalmente la variante *ResNet-50-4scale* (R50-4s) logra estabilizarse en un nivel de pérdida ligeramente inferior hacia el final del entrenamiento. Por otro lado, la variante *ResNet-50-5scale* (R50-5s), sí bien logra una convergencia estable cercana al valor de pérdida 20.0, la pérdida es alrededor de 10 veces mayor respecto a las otras variantes previamente analizadas.

Precisión Media Promedio vs Época (*Mean Average Precision vs Epoch*)

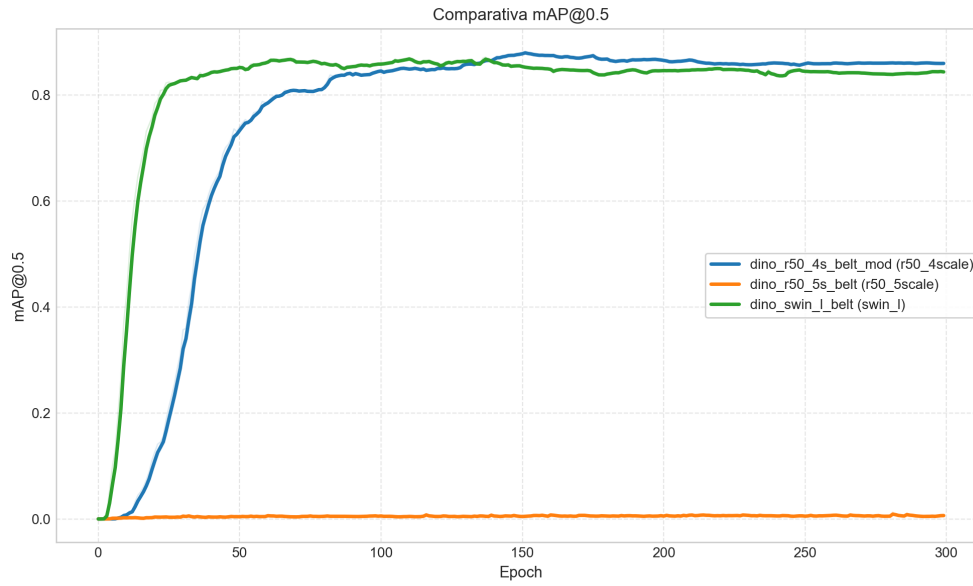


Figura 4.3.29: Precisión Media Promedio (mAP) vs época con umbral IoU 0,5 para todas las variantes del modelo DINO.

A partir de la Figura 4.3.29, la variante *Swin-L* (Swin-L) demuestra una superioridad temprana y mejor convergencia, alcanzando un pico de 0.8710. La arquitectura *ResNet-50-4scale* (R50-4s), a pesar de mostrar una menor velocidad de convergencia temprana, alcanza un mayor valor pico de 0.8809, logrando superar a la variante *Swin-L* (Swin-L) en el entrenamiento tardío. Véase que, la variante *ResNet-50-5scale* (R50-5s) no logra superar en ningún momento el umbral del valor de precisión media promedio de 0.01 y obtener un valor de métrica aceptable para este caso, lo cual se relaciona con la alta pérdida mostrada en el gráfico anterior.

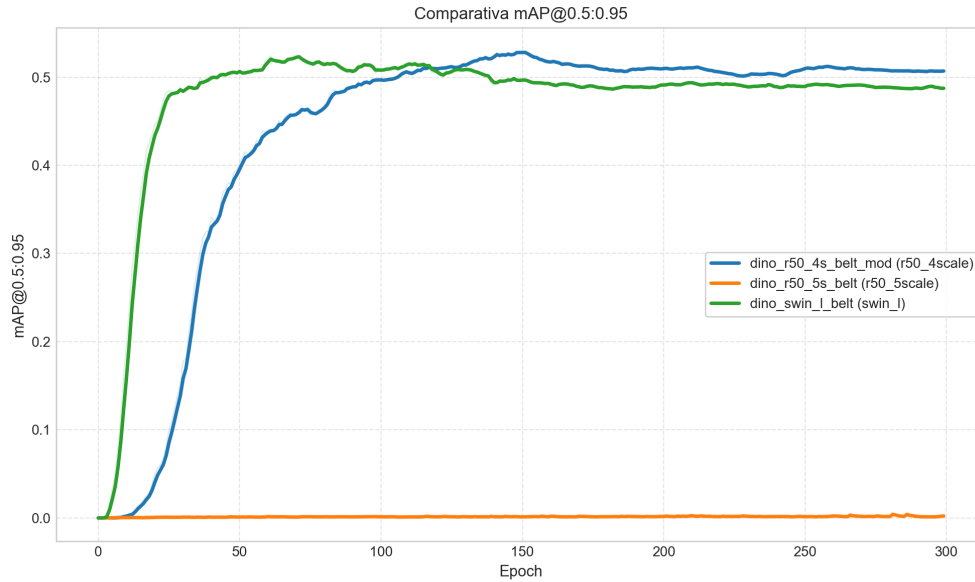


Figura 4.3.30: Precisión Media Promedio (mAP) vs época con umbral IoU desde 0,5 hasta 0,95 para todas las variantes del modelo DINO.

Al evaluar bajo el umbral IoU estricto en la Figura 4.3.30, se confirma la ventaja de la variante *ResNet-50-4scale* (R50-4s). Esta configuración establece el máximo en 0.5298, manteniendo una diferencia consistente sobre la variante *Swin-L* (Swin-L) que obtuvo un valor máximo de 0.5240. Al igual que el gráfico anterior, la variante *ResNet-50-5scale* (R50-5s), no logra obtener una métrica aceptable, debido a su ineficiente convergencia.

F1-Score vs Época

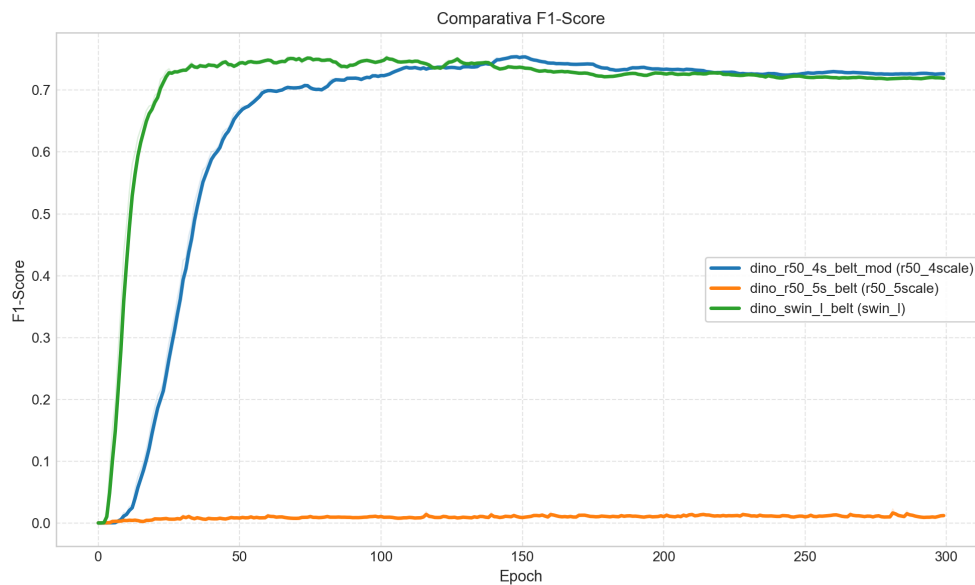


Figura 4.3.31: F1-Score vs época para todas las variantes del modelo DINO.

La curva de la Figura 4.3.31 ilustra un similar comportamiento de convergencia entre las variantes *ResNet-50-4scale* (R50-4s) y *Swin-L* (Swin-L), las cuales obtienen valores máximos de 0.7550 y 0.7554 respectivamente, en donde la última variante mencionada domina ligeramente en magnitud máxima y comportamiento temprano de convergencia. Al igual que los casos anteriores, la variante *ResNet-50-5scale* (R50-5s) no logra converger adecuadamente y obtener valores aceptables.

Trade-off Performance vs Costo Computacional

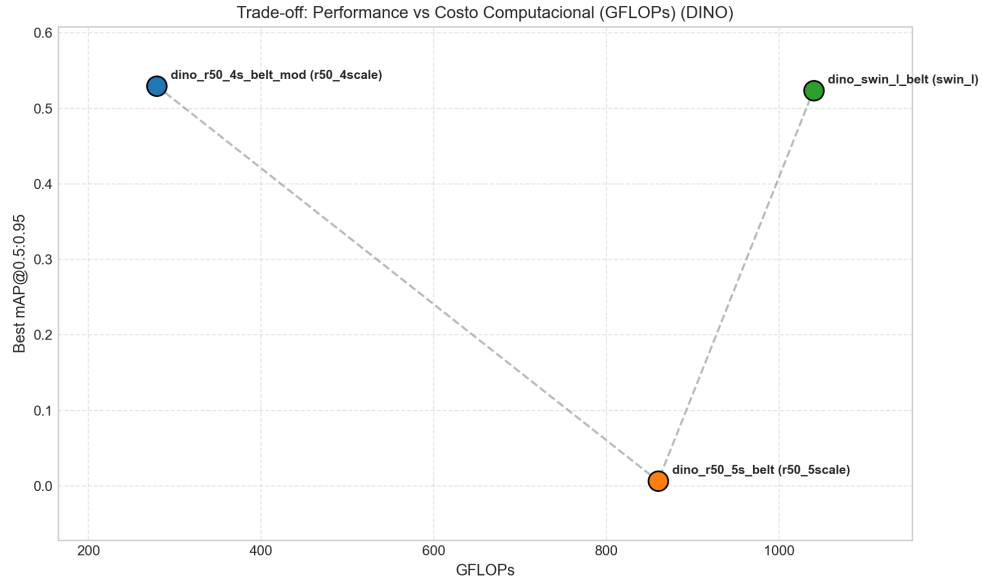


Figura 4.3.32: Tradeoff de rendimiento entre todas las variantes del modelo DINO. Muestra el rendimiento mAP@0.5:0.95 en función de la cantidad de GFlops de cómputo utilizado por la variante.

La Figura 4.3.32 revela un escenario de ineficiencia severa para arquitecturas pesadas. La variante R50-4scale maximiza el rendimiento (0.5298 mAP) con un costo de 279.0 GFLOPs. Implementar el *backbone* Swin-L dispara el costo computacional a 1040.0 GFLOPs (un aumento de $\sim 272\%$) e incrementa los parámetros a 218.0M, resultando irónicamente en un rendimiento métrico marginalmente inferior. De manera similar, la variante *ResNet-50-5scale* (R50-5s) resultó tener métricas inaceptables considerando además su alto costo computacional de 860.0 GFLOPs. De acuerdo a esto, se selecciona la variante *ResNet-50-4scale* (R50-4s) como representante del modelo DINO para comparar con otros modelos.

Resultados específicos mejor variante preliminar (*DINO_R50_4scale*)

Curva de F1-Score vs Confianza

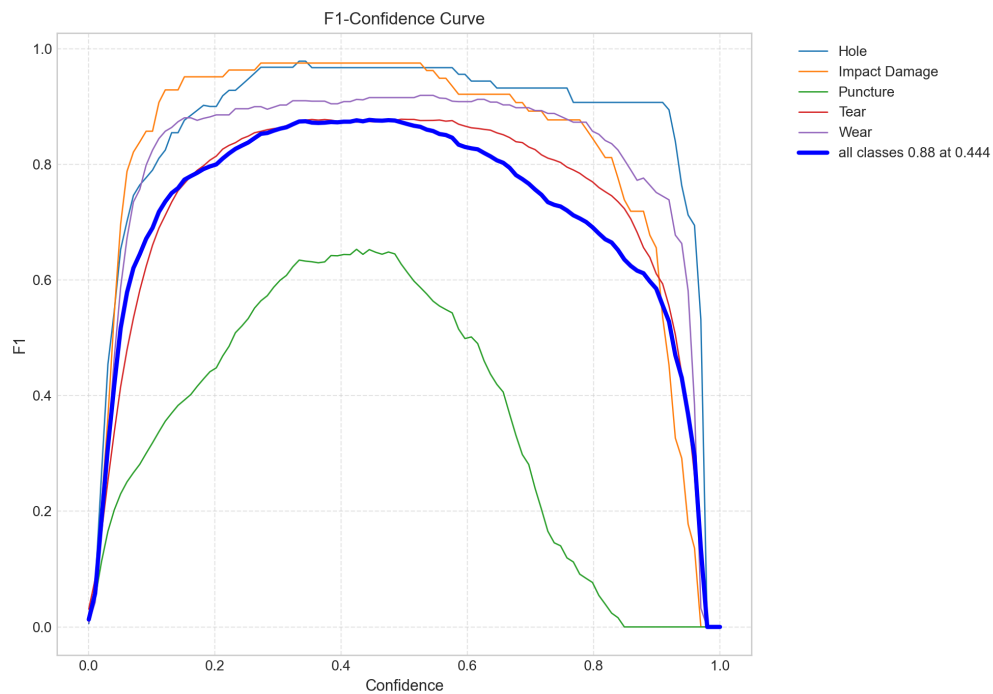


Figura 4.3.33: Curvas de F1-Score en función de la confianza (*confidence*) de detección de la variante DINO-R50-4scale (*ResNet-50*).

En la Figura 4.3.33, el pico óptimo global se establece en 0.88 bajo un umbral de confianza de 0.444. Si bien las clases principales muestran curvas robustas que superan un F1-score de 0.90, la clase perforación (*Puncture*) se mantiene rezagada con un F1-score máximo que apenas supera 0.65, limitando el promedio global.

Curva de Precisión vs Confianza (*Precision Curve vs Confidence*)

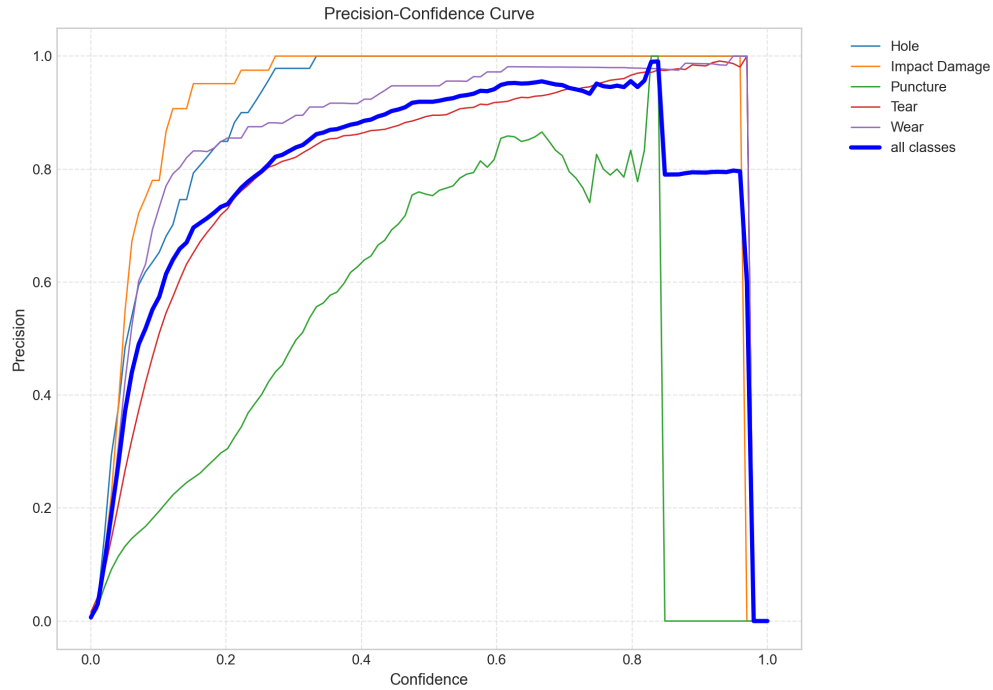


Figura 4.3.34: Curvas de Precision en función de la confianza (*confidence*) de detección de la variante DINO-R50-4scale (*ResNet-50*).

La Figura 4.3.34 muestra una convergencia de alta precisión con umbrales moderados para la mayoría de los defectos. Sin embargo, la clase perforación (*Puncture*) somete la precisión máxima a confianzas cercanas a 0.85, tras lo cual sufre un desprendimiento repentino en sus valores al acercarse a umbrales cercanos a 0.9, comprometiendo el promedio global y provocando un comportamiento anómalo con respecto a otros modelos.

Curva de Exhaustividad vs Confianza (*Recall Curve vs Confidence*)

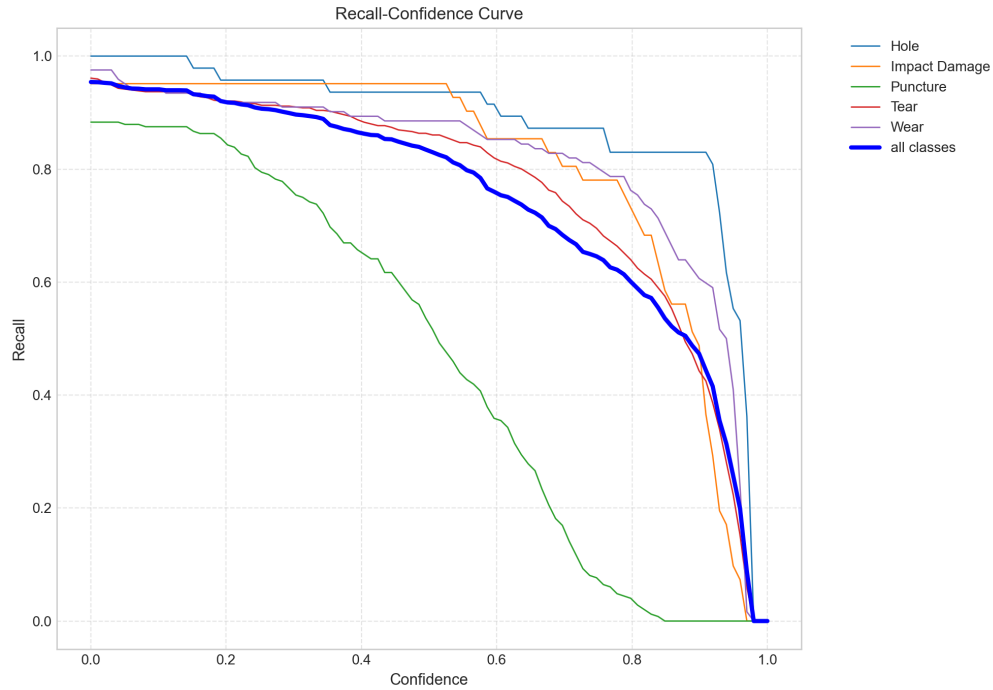


Figura 4.3.35: Curvas de Exhaustividad en función de la confianza (*confidence*) de detección de la variante DINO-R50-4scale (*ResNet-50*).

De acuerdo a la Figura 4.3.35, la variante demuestra unos excelentes valores de exhaustividad para la mayor parte del umbral de confianza, pero la clase perforación (*Puncture*) decrece de forma acelerada y pronunciada conforme aumenta el umbral con respecto a las otras clases. Esto puede atribuirse a la mayor cantidad obtenida de falsos positivos, empeorando la métrica.

Matriz de confusión (*Confusion Matrix*)



Figura 4.3.36: Matriz de confusión de la variante DINO-R50-4scale (*ResNet-50*).

La matriz de la Figura 4.3.36 muestra excelentes valores de predicción para las clases: agujero (hole) con un valor de 0.92; daño por impacto (impact damage) con un valor de 0.95; desgarró (tear) con un valor de 0.91; abrasión (wear) con un valor de 0.92. La clase perforación (*puncture*), aunque posee mejor métrica respecto a las mejores variantes de otros modelos (0.81), también comete predicciones erróneas que son clasificadas como fondo (*background*).

4.4. Inferencia comparativa entre modelos

En esta sección, se entregarán algunos resultados demostrativos de la implementación de detección de objetos, utilizando la partición de pruebas (*test*) de la base de datos confeccionada. Para ello, se utilizaron los mejores pesos de las variantes y se realizó inferencia (predicción) múltiple con cada una de ellas, graficando las cajas delimitadoras reales (en color rojo) y las cajas delimitadoras inferidas (color verde) en diversos casos comunes a comparar; se incorporó además el valor de latencia de procesamiento por modelo en milisegundos. Todo este proceso se llevó a cabo considerando un umbral de corte de confianza por sobre el 70 %, es decir, se mostrarán solamente las predicciones que contengan un valor de confianza mayor o igual a 0.70.



Figura 4.4.1: Inferencia comparativa entre las mejores variantes de cada modelo (*YOLOv5-s*, *SSD512*, *DETR-R50* y *DINO-R50-4scale*). Se incluye el tiempo de latencia de procesamiento por variante y se muestra la leyenda para distinguir las cajas delimitadoras de fondo (reales) y las predichas.

En la Figura 4.4.1, podemos apreciar un caso de inferencia de la falla agujero (*hole*), desempeñada por las mejores variantes antes seleccionadas. Para este caso, las 4 variantes mostradas poseen un desempeño similar en la detección de la falla, obteniendo cajas delimitadoras muy similares con valores de confianza, desde el 0.93 obtenido por *Yolov5-s (small)* hasta confianza de 1.00 obtenida por *DETR-R50 (ResNet-50)*. Además, los valores de latencia de procesamiento, colocan al modelo *Yolov5* como la variante más rápida con *15.8ms*, seguida de *DETR* con *124.1ms* y finalizando con *SSD* y *DINO* con latencias de *293.9ms* y *656.7ms* respectivamente.

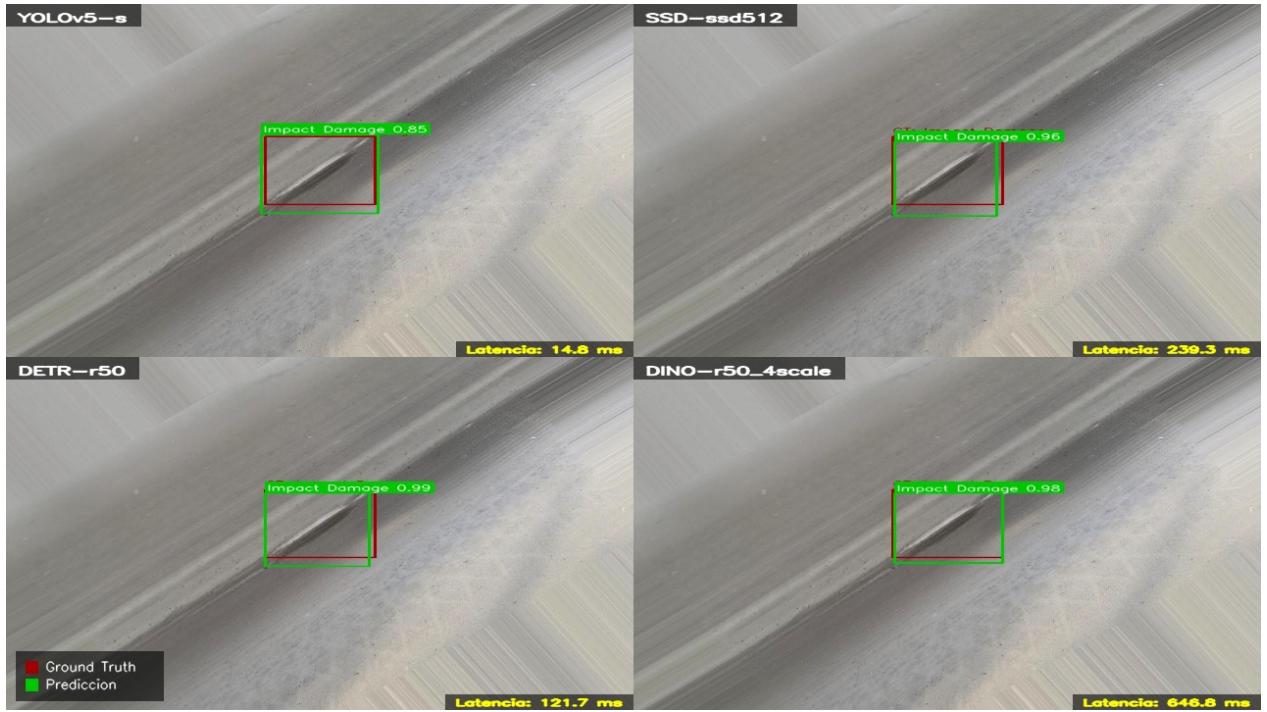


Figura 4.4.2: Inferencia comparativa entre las mejores variantes de cada modelo (*YOLOv5-s*, *SSD512*, *DETR-R50* y *DINO-R50-4scale*). Se incluye el tiempo de latencia de procesamiento por variante y se muestra la leyenda para distinguir las cajas delimitadoras de fondo (reales) y las predichas.

En la Figura 4.4.2, podemos apreciar un caso de inferencia de la falla de daño por impacto (*impact damage*), desempeñada por las mejores variantes antes seleccionadas. Para este caso, las 4 variantes mostradas poseen un desempeño similar en la detección de la falla, obteniendo cajas delimitadoras muy similares con valores de confianza, desde el 0.85 obtenido por *Yolov5-s (small)* hasta el valor de confianza de 0.99 obtenido por *DETR-R50 (ResNet-50)*. Además, los valores de latencia de procesamiento poseen el mismo comportamiento jerárquico considerando a *Yolov5-s (small)* como la variante más rápida (menor tiempo de procesamiento) en procesar la inferencia con respecto a las demás y *DINO-R50-4scale (ResNet-50-4scale)* como la variante de mayor tiempo de procesamiento.



Figura 4.4.3: Inferencia comparativa entre las mejores variantes de cada modelo (*YOLOv5-s*, *SSD512*, *DETR-R50* y *DINO-R50-4scale*). Se incluye el tiempo de latencia de procesamiento por variante y se muestra la leyenda para distinguir las cajas delimitadoras de fondo (reales) y las predichas.

En la Figura 4.4.3, podemos apreciar un caso de inferencia de las fallas de desgarro (*tear*) y perforación (*puncture*), desempeñada por las mejores variantes antes seleccionadas. Para este caso, las 4 variantes mostradas poseen un desempeño diferente a lo esperado. Para la falla de desgarro, todas las variantes logran detectar el desgarro grande, donde la confiabilidad es mayor en los modelos basados en transformers (*DETR-R50* y *DINO-R50-4scale*) con valores de confianza de 1.00 y 0.99 respectivamente. Sin embargo, el desgarro pequeño superior, solo es detectado por las variantes (*DETR-R50* y *DINO-R50-4scale*), mientras que las variantes de una etapa (*Yolov5-s* y *SSD512*) no infieren sobre ese desgarro. La principal diferencia radica en la *detección de las perforaciones*: mientras que las variantes de una etapa (*Yolov5-s* y *SSD512*) no son capaces de detectar la mayor parte de las fallas (a excepción de *Yolov5-s* con una detección correcta de confianza 0.76), las variantes (*DETR-R50* y *DINO-R50-4scale*) logran detectar la mayor parte de las perforaciones con una confianza relativamente alta por encima del umbral 0.90 aproximadamente. Esto permite visualizar el comportamiento descrito en la Sección 4.3 con respecto al F1-score y como la clase perforación posee resultados ligeramente mejores en los modelos basados en transformers (*DETR-R50* y *DINO-R50-4scale*) con respecto a los modelos de una sola etapa (*Yolov5-s* y *SSD512*).

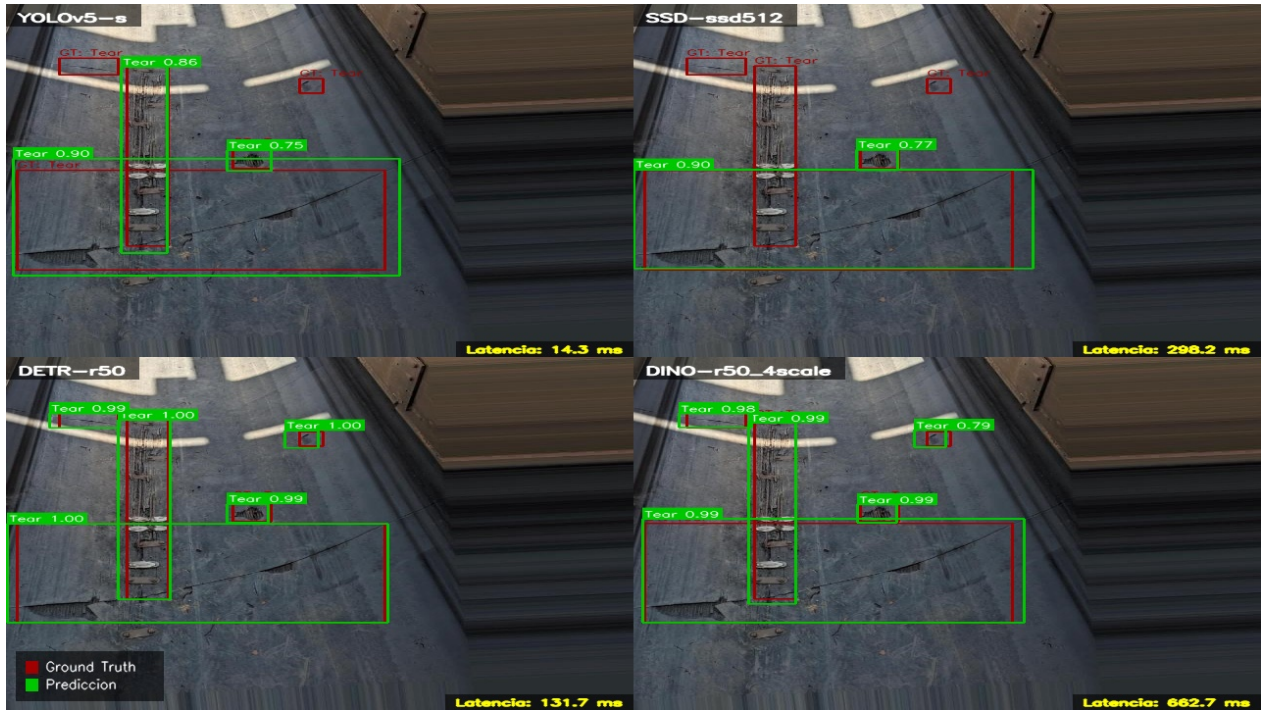


Figura 4.4.4: Inferencia comparativa entre las mejores variantes de cada modelo (*YOLOv5-s*, *SSD512*, *DETR-R50* y *DINO-R50-4scale*). Se incluye el tiempo de latencia de procesamiento por variante y se muestra la leyenda para distinguir las cajas delimitadoras de fondo (reales) y las predichas.

En la Figura 4.4.4, podemos apreciar un caso de inferencia de la falla desgarro (*tear*), desempeñada por las mejores variantes antes seleccionadas. Para este caso, las 4 variantes mostradas poseen un desempeño similar en la detección de la falla, obteniendo cajas delimitadoras muy similares con valores de confianza, desde el 0.75 obtenido por *Yolov5-s (small)* hasta el valor de confianza de 1.00 obtenido por *DETR-R50 (ResNet-50)*. Cabe destacar que en este caso, cuando existe intersección múltiple de una misma falla o diferentes, casi todas las variantes son capaces de hacer la detección y discriminación entre diferentes cajas delimitadoras (evitando solapamiento de cajas delimitadoras), pero, en la detección de los desgarros de menor tamaño y aislados, las variantes *DETR-R50* y *DINO-R50-4scale* logran detectar ambos casos separados de los 3 desgarros centrales (ver desgarros pequeños a la izquierda y derecha del desgarro central). Además, los valores de latencia de procesamiento poseen el mismo comportamiento jerárquico considerando a *Yolov5-s (small)* como la variante más rápida (menor tiempo de procesamiento) en procesar la inferencia con respecto a las demás y *DINO-R50-4scale (ResNet-50-4scale)* como la variante de mayor tiempo de procesamiento.

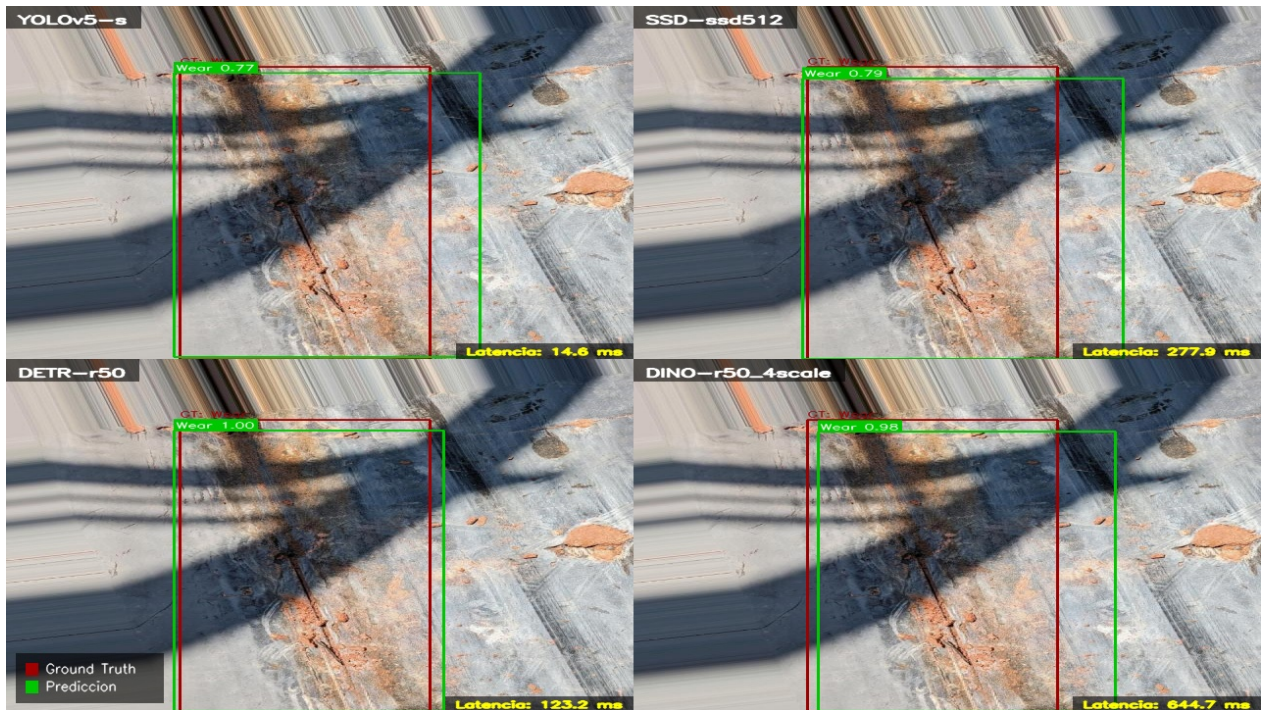


Figura 4.4.5: Inferencia comparativa entre las mejores variantes de cada modelo (*YOLOv5-s*, *SSD512*, *DETR-R50* y *DINO-R50-4scale*). Se incluye el tiempo de latencia de procesamiento por variante y se muestra la leyenda para distinguir las cajas delimitadoras de fondo (reales) y las predichas.

En la Figura 4.4.5, podemos apreciar un caso de inferencia de la falla de abrasión (*wear*), desempeñada por las mejores variantes antes seleccionadas. Para este caso, las 4 variantes mostradas poseen un desempeño similar en la detección de la falla, obteniendo cajas delimitadoras espacialmente similares, pero con diferentes valores de confianza que van, desde el 0.77 obtenido por *Yolov5-s (small)* hasta confianza de 1.0 obtenida por *DETR-R50 (ResNet-50)*. Además, los valores de latencia de procesamiento poseen el mismo comportamiento jerárquico considerando a *Yolov5-s (small)* como la variante más rápida (menor tiempo de procesamiento) en procesar la inferencia con respecto a las demás y *DINO-R50-4scale (ResNet-50-4scale)* como la variante de mayor tiempo de procesamiento.



Figura 4.4.6: Inferencia comparativa entre las mejores variantes de cada modelo (*YOLOv5-s*, SSD512, DETR-R50 y DINO-R50-4scale). Se incluye el tiempo de latencia de procesamiento por variante y se muestra la leyenda para distinguir las cajas delimitadoras de fondo (reales) y las predichas.

En la Figura 4.4.6, podemos apreciar un caso de inferencia de fondo (*background*), desempeñada por las mejores variantes antes seleccionadas. Este caso de control permite ilustrar que todos los modelos son capaces de distinguir correctamente el caso de fondo en ausencia de fallas y evitar inferir de forma incorrecta, generando falsos positivos. Además, los valores de latencia de procesamiento poseen el mismo comportamiento jerárquico considerando a *Yolov5-s* (*small*) como la variante más rápida (menor tiempo de procesamiento) en procesar la inferencia con respecto a las demás y *DINO-R50-4scale* (*ResNet-50-4scale*) como la variante de mayor tiempo de procesamiento.



Figura 4.4.7: Inferencia comparativa entre las mejores variantes de cada modelo (*YOLOv5-s*, *SSD512*, *DETR-R50* y *DINO-R50-4scale*). Se incluye el tiempo de latencia de procesamiento por variante y se muestra la leyenda para distinguir las cajas delimitadoras de fondo (reales) y las predichas.

En la Figura 4.4.7, podemos apreciar un caso de inferencia de la falla de daño por impacto (*impact damage*), desempeñada por las mejores variantes antes seleccionadas. Si bien la falla principal de daño por impacto en el centro de la imagen es correctamente detectada por las 4 variantes desde valores de confianza de 0.91 para *Yolov5-s (small)* y 1.00 para *DETR-R50 (ResNet-50)*, también existen múltiples falsos positivos debido a la ausencia de etiquetas para la clase perforación (*puncture*) que fue detectada en múltiples ocasiones en este caso. Si bien para el caso de pruebas esta imagen solo contiene la falla de daño por impacto, la inferencia realizada por los modelos (principalmente los basados en transformers), exhibe una alta capacidad para identificar defectos reales que fueron omitidos en el etiquetado original. No obstante, al no poseer una caja delimitadora de referencia en la verdad de terreno (*ground truth*), estas detecciones son penalizadas matemáticamente como falsos positivos. Este comportamiento, probablemente reiterativo con las diversas imágenes similares en la base de datos, explica el bajo F1-Score mostrado en la Sección 4.3 al analizar cada variante, demostrando que *la métrica se ve afectada por la falta de anotaciones completas en el conjunto de datos y no por una deficiencia en la capacidad de detección del modelo*. Además, los valores de latencia de procesamiento poseen el mismo comportamiento jerárquico considerando a *Yolov5-s (small)* como la variante más rápida (menor tiempo de procesamiento) en procesar la inferencia con respecto a las demás y *DINO-R50-4scale (ResNet-50-4scale)* como la variante de mayor tiempo de procesamiento.

4.5. Discusión general de resultados

Las métricas expuestas y analizadas individualmente para cada modelo fueron recibidas tras el entrenamiento y validación de acuerdo a lo descrito en la Sección 2.7. Además, se incluyeron otros resultados relevantes que los modelos de aprendizaje profundo (*deep learning*) siempre entregan al momento de ejecutarse (como la pérdida total vs época, tiempos de entrenamiento y uso de recursos computacionales). La base de datos confeccionada presenta un alto número de etiquetas para dos clases de forma predominante (ver Figura 4.1.1), en particular la clase desgarro (*tear*) y perforación (*puncture*), concentrando aproximadamente el 81 % de las etiquetas totales de entrenamiento según lo estimado de acuerdo a la Tabla 4.1.1. A pesar de este desbalance de clases y etiquetas, la clase desgarro (*tear*) obtiene siempre, en todos los modelos, valores F1-score por encima del promedio, junto a las otras clases (agujero, abrasión, daño por impacto) con menor número de etiquetas. Este comportamiento, sumado a lo ilustrado en la Figura 4.1.3, permite ver que el principal problema no es el desbalance de clases o la cantidad de imágenes, sino la *asignación del tamaño de etiquetas*; a diferencia de las otras clases, la perforación (*puncture*) posee cajas delimitadoras (*bounding boxes*) muy pequeñas con respecto a las otras clases mostradas. Esto explica principalmente su rendimiento deficiente global en todos los entrenamientos de cada modelo.

Los modelos de detección de una sola etapa (YOLOv5 y SSD) presentan una convergencia inicial rápida y estable, señalado principalmente por las curvas de pérdida ¹ (ver Figuras 4.3.1 y 4.3.10). Este comportamiento era el esperado debido a que ambas arquitecturas son livianas y optimizadas para lograr inferencias más rápidas de acuerdo a la revisión de Zhang et al. [4]. Tal como se anticipaba en la revisión del estado del arte, la cual posicionaba a las versiones recientes de YOLO con precisiones superiores a SSD (ver Sección 2.5.2, Figura 2.5.2). La variante YOLOv5-s (mejor variante representante de YOLOv5) logró obtener un F1-Score máximo de 0.8897 y un mAP@0.5:0.95 de 0.5517, superando ampliamente a SSD512 (mejor variante representante de SSD), que obtuvo solamente un F1-Score de 0.7424 y un mAP@0.5:0.95 de 0.4277.

En contraparte, los modelos de detección de objetos basados en Transformers (DETR y DINO) exhiben un comportamiento de convergencia mejor y más acelerado durante las primeras 50 a 100 épocas (exceptuando la variante fallida DINO-R50-5scale)². Esto se puede apreciar en las Figuras 4.3.19 y 4.3.28. Asimismo, de acuerdo a la Tabla 2.6.2, la evolución y mejora de DINO es notoria respecto a la estabilización de los gradientes tras introducir el mecanismo de ruido contrastivo y las mejoras propuestas por Zhang et al [17] al modelo

¹ Cabe señalar que los valores de pérdida de cada modelo poseen diferentes escalas debido a las diferentes funciones de pérdida implementadas por cada arquitectura (ver Secciones 2.5.3.4 y 2.5.4.3). Por ende, lo más relevante y analizado fue el decrecimiento (pendiente) en las primeras 50 épocas.

² Esta comparación es respecto a los modelos de detección de una sola etapa (*one stage detectors*). Los transformers implementan sus propias funciones de pérdida según lo descrito en las Secciones 2.6.3.3 y 2.6.4.2

DETR. Mientras que a las variantes de DETR les demora cerca de 150 épocas estabilizarse, DINO lo logra a las 100 épocas e incluso a las 50 épocas al implementar su variante Swin-L. De la misma forma, las métricas globales demuestran una mejora para el modelo DINO con respecto a DETR (ver Sección 2.6.6, Tabla 2.6.2). La variante DETR-R50 (mejor variante representante de DETR) logró obtener un F1-Score máximo de 0.6959 y un $\text{mAP}@0.5:0.95$ de 0.5033, lo cual estuvo por debajo de DINO-R50-4scale (mejor variante representante de DINO), que obtuvo un F1-Score de 0.7550 y un $\text{mAP}@0.5:0.95$ de 0.5298. Ambos resultados en términos de magnitud superan a los obtenidos por la variante SSD512 (mejor variante representante de SSD), colocando esta variante como la que muestra un peor desempeño respecto a todos los modelos implementados.

El análisis de las curvas de F1-Score, Precisión y Exhaustividad en función del umbral de confianza para cada mejor variante según el tipo de modelo implementado, revela un problema común: la identificación y detección deficiente de la clase perforación (*Puncture*), en particular cabe destacar que la falla mostrada en la Figura 4.1.3.c presenta cajas delimitadoras muy pequeñas y alta multiplicidad según lo mostrado en la Figura 4.1.1 (ver cantidad de etiquetas para la clase perforación), donde se encuentra como la segunda clase con mayor cantidad de etiquetas de la base de datos (919 para entrenamiento). En las variantes YOLOv5-s y SSD512, esta clase restringe severamente el promedio global debido a una alta tasa de falsos positivos y falsos negativos, alcanzando un valor máximo de predicción verdadera del 78 % de los casos para la variante YOLOv5-s y del 73 % de los casos para la variante SSD512. Este comportamiento es confirmado de forma gráfica de acuerdo a lo expuesto en la Sección 4.4, particularmente lo analizado en las Figuras 4.4.3 y 4.4.7, donde los modelos basados en transformers logran inferir de mejor forma las perforaciones, pero debido a casos donde no hay etiquetas para perforaciones muy pequeñas, su precisión se termina castigando por la detección de falsos positivos. Así, los modelos basados en Transformers demuestran su ventaja teórica esperada con respecto a los modelos tradicionales basados solamente en redes neuronales convolucionales (CNN). Gracias a los mecanismos de autoatención y el contexto global de la imagen sin limitarse a las extracciones de características locales convolucionales, la variante DINO-R50-4scale logra mitigar ligeramente este déficit, obteniendo un mayor porcentaje de predicción verdadera del 81 % de los casos en la clase perforación (*puncture*), lo que supera el valor de casos de la variante YOLOv5-s antes mencionada. Sin embargo, debido a la ligera disminución de predicciones verdaderas en otras clases de acuerdo a la matriz de confusión, esto empeora ligeramente la métrica global $\text{mAP}@0.5:0.95$, lo que provoca que el modelo de una sola etapa YOLOv5-s supere con un valor aproximado de 0.55 a la variante DINO-R50-4scale, que obtuvo finalmente un valor aproximado de 0.53.

Es fundamental destacar que este análisis comparativo se estructuró bajo un régimen estandarizado (configuraciones similares y base de datos común). Además, *no se ejecutaron optimizaciones de hiperparámetros* para favorecer a ninguna arquitectura en particular. La mayor parte de las configuraciones base (o por defecto) se mantuvieron intactas, aplicando

ajustes únicamente cuando fuera necesario para asegurar que el proceso de entrenamiento se cumpliera con éxito. Un ejemplo de esto fue la reducción del tamaño de lote (*batch size*) a 4 o 2 en las arquitecturas basadas en Transformers, en contraste con el tamaño de lote de 16 empleado en los modelos *One-Stage*. Esta configuración utilizada en los modelos de detección basados de transformers fue estimada de acuerdo a las condiciones de entrenamiento usadas para el modelo DETR en un principio, de acuerdo a Carion et al [16], donde se señala que ocuparon un tamaño de lote de 64 repartido entre 16 tarjetas gráficas (lo que nos da un tamaño de lote de 4 imágenes por tarjeta gráfica).

Bajo este contexto y evaluando la viabilidad operativa a través del balance entre rendimiento y costo computacional (*Trade-off*), la variante *small* (s) de YOLOv5 se erige indiscutiblemente como el modelo más eficiente del estudio. Al requerir únicamente 15.8 GFLOPs de cómputo y 7.2M de parámetros, YOLOv5-s obtiene métricas que igualan o superan a arquitecturas más densas. Esto exige una fracción ínfima del hardware necesario en comparación a DINO-R50-4scale, el cual consume 279.0 GFLOPs y 47.0M de parámetros. Esta eficiencia paramétrica confirma que, para aplicaciones industriales de monitoreo continuo donde la velocidad de inferencia y la capacidad de recursos físicos (*hardware*) es un requisito estricto, las arquitecturas *One-Stage* presentan una mejor viabilidad práctica.

Finalmente, es importante señalar que el desempeño de las arquitecturas Transformer en este estudio se encuentra intrínsecamente acotado tanto por la escala de los datos disponibles como por las restricciones de recursos físicos (*hardware*). A diferencia de las redes neuronales convolucionales (CNN) como YOLO y SSD, que extraen características mediante un procesamiento local (asumiendo que los píxeles cercanos se relacionan espacialmente gracias a sus sesgos inductivos predefinidos), los Transformers como DETR y DINO carecen de estas reglas. Al basarse en un mecanismo de atención global, un Transformer debe calcular matemáticamente como se relaciona cada parche o trozo de la imagen con todos los demás de forma simultánea, debiendo deducir la estructura semántica visual desde cero. Esta exigencia de modelar dependencias globales sin reglas espaciales preprogramadas, sumada a la transferencia de pesos preentrenados sobre conjuntos de datos genéricos (diferentes al contexto de imágenes de correas transportadoras utilizado) y a la optimización simultánea de un número masivo de consultas espaciales (*object queries*), induce una severa inestabilidad inicial debido a la brecha entre consultas, clases correctas y fondo que son inferidos. Aunque los modelos logran corregir esta discrepancia de forma efectiva a lo largo de las épocas, la velocidad de esta estabilización se vio directamente afectada por limitaciones computacionales. A pesar de contar con capacidad física (*hardware*) especializado, el alto consumo de memoria VRAM de estas arquitecturas, obligó a restringir drásticamente el tamaño de lote (*batch size*) a 2 o 4 muestras por iteración. En modelos basados en atención, lotes reducidos penalizan el aprendizaje y la propagación de gradientes asociados a la función de pérdida, ralentizando la convergencia y prolongando de forma crítica los tiempos totales de entrenamiento, tal como se evidenció en la Tabla 4.2.1.

Capítulo 5

Conclusiones

En el presente trabajo de memoria se cumplió con el objetivo general de implementar y evaluar algoritmos de detección de objetos, basados en inteligencia artificial, para la identificación automática de fallas en correas transportadoras. Asimismo, los objetivos específicos y alcances propuestos fueron logrados con éxito. Se implementaron y evaluaron dos arquitecturas convolucionales asociadas a modelos de detección de objetos de una sola etapa (YOLOv5 y SSD) junto con modelos basados en mecanismos de atención global con arquitectura (*Transformer*) (DETR y DINO). Todo el proceso de entrenamiento, validación y prueba se realizó utilizando una base de datos pública debidamente estructurada, considerando cinco clases de falla y un esquema estandarizado que permitió una evaluación cuantitativa ordenada y precisa.

Respecto a los resultados y discusiones, la variante *small* (s) de YOLOv5 se consolidó como el modelo superior al maximizar la eficiencia, mientras que SSD512 obtuvo el rendimiento más deficiente. El mayor desafío recayó en la detección deficiente de la clase perforación (*Puncture*); no obstante, DINO-R50-4scale demostró ligera superioridad prediciendo este defecto gracias a su atención global y su mecanismo mejorado de eliminación de ruido contrastivo.

El desempeño de las arquitecturas *Transformers* se vio penalizado por la escala de los datos, la brecha de dominio y las restricciones físicas. Su extremo consumo de memoria forzó a reducir el tamaño de lote, ralentizando la convergencia de los gradientes y aumentando los tiempos de entrenamiento. Al mantener un régimen estandarizado sin optimización profunda de hiperparámetros, los detectores convolucionales *One-Stage* se ratifican como la alternativa de mayor viabilidad práctica e industrial en la actualidad.

Como trabajo futuro se propone: una optimización de hiperparámetros para cada modelo; la ejecución de entrenamientos en escritorios fijos dedicados, en vez de equipos portátiles; y la implementación de arquitecturas híbridas (como RT-DETR o DELO), orientadas a fusionar la eficiencia de extracción local de las CNN con el mecanismo de atención global de los *Transformers*.

Bibliografía

- [1] Khanna, A., Singh, B., y Kumar, S., “Types and causes of damage to the conveyor belt – Review, classification and mutual relations”, *Engineering Failure Analysis*, vol. 141, p. 106553, 2022, [doi:10.1016/j.engfailanal.2022.106553](https://doi.org/10.1016/j.engfailanal.2022.106553).
- [2] Fuente, V. B. A. D. L., “Detección temprana de fallas en cintas transportadoras mediante visión computacional y deep learning”, memoria para optar al título de ingeniera civil mecánica, Universidad de Chile, Santiago, Chile, 2024. Acceso abierto.
- [3] Zhang, M., Shi, H., Zhang, Y., Yu, Y., y Zhou, M., “Deep learning-based damage detection of mining conveyor belt”, *Measurement: Journal of the International Measurement Confederation*, vol. 175, p. 109130, 2021, [doi:10.1016/j.measurement.2021.109130](https://doi.org/10.1016/j.measurement.2021.109130).
- [4] Zhang, Y., Li, X., Wang, F., Wei, B., y Li, L., “A Comprehensive Review of One-stage Networks for Object Detection”, en *2021 IEEE International Conference on Signal Processing, Communications and Computing (ICSPCC)*, IEEE, 2021, [doi:10.1109/ICSPCC52875.2021.9564613](https://doi.org/10.1109/ICSPCC52875.2021.9564613).
- [5] Gupta, J., Pathak, S., y Kumar, G., “Deep Learning (CNN) and Transfer Learning: A Review”, *Journal of Physics: Conference Series*, vol. 2273, no. 1, p. 012029, 2022, [doi:10.1088/1742-6596/2273/1/012029](https://doi.org/10.1088/1742-6596/2273/1/012029).
- [6] Kaur, J. y Singh, W., “Tools, techniques, datasets and application areas for object detection in an image: a review”, *Multimedia Tools and Applications*, vol. 81, no. 27, pp. 38297–38351, 2022, [doi:10.1007/s11042-022-13153-y](https://doi.org/10.1007/s11042-022-13153-y).
- [7] Arkin, E., Yadikar, N., Xu, X., Aysa, A., y Ubul, K., “A survey: object detection methods from CNN to transformer”, *Multimedia Tools and Applications*, vol. 82, no. 14, pp. 21353–21383, 2023, [doi:10.1007/s11042-022-13801-3](https://doi.org/10.1007/s11042-022-13801-3).
- [8] Ajit, A., Acharya, K., y Samanta, A., “A review of convolutional neural networks”, en *2020 International Conference on Emerging Trends in Information Technology and Engineering (ic-ETITE)*, IEEE, 2020.
- [9] Badillo, F. L., Hernández, C. A. R., Narváez, B. M., y Trillos, Y. E. A., “Redes neuronales convolucionales: un modelo de Deep Learning en imágenes diagnósticas. Revisión de tema”, *Revista Colombiana de Radiología*, 2023, [doi:10.53903/01212095.161](https://doi.org/10.53903/01212095.161).
- [10] Nyuytiymbiy, K., “Parameters and Hyperparameters in Machine Learning and Deep Learning”, *Towards Data Science*, December 2020, <https://towardsdatascience.com/parameters-and-hyperparameters-in-machine-learning-and-deep-learning/>

ameters-and-hyperparameters-aa609601a9ac.

- [11] Ultralytics, “Ultralytics YOLO hyperparameter tuning guide”. <https://docs.ultralytics.com/es/guides/hyperparameter-tuning/>, 2023.
- [12] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., y Polosukhin, I., “Attention Is All You Need”, en Advances in Neural Information Processing Systems, vol. 30, pp. 5998–6008, 2017, <http://arxiv.org/abs/1706.03762>.
- [13] Terven, J., Córdova-Esparza, D., y Romero-González, J., “A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS”, Machine Learning and Knowledge Extraction, vol. 5, no. 4, pp. 1680–1716, 2023, doi: 10.3390/make5040083.
- [14] Jani, M., Fayyad, J., Al-Younes, Y., y Najjaran, H., “Model compression methods for yolov5: A review”, arXiv preprint arXiv:2307.11904, 2023, doi:10.48550/arXiv.2307.11904.
- [15] Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C., y Berg, A. C., “SSD: Single Shot MultiBox Detector”, en Computer Vision – ECCV 2016, Lecture Notes in Computer Science, Vol. 9905, pp. 21–37, Springer International Publishing, 2016, doi:10.1007/978-3-319-46448-0_2.
- [16] Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., y Zagoruyko, S., “End-to-End Object Detection with Transformers”, en Proceedings of the European Conference on Computer Vision (ECCV), p. 213–229, 2020, doi:10.1007/978-3-030-58452-8_13.
- [17] Zhang, H., Li, F., Liu, S., Zhang, L., Su, H., Zhu, J., Ni, L. M., y Shum, H.-Y., “DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection”. arXiv preprint arXiv:2203.03605, 2022. Accepted at ICLR 2023.
- [18] Schlosser, T., Friedrich, M., Meyer, T., y Kowerko, D., “A Consolidated Overview of Evaluation and Performance Metrics for Machine Learning and Computer Vision”, August 2024, https://www.researchgate.net/publication/379982431_A_Consolidated_Overview_of_Evaluation_and_Performance_Metrics_for_Machine_Learning_and_Computer_Vision. Accessed manuscript, unpublished.
- [19] Ultralytics, “Ultralytics YOLO hyperparameter tuning guide”. <https://docs.ultralytics.com/datasets#steps-to-contribute-a-new-dataset>, 2023.
- [20] N., F., “Implementation of Object Detection for Conveyor Belt Defect Detection”. <https://github.com/FernandoN23/Implementation-of-Object-Detection-for-Conveyor-Belt-Defect-Detection>, 2024.

Capítulo 6

Anexos

6.1. Tablas de Hiperparámetros

Tabla 6.1.1: Hiperparámetros principales seleccionados para el entrenamiento de YOLOv5.

Hiperparámetro	Valor seleccionado	Nombre
<i>variant</i>	$\{n, s, m, l, x\}$	Variante del modelo
<i>img_size</i>	640	Tamaño imagen de entrada
<i>epochs</i>	300	Épocas de entrenamiento
<i>optimizer</i>	SGD	Optimizador de entrenamiento
<i>lr0</i>	0.01	Tasa de aprendizaje inicial
<i>lrf</i>	0.01	Factor de tasa final
<i>momentum</i>	0.937	Momento del optimizador
<i>weight_decay</i>	0.0005	Regularización L_2
<i>batch_size</i>	8	Tamaño de lote de entrenamiento
<i>warmup_bias_lr</i>	0.1	Tasa de aprendizaje sesgos calentamiento
<i>box</i>	0.05	Peso pérdida de cajas
<i>cls</i>	0.5	Peso pérdida de clasificación
<i>obj</i>	1.0	Peso pérdida de objeto
<i>iou_t</i>	0.2	Umbral IoU para anclas
<i>anchor_t</i>	4.0	Umbral de ajuste de anclas
<i>hsv_h</i>	0.015	Aumentación en tono HSV
<i>hsv_s</i>	0.7	Aumentación en saturación HSV
<i>hsv_v</i>	0.4	Aumentación en valor HSV
<i>translate</i>	0.1	Traslación geométrica
<i>scale</i>	0.5	Escalamiento aleatorio
<i>fliplr</i>	0.5	Volteo horizontal
<i>mosaic</i>	1.0	Aumentación tipo mosaico

Tabla 6.1.2: Hiperparámetros principales seleccionados para el entrenamiento de SSD.

Hiperparámetro	Valor seleccionado	Nombre / Descripción
<i>variant</i>	$\{ssd300/ssd512\}$	Variante del modelo
<i>img_dim</i>	300/512	Tamaño imagen (300×300 / 512×512)
<i>epochs</i>	300	Épocas de entrenamiento
<i>opt</i>	SGD	Optimizador
<i>lr</i>	0.001	Tasa de aprendizaje inicial
<i>momentum</i>	0.9	Momento del optimizador
<i>weight_decay</i>	0.001	Regularización L_2
<i>gamma</i>	0.1	Factor de decaimiento de <i>lr</i>
<i>lr_steps</i>	[14000, 17000]	Iteraciones de ajuste de <i>lr</i>
<i>batch_size</i>	16	Tamaño de lote de entrenamiento
<i>max_iter</i>	20000	Iteraciones máximas de entrenamiento
<i>overlap_thresh</i>	0.5	Umbral IoU para <i>matching</i> positivo
<i>neg_pos_ratio</i>	3	Relación negativos:positivos
<i>conf_thresh</i>	0.01	Umbral mínimo de confianza
<i>nms_thresh</i>	0.45	Umbral IoU para NMS
<i>top_k</i>	200	Máximo de detecciones consideradas
<i>hue_delta</i>	18	Variación máxima de tono (HSV)
<i>saturation_range</i>	[0.5, 1.5]	Rango de saturación (HSV)
<i>brightness_delta</i>	32	Variación de brillo
<i>contrast_range</i>	[0.5, 1.5]	Rango de contraste
<i>random_mirror_prob</i>	0.5	Probabilidad de volteo horizontal

Tabla 6.1.3: Hiperparámetros principales seleccionados para el entrenamiento de DETR.

Hiperparámetro	Valor seleccionado	Nombre / Descripción
<i>variant</i>	$\{R50, R50-DC5, R101, R101-DC5\}$	Variante del modelo
<i>epochs</i>	300	Épocas totales de entrenamiento
<i>batch_size</i>	4	Tamaño de lote de entrenamiento
<i>lr</i>	0.0001	Tasa de aprendizaje base
<i>lr_backbone</i>	1.0e-05	Tasa de aprendizaje del extractor
<i>lr_drop_epochs</i>	[150, 250]	Épocas para decaimiento por saltos
<i>weight_decay</i>	0.0001	Regularización L_2 global
<i>hidden_dim</i>	256	Dimensión del espacio latente
<i>nheads</i>	8	Cantidad de cabezas de atención
<i>enc_layers</i> / <i>dec_layers</i>	6 / 6	Capas operativas de codificador y decodificador
<i>num_queries</i>	100	Ranuras de predicción
<i>set_cost_class</i>	1.0	Ponderador de clase en asignación
<i>set_cost_bbox</i>	5.0	Ponderador distancia L_1 en asignación
<i>set_cost_giou</i>	2.0	Ponderador GIoU en asignación
<i>bbox_loss_coef</i>	5.0	Peso final para pérdida L_1
<i>giou_loss_coef</i>	2.0	Peso final para pérdida GIoU

Tabla 6.1.4: Hiperparámetros principales seleccionados para el entrenamiento de DINO.

Hiperparámetro	Valor seleccionado	Nombre / Descripción
<i>variant</i>	$\{R50-4scale, R50-5scale, Swin_L\}$	Variante del modelo
<i>epochs</i>	300	Épocas totales de entrenamiento
<i>batch_size</i>	4 (R50-4scale) 2 (R50-5scale, Swin_L)	Tamaño de lote de entrenamiento
<i>lr</i>	0.0001	Tasa de aprendizaje base
<i>lr_backbone</i>	1.0e-05	Tasa de aprendizaje del extractor
<i>lr_drop_epochs</i>	[150, 250]	Épocas para decaimiento por saltos
<i>weight_decay</i>	0.0001	Regularización L_2 global
<i>hidden_dim</i>	256	Dimensión del espacio latente
<i>enc_layers</i> / <i>dec_layers</i>	6 / 6	Capas operativas de codificador y decodificador
<i>num_queries</i>	100	Consultas posicionales
<i>num_feature_levels</i>	4 ó 5	Niveles de características extraídas
<i>use_dn</i>	True	Entrenamiento contrastivo (CDN)
<i>dn_number</i>	50	Cantidad de consultas de ruido
<i>set_cost_class</i>	2.0	Ponderador de clase en asignación
<i>bbox_loss_coef</i>	5.0	Peso final para pérdida L_1
<i>giou_loss_coef</i>	2.0	Peso final para pérdida GIoU
<i>cls_loss_coef</i>	1.0	Peso final para Focal Loss